

Sum of Nth Powers of Unique Natural Numbers

Ek Complete Hinglish Guide Book

Chapter 1: Problem Ko Samajhna (Understanding the Problem)

Real-Life Story Se Shuru Karte Hain

Maan lo aapke paas ek bag hai. Aapko us bag mein kuch kitabein rakhni hain. Condition ye hai ki:

- Har kitaab ka weight perfect square hona chahiye (jaise 1 kg, 4 kg, 9 kg, 16 kg...)
- Total weight exactly 10 kg hona chahiye
- Har kitaab ka weight alag-alag hona chahiye (duplicate allowed nahi)

Ab aap sochiye - kaun kaun si kitabein chunenge?

Possible combinations:

- $1^2 + 3^2 = 1 + 9 = 10$ ✓ (1 aur 3 chuna)
- $1^2 + 2^2 + \dots$ (ye 10 se kam ya zyada ho jayega)

Yehi basic idea hai **"Sum of Nth Powers"** problem ka!

Problem Statement (Simple Words Mein)

Humein batana hai ki **kitne different ways** hain jisme hum **unique natural numbers** ki **Nth power** ko jod kar **exactly X** bana sakte hain.

Keywords Ko Samjhte Hain:

1. Sum of Nth Powers

“Sum” matlab jod. “Nth Power” matlab kisi number ko N baar multiply karna.

Examples:

- Sum of 2nd powers (squares): $1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$
- Sum of 3rd powers (cubes): $1^3 + 2^3 = 1 + 8 = 9$

2. Unique Natural Numbers

- **Natural Numbers:** 1, 2, 3, 4, 5, ... (0 include nahi hota generally, lekin yahan 1 se start)
- **Unique:** Har number ek baar hi use kar sakte hain. Agar 2^2 use kiya to dobara 2^2 use nahi kar sakte.

Iska matlab: $\{1^2, 1^2, 2^2\}$ ❌ GALAT (1 do baar aaya)

Iska matlab: $\{1^2, 2^2, 3^2\}$ ✓ SAHI

3. Target Number X

Yeh woh number hai jo humein exactly achieve karna hai. Jaise $X=10$ ka matlab humein powers ka sum exactly 10 banana hai.

4. Power N

Yeh batata hai ki kis power ka use karna hai.

- $N=2$: squares use karenge ($1^2, 2^2, 3^2, 4^2, \dots$)
- $N=3$: cubes use karenge ($1^3, 2^3, 3^3, \dots$)
- $N=4$: fourth powers ($1^4, 2^4, 3^4, \dots$)

5. Ways (Tareeke)

Different combinations kitne possible hain. Order matter nahi karta.

$\{1^2, 3^2\}$ aur $\{3^2, 1^2\}$ same way consider hoga.

Detailed Examples Se Samjhte Hain

Example 1: $X = 10$, $N = 2$

Hindi: Humein 10 banana hai squares ke sum se, unique numbers use karte hue.

Possible squares: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16$ ($16 > 10$, to 4 se aage nahi jayenge)

Check karte hain combinations:

- Sirf $1^2 = 1 \neq 10$ ✗
- Sirf $2^2 = 4 \neq 10$ ✗
- Sirf $3^2 = 9 \neq 10$ ✗
- $1^2 + 2^2 = 1+4 = 5 \neq 10$ ✗
- $1^2 + 3^2 = 1+9 = 10$ ✓ **MIL GAYA!** (Way 1)
- $2^2 + 3^2 = 4+9 = 13 \neq 10$ ✗
- $1^2 + 2^2 + 3^2 = 1+4+9 = 14 \neq 10$ ✗

Answer: 1 way ($1^2 + 3^2$)

Example 2: $X = 13$, $N = 2$

Squares: 1, 4, 9, 16(>13, stop)

Check karte hain:

- $1^2 + 2^2 + 3^2 = 1+4+9 = 14 > 13$ ✗
- $2^2 + 3^2 = 4+9 = 13$ ✓ **WAY 1**
- $1^2 + ? =$ sochiye... $1^2 + 2^2 = 5$, $1^2 + 3^2 = 10$, $1^2 + 4^2 = 17$... nahi ban raha

Kya $1^2 + 2^2 +$ kuch aur? $1+4=5$, $13-5=8$, kya 8 kisi ka perfect square hai? 2.8^2 ... nahi.

Answer: 1 way ($2^2 + 3^2$)

Example 3: X = 100, N = 2

Ye thoda bada example hai. Squares: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100

Possible ways:

- $10^2 = 100$ ✓ (Way 1)
- $6^2 + 8^2 = 36 + 64 = 100$ ✓ (Way 2)
- $1^2 + 3^2 + 4^2 + 5^2 + 7^2 = 1+9+16+25+49 = 100$ ✓ (Way 3)

Aur bhi combinations ho sakti hain! Isko manual count karna mushkil hai - isliye humein programming ki zaroorat hai.

Example 4: X = 100, N = 3

Cubes: $1^3=1$, $2^3=8$, $3^3=27$, $4^3=64$, $5^3=125(>100, \text{stop})$

Check karein:

- $4^3 = 64 \neq 100$ ✗
- $3^3 + 4^3 = 27+64 = 91 \neq 100$ ✗
- $2^3 + 3^3 + 4^3 = 8+27+64 = 99 \neq 100$ ✗
- $1^3 + 2^3 + 3^3 + 4^3 = 1+8+27+64 = 100$ ✓ **WAY 1**

Answer: 1 way ($1^3 + 2^3 + 3^3 + 4^3$)

Visual Representation

Maan lo aapke paas numbers hain: 1, 2, 3, 4, ...

Har number ke saath ek choice hai: **lena hai ya nahi lena hai**

Number 1: [Lena] ya [Nahi Lena]

Number 2: [Lena] ya [Nahi Lena]

Number 3: [Lena] ya [Nahi Lena]

...

Har decision ke baad aage badhte hain. Isse **Decision Tree** kehte hain.

Is problem mein:

- **Lena** = current number ki power ko sum mein add karo
- **Nahi Lena** = skip karo, agle number par jao

Problem Ki Key Points (Summary)

1. **Input:** Do numbers - X (target sum) aur N (power)
2. **Output:** Kitne ways hain X ko unique natural numbers ki Nth powers ke sum ke roop mein likhne ke
3. **Rules:**
 - Har number sirf ek baar use kar sakte hain (unique)
 - Powers ko jodna hai
 - Sum exactly X ke barabar hona chahiye
 - Natural numbers 1 se start hote hain (kabhi kabhi 0 se, lekin yahan 1 se)
 - Order matter nahi karta

Exercise (Aap Try Karein)

Pen-paper le kar ye solve karein:

1. X = 5, N = 2 (Hint: squares 1,4 use karo)
2. X = 25, N = 2
3. X = 9, N = 3
4. X = 12, N = 2

Answers check karne ke baad aage badhenge.

Is chapter mein humne problem ko real-life example se samjha. Aage badhne se pehle ensure karein ki aap "unique natural numbers", "Nth power", aur "ways" ka concept clear hai.

Chapter 2: Powers Ko Samajhna (Understanding Powers)

Power Kya Hai? (What is Power?)

Jab hum kisi number ko baar-baar multiply karte hain, use **power** kehte hain.

Real-Life Analogy: Paper Fold Karna

Sochiye aapke paas ek paper hai. Usse fold karne par:

- 1 fold: 2 layers ($2^1 = 2$)
- 2 folds: 4 layers ($2^2 = 2 \times 2 = 4$)
- 3 folds: 8 layers ($2^3 = 2 \times 2 \times 2 = 8$)
- N folds: 2^N layers

Har baar aap previous result ko 2 se multiply kar rahe ho. Yehi power ka concept hai.

Mathematical Definition

a^n ka matlab: a ko n baar multiply karo

$$a^n = a \times a \times a \times \dots \times a \text{ (n times)}$$

Examples:

- $2^2 = 2 \times 2 = 4$ (2 ka square)
- $2^3 = 2 \times 2 \times 2 = 8$ (2 ka cube)
- $3^2 = 3 \times 3 = 9$
- $3^3 = 3 \times 3 \times 3 = 27$
- $5^4 = 5 \times 5 \times 5 \times 5 = 625$

Terminology (Naam Yaad Rakho)

- **Base (a):** Jis number ki power nikalni hai
- **Exponent/Power (n):** Kitni baar multiply karna hai
- **Result (a^n):** Final value

Jaise 2^3 mein:

- Base = 2
- Exponent = 3
- Result = 8

Different Powers Ki Samajh

Square (Power 2 - N=2)

Kisi number ka square = number $\times$ number

$$1^2 = 1 \times 1 = 1$$

$$2^2 = 2 \times 2 = 4$$

$$3^2 = 3 \times 3 = 9$$

$$4^2 = 4 \times 4 = 16$$

$$5^2 = 5 \times 5 = 25$$

$$6^2 = 36$$

$$7^2 = 49$$

$$8^2 = 64$$

$$9^2 = 81$$

$$10^2 = 100$$

Square numbers: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, ...

Visual: Agar aapke paas 3×3 ka grid hai, to usme total 9 cells hain. Isliye $3^2 = 9$.

Cube (Power 3 - N=3)

Number $\times$ number $\times$ number

$$1^3 = 1 \times 1 \times 1 = 1$$

$$2^3 = 2 \times 2 \times 2 = 8$$

$$3^3 = 3 \times 3 \times 3 = 27$$

$$4^3 = 4 \times 4 \times 4 = 64$$

$$5^3 = 5 \times 5 \times 5 = 125$$

Cube numbers: 1, 8, 27, 64, 125, 216, 343, 512, 729, 1000

Visual: Agar aapke paas $3 \times 3 \times 3$ ka cube hai, usme 27 chhote cubes hain. Isliye $3^3 = 27$.

Higher Powers

$$N=4: 1^4=1, 2^4=16, 3^4=81, 4^4=256$$

$$N=5: 1^5=1, 2^5=32, 3^5=243$$

Jaise N badhta hai, values bahut tezi se badhti hain. Ye observation important hai - isi wajah se humara search space chhota rehta hai.

Python Mein Powers Calculate Karna

Python mein power nikalne ke **do popular tarike** hain:

Method 1: ** Operator

```
# ** operator use karna
result = 2 ** 3    # 2³ = 8
print(result)      # Output: 8

square = 5 ** 2    # 5² = 25
cube = 5 ** 3      # 5³ = 125

# Dynamic power
base = 3
power = 4
result = base ** power    # 3⁴ = 81
print(result)
```


Kab use karein? Jab aapko integer powers chahiye aur simple code likhna hai. `**` operator seedha-saada hai aur fast hai.

Method 2: `math.pow()` Function

```
import math

result = math.pow(2, 3)    # 23 = 8.0
print(result)              # Output: 8.0 (float!)

square = math.pow(5, 2)    # 25.0
print(square)
```

Dono Mein Difference (Important!)

Feature	<code>**</code> Operator	<code>math.pow()</code>
Return Type	Integer (agar dono integer)	Hamesha Float
Speed	Thoda fast	Thoda slow
Import	Zaroorat nahi	import math karna padega
Use case	Simple integer powers	Scientific calculations

Hamari problem ke liye konsa use karein?

```
# Ye better hai:
value = i ** n    # Integer return karega

# Ye avoid karein:
value = math.pow(i, n)    # Float return karega, comparison mein problem
```

Kyun? Kyunki hum integer comparisons kar rahe hain:

```
if i**n == x:    # Integer comparison - safe
    ...
```

```
if math.pow(i, n) == x:    # Float comparison - risky (0.1 + 0.2 != 0.3 type is
    ...
```

Built-in pow() Function (Teesra Option)

```
result = pow(2, 3)    # 8
result = pow(2, 3, 10) # 23 % 10 = 8 (modulo ke saath)
```

Recommendation: Hamari problem mein `**` operator use karein. Simple, fast, aur integer return karta hai.

Powers Ki Properties Jo Humein Kaam Aayengi

Property 1: Monotonically Increasing

Jaise-jaise base badhta hai, power ki value bhi badhti hai.

Agar $a < b$, to $a^n < b^n$ (for positive numbers)

```
12 = 1
22 = 4    (4 > 1) ✓
32 = 9    (9 > 4) ✓
42 = 16   (16 > 9) ✓
```

Is property ki wajah se hum ek point ke baad search rok sakte hain.

Property 2: Power Ki Value Bahut Tezi Se Badhti Hai

```
For N=2: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100
For N=3: 1, 8, 27, 64, 125, 216, 343, 512, 729, 1000
For N=4: 1, 16, 81, 256, 625, 1296
For N=5: 1, 32, 243, 1024
```

Notice karo: N jitna bada, values utni tezi se badhti hain. Iska matlab humara recursion tree utna chhota hoga.

Practice Examples (Python Mein)

Example 1: Sabhi Squares Print Karo Jo X Se Chhote Ya Barabar Hon

```
def print_squares_upto(x):  
    """X se chhote ya barabar sabhi perfect squares print karo"""  
    i = 1  
    while i*i <= x:    # i*i is same as i**2  
        print(f"{i}^2 = {i*i}")  
        i += 1  
  
# Test  
print("Squares up to 20:")  
print_squares_upto(20)
```

Output:

```
1^2 = 1  
2^2 = 4  
3^2 = 9  
4^2 = 16
```

Example 2: Check If Number Is Perfect Power

```
def is_perfect_power(num, n):  
    """Check if num is nth power of some integer"""  
    # Brute force approach  
    i = 1  
    while i**n <= num:  
        if i**n == num:  
            return True, i  
        i += 1  
    return False, None  
  
# Test  
print(is_perfect_power(64, 2))    # True, 8 (8^2=64)  
print(is_perfect_power(64, 3))    # True, 4 (4^3=64)  
print(is_perfect_power(10, 2))    # False, None
```

Example 3: Generate Powers List

```
def generate_powers(x, n):  
    """List of nth powers <= x"""  
    powers = []  
    i = 1  
    while True:  
        power = i ** n  
        if power > x:  
            break  
        powers.append(power)  
        i += 1  
    return powers  
  
# Test  
print(generate_powers(50, 2))    # [1, 4, 9, 16, 25, 36, 49]  
print(generate_powers(50, 3))    # [1, 8, 27]  
print(generate_powers(100, 3))   # [1, 8, 27, 64]
```

Common Mistakes With Powers

Mistake 1: Power aur Multiplication Ko Confuse Karna

```
# Galat  
2^3 = ? # Python mein ^ XOR operator hai, power nahi!  
  
# Sahi  
2 ** 3 = 8
```

Mistake 2: Zero Power

```
5 ** 0 = 1    # Koi bhi number ki power 0 = 1  
10 ** 0 = 1  
100 ** 0 = 1
```

Mistake 3: Negative Numbers

```
(-2) ** 2 = 4      # Negative × Negative = Positive
(-2) ** 3 = -8     # Odd power: negative hi rehta hai
```

Hamari problem mein sirf positive numbers se kaam hai, to ye tension nahi leni.

Problem-Specific Power Understanding

Hamari problem mein:

- Hum 1 se shuru karte hain (1^n , 2^n , 3^n , ...)
- Jab kisi number ki power X se badi ho jaye, hum wahan ruk jaate hain
- Sirf unique numbers use kar sakte hain, to ek number ki power ek hi baar use hogi

```
# Maximum number jiski power humein consider karni hai
max_num = int(X ** (1/N)) # N-th root of X
# Ya loop se:
i = 1
while i**N <= X:
    # i**N ko consider karo
    i += 1
```

Chapter Summary

- Power ka matlab: number ko baar-baar multiply karna
- `**` operator simple aur efficient hai integers ke liye
- Powers monotonically badhti hain - isliye search space limited hai
- Higher N ke liye search space aur chhota ho jata hai

Aage badhne se pehle: Ensure karein ki aap comfortably powers calculate kar sakte hain aur samajhte hain ki kyun hum sirf limited numbers consider karte hain.

Chapter 3: Programmer Ki Tarah Sochna (Thinking Like a Programmer)

Programming Sirf Code Likhna Nahi Hai

Programming = **Problem Solving + Code Writing**

Pehle dimaag mein solution clear hona chahiye, phir code likhna easy ho jata hai.

Real-Life Analogy: Ghar Banana

Jab ghar banta hai:

1. Pehle **architect sochta hai** - kitne rooms, kahan kitchen, kahan bathroom
2. Phir **blueprint** (naksha) banata hai
3. Phir **construction** shuru hoti hai

Aise hi programming mein:

1. Pehle **problem samjho**
2. Phir **solution ka structure socho** (algorithm design karo)
3. Phir **code likho**

Bina soche code likhna = bina naksha ke ghar banana = disaster!

Step-by-Step Thinking Process

Step 1: Problem Ko Break Karo (Divide and Conquer)

Badi problem ko chhoti-chhoti problems mein todo.

Hamari Problem: Kitne ways hain X ko unique Nth powers ke sum se banane ke?

Break Down:

1. Sabse pehle - kaun kaun si powers available hain?
2. Har power ke liye decision - leni hai ya nahi?
3. Sum track karo - exactly X achieve karna hai
4. Unique numbers maintain karo

Step 2: Choices Identify Karo

Har decision point par kya-kya choices hain?

Maan lo current number = 1, N = 2:

- **Choice 1:** $1^2 = 1$ ko sum mein add karo, phir aage badho
- **Choice 2:** 1^2 ko skip karo, seedha aage badho

Yehi choices har number ke liye repeat hongii.

Step 3: Constraints Identify Karo (Rukna Kab Hai?)

Kab rukna hai decisions lena?

- **Success:** Sum exactly X ke barabar ho jaye → Ek way mil gaya!
- **Failure 1:** Sum X se zyada ho jaye → Ye path galat hai, wapas jao
- **Failure 2:** Current number ki power X se zyada ho → Aage ke sab numbers bhi zyada denge, wapas jao
- **Failure 3:** Saare numbers check ho gaye, sum X nahi bana → Koi way nahi mila

Manual Simulation (Pen-Paper Exercise)

Problem: X = 10, N = 2

Available Powers (jo 10 se chhoti ya barabar hain):

$$1^2 = 1$$

$$2^2 = 4$$

$$3^2 = 9$$

($4^2 = 16 > 10$, isliye 4 aur usse aage nahi lenge)

Ab Systematic Exploration

Starting: sum = 0, number = 1

Number 1 (power=1):

```

├─ Include 1: sum=1, next number=2
|   ├─ Include 2: sum=1+4=5, next=3
|   |   ├─ Include 3: sum=5+9=14 > 10 X (over, backtrack)
|   |   └─ Skip 3: sum=5, next=4 ( $4^2=16 >$  remaining 5, stop)
|   └─ Skip 2: sum=1, next=3
|       └─ Include 3: sum=1+9=10 ✓ FOUND!
|           └─ Skip 3: sum=1, next=4 (stop)
└─ Skip 1: sum=0, next=2
    ├─ Include 2: sum=4, next=3
    |   ├─ Include 3: sum=4+9=13 > 10 X
    |   └─ Skip 3: sum=4 (stop)
    └─ Skip 2: sum=0, next=3
        └─ Include 3: sum=9, next=4 (stop)
            └─ Skip 3: sum=0 (stop)
  
```

Way found: $1^2 + 3^2 = 10$

Is systematic exploration ko **Recursion Tree** ya **Decision Tree** kehte hain.

Programmer Ki Soch Ka Template

Har problem ke liye ye pattern follow karo:

1. State Define Karo

“State” matlab - kahan ho abhi? Kya-kya information hai?

Hamari problem mein state:

- `remaining_sum` : X mein se kitna baki hai (ya current sum kitna hai)
- `current_num` : Kaun sa number consider kar rahe hain
- `power_values` : N decide ho chuka hai, change nahi hoga

2. Choices Per State

Har state se kahan-kahan ja sakte hain?

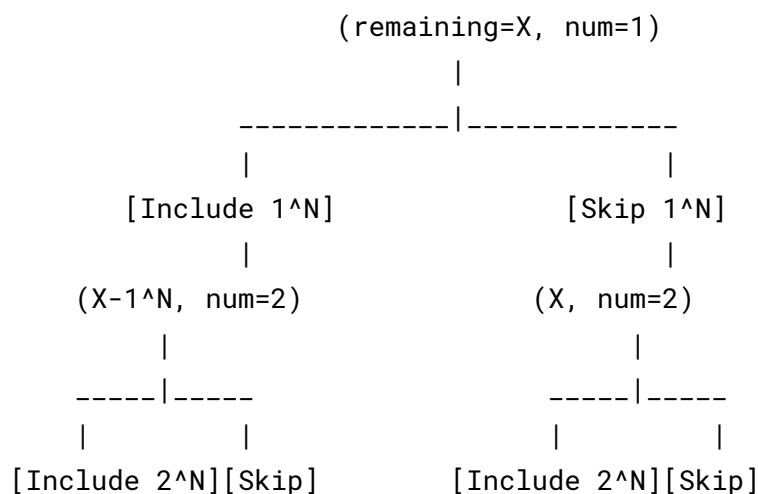
- `State(remaining, num)` se:
 - Agar num^N include karein: `State(remaining - numN, num+1)`
 - Agar skip karein: `State(remaining, num+1)`

3. Base Cases (Rukne Ki Conditions)

Kab aur kaise rukna hai?

- `remaining == 0` : Ek valid way mil gaya $\rightarrow$ return 1
- $\text{num}^N > \text{remaining}$: Aage nahi ja sakte $\rightarrow$ return 0
- `remaining < 0` : Already zyada ho gaya $\rightarrow$ return 0

Visual Representation: Decision Tree Sochna



Yahi tree grow hota hai jab tak ya to sum 0 ho jaye (success) ya num^N remaining se bada ho jaye (failure).

Transition From Thinking To Code

Sochne ke baad code likhne ka structure:

```
def solve(remaining, n, current_num):  
    # Base cases (rukna kab hai)  
    if remaining == 0:    # Target achieve!  
        return 1  
    if current_num**n > remaining: # Aage nahi ja sakte  
        return 0  
  
    # Recursive cases (choices explore karo)  
    # Choice 1: Include current number  
    include = solve(remaining - current_num**n, n, current_num + 1)  
  
    # Choice 2: Skip current number  
    skip = solve(remaining, n, current_num + 1)  
  
    # Total ways = include ke ways + skip ke ways  
    return include + skip
```

Exercise: Sochne Ki Practice

In problems ko bina code likhe, sirf pen-paper se solve karo:

1. **X = 5, N = 2**: Kitne ways hain? Tree draw karo.
2. **X = 13, N = 2**: Kitne ways hain? Tree draw karo.
3. **X = 9, N = 3**: Kitne ways hain? Tree draw karo.

Hint har problem ke liye:

1. Pehle available powers list karo
2. Phir systematically har combination try karo

3. Valid combinations count karo

Important Thinking Habits

Habit 1: Small Examples Se Shuru Karo

Bade X, N ke liye seedha code mat likho. Pehle chhote examples manually solve karo.

Habit 2: Edge Cases Socho

- $X = 1, N = 2: 1^2 = 1 \rightarrow 1$ way
- $X = 2, N = 2: 1^2 = 1$, need 1 more lekin next $2^2=4 > 2 \rightarrow 0$ ways
- $X = 0, N = \text{anything}$: Kya karna hai? (Hint: empty sum = 0)

Habit 3: Pattern Pehchano

Notice karo ki problem structure same rehta hai, sirf X aur N change hote hain. Isliye ek general solution possible hai.

Chapter Summary

- **Socho pehle, code baad mein**
- Problem ko chhote pieces mein break karo
- Choices aur constraints clearly identify karo
- Pen-paper se manually solve karke logic samjho
- Phir code mein translate karo

Key Takeaway: Programmer ka dimaag algorithm sochta hai, syntax baad ki baat hai.

Chapter 4: Simple Loops Kyun Kaam Nahi Karenge

Introduction: Loops Ki Limitations

Bahut saare beginners sochte hain - "Loops se sab kuch ho jata hai!" Lekin aisa nahi hai. Kuch problems aisi hain jahan loops fail ho jate hain ya bahut complicated ho jate hain.

Hamari problem aisi hi hai. Samjhte hain kyun.

Approach 1: Simple For Loop (Fail)

Koshish: Ek variable sum karte jao

```
def power_sum_loop1(x, n):  
    total = 0  
    ways = 0  
    for i in range(1, int(x**(1/n)) + 1):  
        total += i**n  
        if total == x:  
            ways += 1  
    return ways
```

Kyun fail? Ye sirf consecutive numbers ka sum check karega ($1^2 + 2^2 + 3^2 \dots$). Ye check nahi karega ki kuch numbers skip karke bhi sum ban sakta hai.

Example: $X = 10$, $N = 2$

- Loop: $1^2=1$, $1^2+2^2=5$, $1^2+2^2+3^2=14$ (10 kabhi aaya hi nahi)
- Actual answer: $1^2 + 3^2 = 10$ (2 skip karna pada)

Loop ne skip karna consider hi nahi kiya!

Approach 2: Nested Loops (Partial Solution)

Socho agar hum jante hain ki maximum 3 numbers tak use honge:

```
# Assume maximum 3 numbers needed
def power_sum_nested(x, n):
    ways = 0
    limit = int(x**(1/n)) + 1

    for i in range(1, limit):
        for j in range(i+1, limit):
            for k in range(j+1, limit):
                if i**n + j**n + k**n == x:
                    ways += 1
                print(f"Found: {i}**{n} + {j}**{n} + {k}**{n}")

    return ways
```

Kyun fail?

1. **Kitne nested loops lagaye?** Pata nahi kitne numbers lagenge. Kabhi 2, kabhi 5, kabhi 10. Fixed nested loops nahi likh sakte.
2. **Readability:** 10 nested loops likhna pagalpan hai.
3. **Maintainability:** Agar kuch change karna ho to?

Approach 3: While Loop With Backtracking Logic (Complicated)

```
def power_sum_while(x, n):
    ways = 0
    current_nums = [] # Stack banakar try karte hain
    current_sum = 0
    i = 1

    while i**n <= x or len(current_nums) > 0:
        # Bahut complex logic likhna padega
        # Kab push karna, kab pop karna, kab i increment karna
        # Ye basically recursion reinvent kar raha hai
```

pass
return ways

Ye possible hai lekin:

- Code bahut complex ho jayega
- Bug-prone hoga
- Recursion isko naturally handle karta hai

Fundamental Problem: Unknown Depth

Hamari problem ka fundamental challenge ye hai:

Humein nahi pata ki kitne numbers ki zaroorat padegi.

Kabhi 2 numbers se sum ban jayega:

- $1^2 + 3^2 = 10$

Kabhi 4 numbers lagenge:

- $1^3 + 2^3 + 3^3 + 4^3 = 100$

Kabhi 7 numbers lagenge:

- $1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 = 140$

Loops ke liye humein pehle se pata hona chahiye ki kitni depth tak jana hai. Recursion is problem ko naturally solve karta hai.

Visualization: Loop vs Recursion

Loop ka structure (Fixed depth):

```
for i1:  
    for i2:
```

```
for i3:  
    # Fixed 3-level deep
```

Recursion ka structure (Variable depth):

```
check(num=1):  
    check(num=2):  
        check(num=3):  
            check(num=4): # Jitni zaroorat utni depth  
            ...
```

Recursion automatically manage karta hai ki kitni depth mein jana hai.

Comparison Table

Feature	Loops	Recursion
Depth known?	Required	Not required
Code complexity	High for unknown depth	Low
Readability	Poor for nested	Clear logic
Bug-prone	Yes (complex logic)	Less (simple logic)
Natural fit	For fixed iterations	For branching choices

When Loops Work (Counter Example)

Loops tab best hain jab:

- Fixed number of iterations ho
- Linear ya simple nested structure ho
- State maintain karna simple ho

Example jahan loop perfect hai:

```
# Sum of first N squares (fixed range, known iterations)
def sum_first_n_squares(n):
    total = 0
    for i in range(1, n+1):
        total += i**2
    return total
```

Transition To Recursion

Ab samajh aa gaya hoga ki humein recursion ki zaroorat kyun hai:

1. **Unknown depth** of choices
2. **Branching decisions** (include/skip)
3. **Need to backtrack** (try different combinations)
4. **Clean, readable code**

Agle chapter mein hum recursion ko bilkul basic se seekhenge, bilkul aise jaise pehli baar seekh rahe hon.

Chapter Summary

- Simple loops fixed number of elements consider karte hain, combinations nahi
- Nested loops tabhi kaam karte hain jab depth pehle se pata ho
- Hamari problem mein depth unknown hai - kitne bhi numbers lag sakte hain
- Recursion is problem ke liye natural solution hai

Remember: Right tool for the right job. Loops galat nahi hain, bas ye problem loops ke liye suitable nahi hai.

Chapter 5: Recursion - Bilkul Basic Se (Introduction to Recursion)

Recursion Kya Hai? (Real-Life Analogy)

Analogy 1: Russian Dolls (Matryoshka)

Sochiye aapke paas Russian dolls hain. Ek badi doll ke andar chhoti doll, uske andar aur chhoti... aise kai layers.

Jab aap dolls kholte hain:

1. Sabse badi doll kholo
2. Andar se chhoti doll nikalo
3. Usse kholo
4. Aur chhoti niklegi
5. ...aise chalta rehta hai
6. Jab last doll (sabse chhoti) aati hai - woh khulti nahi, solid hai

Yehi recursion hai! Har doll same structure hai (ek doll jo khulti hai), lekin chhoti hoti jaati hai. End mein ek doll aati hai jo “base case” hai.

Analogy 2: Countdown

Socho aap 5 se 1 tak countdown kar rahe ho:

- 5... (ruk kar socho: ab 4 se countdown karo)
- 4... (ruk kar socho: ab 3 se countdown karo)
- 3... (ruk kar socho: ab 2 se countdown karo)
- 2... (ruk kar socho: ab 1 se countdown karo)
- 1... (ye last hai, ruk jao)
- 2 done
- 3 done

4 done

5 done

Har step par same kaam ho raha hai: number bolo, phir chhote number ke liye same process repeat karo. Jab 1 aaye, ruk jao.

Recursion Ki Simple Definition

Recursion: Jab ek function khud ko call kare.

Ya aur simply: Ek aina jo aine ke saamne rakha ho - infinite reflections, lekin ek point par rukna zaroori hai.

Technical Definition:

Recursion ek programming technique hai jisme ek function apne aap ko chhote input ke saath call karta hai jab tak ek stopping condition (base case) na reach ho jaye.

Pehla Recursive Function (5 Minutes Mein)

Dekhte hain sabse simple recursive function:

```
def countdown(n):  
    """  
    Countdown karo n se 1 tak  
    """  
    # Base Case: Rukne ki condition  
    if n == 0:  
        print("Go!")  
        return  
  
    # Recursive Case  
    print(n)  
    countdown(n - 1)    # Khud ko chhote input se call  
    print(f"Returning from n={n}")
```

```
# Call the function  
countdown(5)
```

Output:

```
5  
4  
3  
2  
1  
Go!  
Returning from n=1  
Returning from n=2  
Returning from n=3  
Returning from n=4  
Returning from n=5
```

Kya Ho Raha Hai? (Line By Line)

1. countdown(5) call hua

- n=5, base case nahi ($5 \neq 0$)
- Print "5"
- countdown(4) call karo

2. countdown(4) ab execute ho raha hai

- n=4, base case nahi
- Print "4"
- countdown(3) call karo

3. countdown(3) :

- n=3
- Print "3"
- countdown(2) call karo

4. countdown(2) :

- $n=2$
- Print "2"
- `countdown(1)` call karo

5. `countdown(1)` :

- $n=1$
- Print "1"
- `countdown(0)` call karo

6. `countdown(0)` :

- **BASE CASE!** $n=0$
- Print "Go!"
- Return (function khatam, neech wale ko control wapas)

7. Control `countdown(1)` mein wapas:

- Print "Returning from $n=1$ "
- Function khatam

8. Control `countdown(2)` mein wapas:

- Print "Returning from $n=2$ "
- Function khatam

... aur aise hi 5 tak wapas aata hai.

Recursion Ke Do Zaroori Parts

Part 1: Base Case (Rukna Kab Hai?)

Ye sabse important part hai! Bina base case ke recursion infinite ho jayegi.

```
if n == 0:    # Base case
    return    # Stop condition
```

Real-life mein: Jab Russian doll ki last, solid doll mile - ruk jao, usse mat kholo.

Without base case:

```
def infinite_countdown(n):
    print(n)
    infinite_countdown(n-1) # Kabhi nahi rukega!
```

Part 2: Recursive Case (Problem Chhoti Karna)

Har recursive call mein problem thodi chhoti honi chahiye.

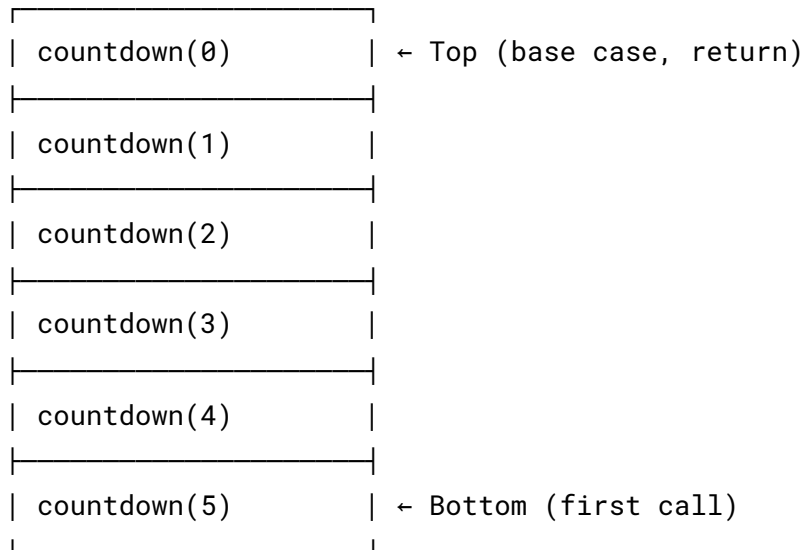
```
countdown(n-1) # Problem size kam ho raha hai (n → n-1)
```

Agar problem size kam nahi hua to base case kabhi reach nahi hoga.

Recursion Ka Execution Flow (Call Stack)

Jab function khud ko call karta hai, har call memory mein stack ki tarah store hoti hai.

Stack (neeche se upar):



Flow:

1. Calls stack mein add hoti jaati hain (push)
2. Base case tak jaate hain
3. Phir return hote hain (pop)
4. Har return ke saath stack se ek call remove hoti hai

Recursion Jo Value Return Kare (Important!)

Ab tak ka example sirf print kar raha tha. Lekin recursion aksar value return karta hai:

```
def sum_till_n(n):  
    """  
    1 + 2 + 3 + ... + n calculate karo recursively  
    """  
  
    # Base case  
    if n == 1:  
        return 1  
  
    # Recursive case  
    smaller_sum = sum_till_n(n - 1)    # Chhote problem ka answer  
    return n + smaller_sum            # Usme current number add karo  
  
# Usage  
print(sum_till_n(5))    # 15 (1+2+3+4+5)
```

Execution:

```
sum_till_n(5)  
= 5 + sum_till_n(4)  
= 5 + (4 + sum_till_n(3))  
= 5 + (4 + (3 + sum_till_n(2)))  
= 5 + (4 + (3 + (2 + sum_till_n(1))))  
= 5 + (4 + (3 + (2 + 1)))  
= 5 + (4 + (3 + 3))  
= 5 + (4 + 6)  
= 5 + 10  
= 15
```

Recursion Ke Fayde (For Our Problem)

1. **Code simple aur readable:** Include/skip ka logic clearly dikhta hai
2. **Unknown depth handle:** Automatically jitni depth chahiye utni le leta hai
3. **Backtracking natural:** Return karte waqt automatically previous state mein aate hain
4. **Every combination explore:** Tree structure naturally sab kuch cover karta hai

Recursion Ke Nuksan (Awareness)

1. **Memory:** Har call stack mein store hoti hai
2. **Speed:** Function calls ka overhead hota hai
3. **Stack overflow:** Bahut deep recursion stack overflow de sakti hai

Lekin hamari problem ke liye depth limited hai (kyunki powers jaldi badhti hain), isliye safe hai.

Important Rule: Recursion Design Karte Waqt

Rule 1: Maan Lo Recursion Kaam Karega

Jab `sum_till_n(n-1)` likhte hain, to maan lo ki ye sahi answer la dega. "How" ki tension mat lo.

Isse **Leap of Faith** kehte hain. Jaise induction hypothesis maths mein.

Rule 2: Base Case Perfect Rakho

Base case hi recursion ko infinite hone se bachata hai. Isko thoroughly test karo.

Rule 3: Problem Size Definitely Kam Ho

Har call mein ensure karo ki input base case ki taraf move kar raha hai.

Hamari Problem Ke Liye Recursion Structure

Ab samjhte hain ki hamari problem mein recursion kaise lagega:

```
def ways(x, n, current_num):  
    # BASE CASES  
    if x == 0:          # Target achieve kar liya  
        return 1  
    if current_num**n > x:  # Aage ki sab values badi hongi  
        return 0  
  
    # RECURSIVE CASES (Do choices)  
    # Choice 1: Current number include karo  
    include = ways(x - current_num**n, n, current_num + 1)  
  
    # Choice 2: Current number skip karo  
    skip = ways(x, n, current_num + 1)  
  
    # Total ways  
    return include + skip
```

Is structure ko detail mein aage ke chapters mein samjhenge.

Practice: Simple Recursion Problems

Problem 1: Factorial

```
def factorial(n):  
    """  
    n! = n × (n-1) × (n-2) × ... × 1  
    Base case: 0! = 1, 1! = 1  
    """  
    if n <= 1:      # Base case  
        return 1  
    return n * factorial(n - 1)
```



```
# Test
print(factorial(5))    # 120
```

Problem 2: Print Numbers from 1 to N

```
def print_1_to_n(n):
    """1 se n tak print karo"""
    if n == 0:
        return
    print_1_to_n(n - 1)    # Pehle 1 to n-1 print karo
    print(n)              # Phir n print karo

print_1_to_n(5)          # 1 2 3 4 5
```

Problem 3: Sum of Digits

```
def sum_of_digits(n):
    """123 ka sum = 1+2+3 = 6"""
    if n == 0:
        return 0
    return (n % 10) + sum_of_digits(n // 10)

print(sum_of_digits(123))    # 6
```

Chapter Summary

- Recursion = Function khud ko call kare chhote input ke saath
- Do parts: Base case (stop) + Recursive case (continue with smaller problem)
- Call stack automatically manage hota hai
- Leap of Faith rakho - maan lo recursion kaam karega
- Hamari problem ke liye perfect fit hai

Ab aap recursion ki basic samajh rakhate hain. Agle chapter mein hum recursion tree banayenge!

Chapter 6: Recursion Tree (Decision Tree)

Tree Kya Hai? (Computer Science Mein)

Tree ek structure hai jisme:

- Ek **root** (starting point) hota hai
- Root se **branches** (shaakhayein) nikalti hain
- Har branch ke end mein ya to **leaf** hota hai ya aur branches

Real-life mein jaise:

- Family tree (dada se shuru, phir unke bacche, phir unke bacche...)
- Folder structure (main folder, uske andar folders, aur andar...)
- Decision making (kya karein? option A, option B...)

Recursion Tree (Decision Tree) Kya Hai?

Recursion tree ek visual representation hai jo dikhata hai ki recursive function kaise different choices explore karta hai.

Hamari problem mein:

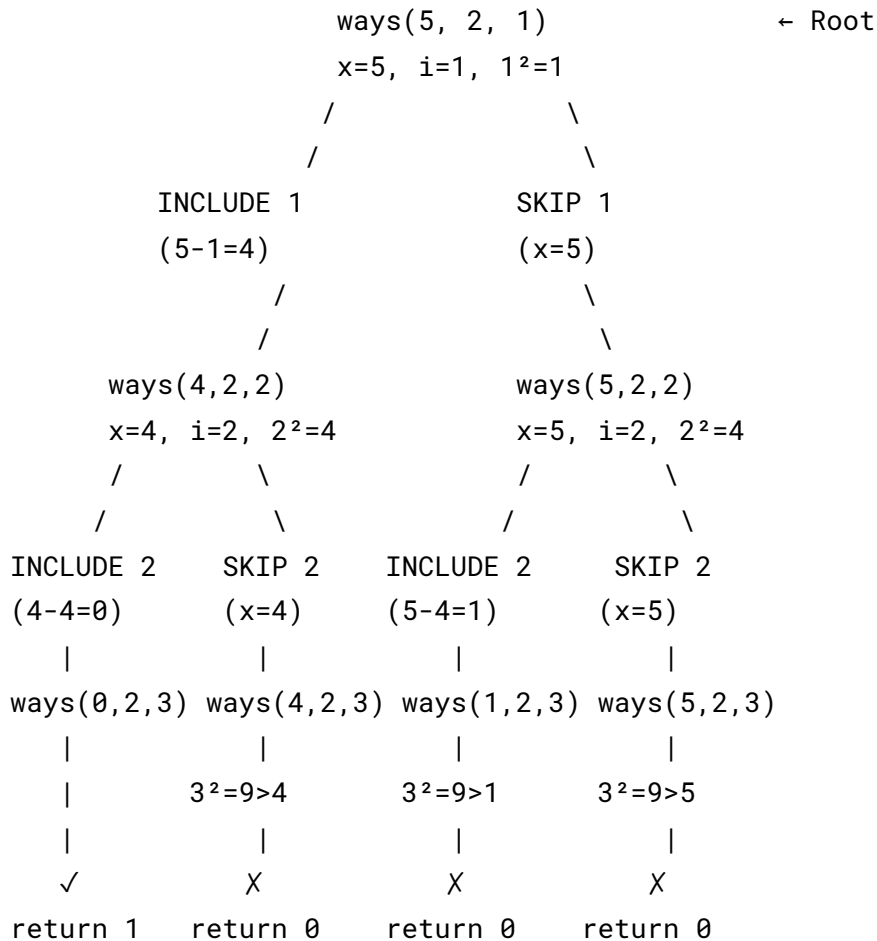
- Har **node** ek function call represent karta hai: `ways(x, n, i)`
- Har **branch** ek choice hai: "include" ya "skip"
- Har **leaf** ek base case hai: ya to success (return 1) ya failure (return 0)

Simple Example Se Tree Samjho

Problem: **X = 5, N = 2**

Powers available: $1^2=1$, $2^2=4$, $3^2=9$ ($9>5$, to 3 consider nahi karenge)

Tree Drawing



Tree Ko Padhna

1. **Root se start:** `ways(5, 2, 1)` - Humein 5 banana hai squares se, 1 se shuru
2. **Do choices:** 1 ko include karein ya skip karein
3. **Left branch (Include 1):** `ways(4, 2, 2)` - Ab 4 banana hai, 2 se shuru
4. **Wahan bhi do choices:** $2^2=4$ include karo ya skip
 - Include 2: `ways(0, 2, 3)` - **SUCCESS!** sum=0, return 1
 - Skip 2: `ways(4, 2, 3)` - $3^2=9>4$, aage nahi ja sakte, return 0
5. **Right branch (Skip 1):** `ways(5, 2, 2)` - 5 banana hai, 2 se shuru
 - Include 2: `ways(1, 2, 3)` - $3^2=9>1$, return 0
 - Skip 2: `ways(5, 2, 3)` - $3^2=9>5$, return 0

Result: 1 way ($1^2+2^2=1+4=5$)

Simple tree draw karne ka tarika:

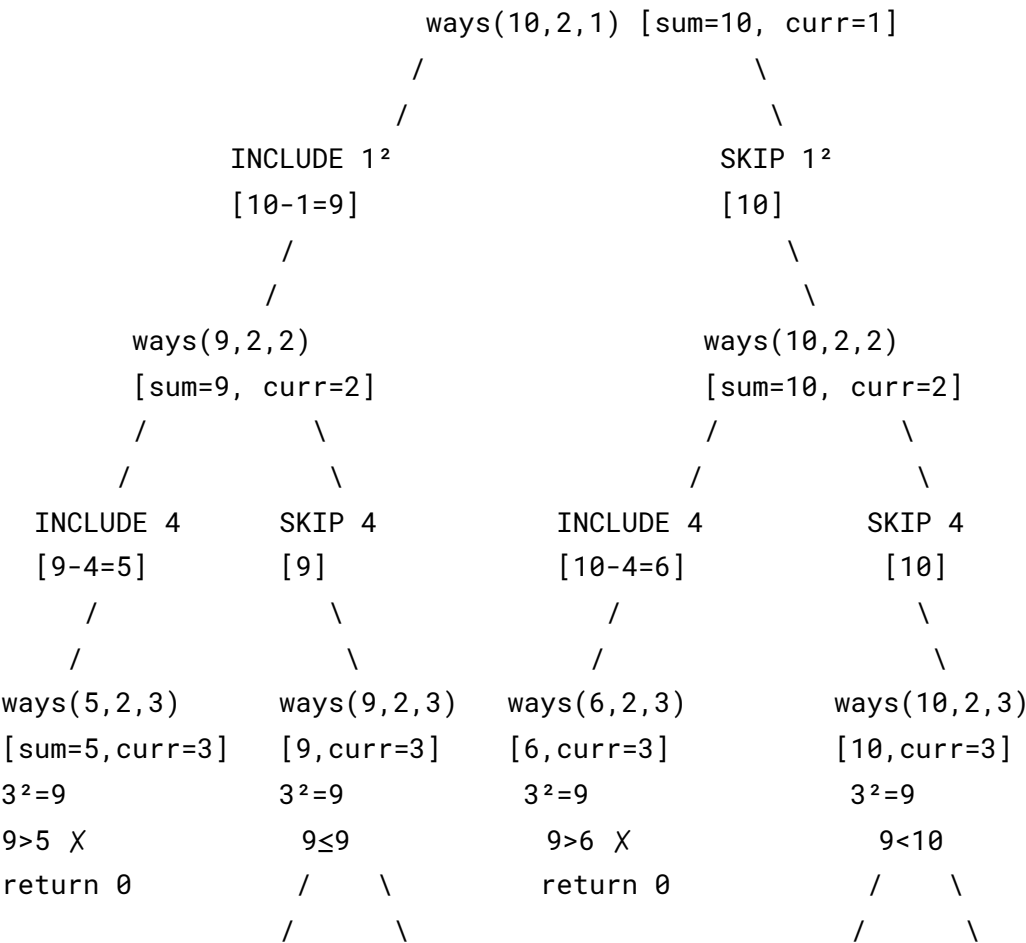
```
Node(value)
├─ Left Child (Include)
└─ Right Child (Skip)
```

Yahan:

-  matlab left branch
-  matlab last branch (right)

Detailed Example: $X=10$, $N=2$

Available: $1^2=1$, $2^2=4$, $3^2=9$



INCLUDE 9	SKIP 9	INCLUDE 9	SKIP 9
[9-9=0]	[9]	[10-9=1]	[10]
ways(0,2,4)	ways(9,2,4)	ways(1,2,4)	ways(10,2,4)
	4 ² =16>9		4 ² =16>10
✓	X	X	X
return 1	return 0	return 0	return 0

Ways Found:

1. Path: Include 1 → Skip 2 → Include 3 = $1^2 + 3^2 = 1 + 9 = 10$ ✓
2. Aur koi? Tree check karo... nahi. Total = 1 way.

Important Tree Concepts

1. Tree Ki Depth

Depth = root se leaf tak ki maximum length.

Is example mein depth = 4 (root se include-include... ya skip-skip...)

Tree depth limited hai kyunki:

- Har level par i badhta hai
- Kabhi na kabhi $i^2 > x$ ho jata hai
- Us point par branch ruk jaati hai

2. Branching Factor

Har node se kitni branches nikalti hain.

Hamari problem mein: **2 branches** (include/skip)

3. Leaf Nodes

Jahan decision khatam ho jaye.

Do type ke leaves:

- **Success leaf:** $x=0$ (target achieve) $\rightarrow$ return 1
- **Failure leaf:** $i^n > x$ (aage nahi ja sakte) $\rightarrow$ return 0

4. Path

Root se leaf tak ka complete route.

Har path ek unique combination hai.

- Include decisions ka set = ek combination
- Jaise path: Include 1, Skip 2, Include 3 $\rightarrow$ combination {1, 3}

Visualization: Tree Traversal Order

Function kis order mein nodes visit karta hai (Depth-First Search - DFS):

```
Order:  1
        / \
       2   5
      / \
     3   4
```

Visit order: 1, 2, 3, 4, 5

(Pehle left subtree poori explore karo, phir right)

Hamare example mein:

1. Root: ways(10,2,1)
2. Include 1: ways(9,2,2)
3. Include 2: ways(5,2,3) $\rightarrow$ return 0
4. Skip 2: ways(9,2,3)
5. Include 3: ways(0,2,4) $\rightarrow$ return 1 ✓
6. Skip 3: ways(9,2,4) $\rightarrow$ return 0

7. Back to root, Skip 1: ways(10,2,2)
8. Include 2: ways(6,2,3) → return 0
9. Skip 2: ways(10,2,3)
10. Include 3: ways(1,2,4) → return 0
11. Skip 3: ways(10,2,4) → return 0

Exercise: Draw Trees Yourself

Exercise 1: X=3, N=2

Available powers: $1^2=1$

Tree draw karo. (Hint: sirf 1 ka square use kar sakte hain)

Exercise 2: X=9, N=3

Available: $1^3=1$, $2^3=8$

Tree draw karo aur batao kitne ways hain.

Exercise 3: X=4, N=2

Available: $1^2=1$, $2^2=4$

Tree draw karo.

Answers

Exercise 1: X=3, N=2

ways(3, 2, 1)

└─ Include 1: ways(2, 2, 2) → $2^2=4 > 2$ → return 0

└─ Skip 1: ways(3, 2, 2) → $2^2=4 > 3$ → return 0

Total = 0 ways

Exercise 2: X=9, N=3

```

ways(9,3,1)
├─ Include 1³: ways(8,3,2)
|   └─ Include 2³: ways(0,3,3) → return 1 ✓
|   └─ Skip 2: ways(8,3,3) → 3³=27>8 → return 0
└─ Skip 1: ways(9,3,2)
    └─ Include 2³: ways(1,3,3) → 3³=27>1 → return 0
    └─ Skip 2: ways(9,3,3) → 27>9 → return 0
Total = 1 way (1³ + 2³ = 9)

```

Exercise 3: X=4, N=2

```

ways(4,2,1)
├─ Include 1²: ways(3,2,2) → 2²=4>3 → return 0
└─ Skip 1: ways(4,2,2)
    └─ Include 2²: ways(0,2,3) → return 1 ✓
    └─ Skip 2: ways(4,2,3) → 3²=9>4 → return 0
Total = 1 way (2² = 4)

```

Chapter Summary

- Recursion tree har function call aur uski choices ko visual karta hai
- Har path ek unique combination hai
- Leaves par base cases handle hote hain
- Tree DFS order mein traverse hota hai (left se right)
- Depth limited hai kyunki powers jaldi badhti hain

Tree ko samajhna = Recursion ko samajhna. Agle chapter mein backtracking seekhenge!

Chapter 7: Backtracking (Explore, Undo, Continue)

Backtracking Kya Hai?

Real-Life Analogy 1: Bhool Bhulaiya (Maze)

Socho aap ek maze mein hain. Aapko exit dhundhna hai:

1. Aap ek path choose karte hain
2. Us path par chalte hain
3. Agar exit mil gaya - great!
4. Agar dead end (band rasta) mila - **wapas aao** (backtrack)
5. Phir doosra path try karo

Yehi hai backtracking! Try, fail, wapaa aao, doosra try karo.

Real-Life Analogy 2: Tala Aur Chaabi

Aapke paas 5 chaabiyaa hain, ek tala kholna hai:

1. Pehli chaabi try ki - nahi khula (backtrack - chaabi wapaa rakho)
2. Doosri try ki - nahi khuli (backtrack)
3. Teesri try ki - khul gaya! ✓

Har galat chaabi ke baad “undo” karte hain aur next try karte hain.

Real-Life Analogy 3: Shopping

Aapke paas ₹100 hain. Aapko exactly ₹100 ka saman khareedna hai:

1. Ek item uthaya (₹30) - bag mein daala
2. Doosra item uthaya (₹40) - bag mein daala (total ₹70)
3. Teesra item (₹50) - ₹120 ho jayenge, budget se zyada
 - **Backtrack:** Teesra item wapaa rakho
4. Choutha item (₹30) - exactly ₹100! ✓

Backtracking vs Simple Recursion

Simple Recursion	Backtracking
Ek hi direction mein jaata hai	Multiple paths try karta hai
Wapas nahi aata	Galat path se wapas aata hai
Ek solution	All possible solutions explore
Example: Factorial	Example: Maze, Sudoku, Our Problem

Hamari problem backtracking isliye hai kyunki:

- Hum combinations try karte hain
- Agar sum zyada ho jaye to “undo” karte hain
- Doosri combinations try karte hain
- **All possible ways** count karte hain

Backtracking Ki Teen Steps

Step 1: CHOOSE (Chuno)

Ek decision lo. Current number ko include karo ya skip karo.

Step 2: EXPLORE (Aage Badho)

Decision ke saath aage ki possibilities explore karo.

`ways(remaining - power, n, next_num)` ya `ways(remaining, n, next_num)`

Step 3: UNCHOOSE / BACKTRACK (Wapas Aao)

Agar path sahi nahi to wapas aao aur doosra path try karo.

Ye automatically hota hai jab function return karta hai!

Important: Hamari problem mein “unchoose” explicit nahi karna padta kyunki hum koi global state modify nahi kar rahe. Parameters pass kar rahe hain to wapas aane par

automatically previous state mil jaati hai.

Backtracking Ka Magic (Code Mein)

```
def ways(x, n, i):
    # Base cases...

    # CHOOSE: Include i
    # EXPLORE with included i
    include_result = ways(x - i**n, n, i + 1)
    # Backtrack automatically! (x wapas wahi hai, kyunki local variable tha)

    # CHOOSE: Skip i
    # EXPLORE with skipped i
    skip_result = ways(x, n, i + 1)
    # Backtrack automatically!

    return include_result + skip_result
```

Notice karo: Humne kuch “undo” nahi kiya explicitly. Kyun?

- $x - i^{**n}$ ek new value hai jo function call ko pass hui
- Original x ki value change nahi hui
- Jab function return karta hai, original state intact hai

Ye hi backtracking ka sabse clean form hai!

Backtracking With Explicit State (Jab Zaroorat Ho)

Kabhi kabhi humein explicit state maintain karni padti hai. Jaise agar hum combinations print karwana chahein:

```
def find_combinations(x, n, i, current_combo):
    """
    current_combo: list of numbers chosen so far
    """
    if x == 0:
```

```

    print("Found:", current_combo) # Print the combination
    return 1

if i**n > x:
    return 0

ways_count = 0

# CHOOSE: Include i
current_combo.append(i)                # Add to combination
ways_count += find_combinations(x - i**n, n, i+1, current_combo)
current_combo.pop()                    # UNCHOOSE: Remove (BACKTRACK!)

# CHOOSE: Skip i
ways_count += find_combinations(x, n, i+1, current_combo)
# No need to unchoose because we didn't modify state

return ways_count

# Call
combos = []
find_combinations(10, 2, 1, combos)

```

Yahan append aur pop explicitly backtracking kar rahe hain.

Visual: Backtracking in Action

X=10, N=2 ke liye backtracking flow:

Start: [] (empty combination), remaining=10

Step 1: Try including 1

Combo: [1], remaining=9

Step 2: Try including 2

Combo: [1,2], remaining=5

Step 3: Try including 3

Combo: [1,2,3], remaining=-4 → FAIL (negative)

BACKTRACK: Remove 3 → [1,2], remaining=5

Step 4: Try skipping 3

Combo: [1,2], remaining=5

```
Next is 4:  $4^2=16 > 5 \rightarrow$  FAIL  
BACKTRACK: Remove 2  $\rightarrow [1]$ , remaining=9  
Step 5: Try skipping 2  
Combo:  $[1]$ , remaining=9  
Step 6: Try including 3  
Combo:  $[1,3]$ , remaining=0  $\rightarrow$  SUCCESS!  $\checkmark$   
BACKTRACK: Remove 3  $\rightarrow [1]$ , remaining=9  
Step 7: Try skipping 3  
Combo:  $[1]$ , remaining=9  
Next is 4:  $16 > 9 \rightarrow$  FAIL  
BACKTRACK: Remove 1  $\rightarrow []$ , remaining=10  
Step 8: Try skipping 1  
Combo:  $[]$ , remaining=10  
... (continue exploring starting from 2)
```

Backtracking Ki Important Properties

1. Systematic Exploration

Har possibility exactly ek baar explore hoti hai. Kuch miss nahi hota.

2. State Restoration

Backtrack karte waqt state wapas wahi aa jaati hai. Jaise kuch hua hi nahi.

3. Pruning (Kaant-Chhaant)

Agar pata chal jaye ki aage kuch nahi milega, to turant wapas aa jao.

Hamari problem mein: `if i**n > x: return 0` - ye pruning hai!

4. Completeness

Agar solution exist karta hai to backtracking zaroor dhundhegi.

Cricket Team Selection Analogy

Socho aapko 11 players ki team chunni hai 15 players mein se:

```
Player 1: [Select] ya [Reject]
Player 2: [Select] ya [Reject]
Player 3: [Select] ya [Reject]
...
Jab 11 select ho jayein: Team complete! ✓
Jab remaining players + selected < 11: Impossible, backtrack X
```

Hamari Problem Mein Backtracking

Hamara code backtracking naturally implement karta hai:

```
def ways(x, n, i):
    # Pruning: Agar current number ki power hi x se zyada
    if i**n > x:
        return 0    # Backtrack! Is path mein kuch nahi milega

    # Success: Target achieve
    if x == 0:
        return 1    # Mil gaya!

    # Explore both choices
    # Choice 1: Include (state automatically restored on return)
    include = ways(x - i**n, n, i + 1)

    # Choice 2: Skip (state intact)
    skip = ways(x, n, i + 1)

    return include + skip
```

Yahan backtracking automatic hai kyunki:

- x ki value change nahi hoti (immutable integer)
- x - i**n ek new value function ko pass hoti hai

- Return hone par purani value wapas mil jaati hai

Chapter Summary

- **Backtracking** = Try, Check, If fail then Undo and Try Next
- Teen steps: Choose → Explore → Unchoose
- Automatically backtrack jab function parameters pass ho (immutable types)
- Explicit backtrack needed for mutable types (lists, etc.)
- Hamari problem natural backtracking example hai
- Pruning se unnecessary exploration avoid hoti hai

Backtracking samajh aane ke baad, ab hum actual problem solve karne ki taraf badhenge!

Chapter 8: Building Towards The Solution (Practice Problems)

Iss chapter mein hum step-by-step 10 practice problems solve karenge. Har problem pehli se thodi mushkil hogi. End tak aate-aate aap original HackerRank problem solve karne ke liye ready ho jaoge.

Har problem ke liye:

1. Problem statement
 2. Solution approach (soch)
 3. Dry run
 4. Python code
 5. Line by line explanation
 6. Complexity analysis
 7. Common mistakes
-

Problem 1: Sum of Squares (Simple Loop Version)

Problem: 1 se N tak ke sabhi numbers ke squares ka sum nikalo.

Input: N = 5

Output: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1+4+9+16+25 = 55$

Solution Approach

Simple loop se kar sakte hain - abhi recursion ki zaroorat nahi.

Python Code

```
def sum_of_squares(n):  
    """Sum of squares from 1 to n"""  
    total = 0  
    for i in range(1, n + 1):  
        total += i ** 2  
    return total  
  
# Test  
print(sum_of_squares(5))    # 55  
print(sum_of_squares(3))    # 14  
print(sum_of_squares(10))   # 385
```

Line by Line

- Line 1-2: Function define kiya
- Line 3: total = 0 - sum store karne ke liye variable
- Line 4: for i in range(1, n+1) - 1 se n tak loop
- Line 5: total += i**2 - current number ka square add karo
- Line 6: Final sum return

Time Complexity: O(N)

Space Complexity: O(1)

Problem 2: Sum of Cubes (Simple Loop)

Problem: 1 se N tak ke cubes ka sum nikalo.

Input: N = 3

Output: $1^3 + 2^3 + 3^3 = 1+8+27 = 36$

Python Code

```
def sum_of_cubes(n):  
    total = 0  
    for i in range(1, n + 1):  
        total += i ** 3  
    return total  
  
# Test  
print(sum_of_cubes(3))    # 36  
print(sum_of_cubes(5))    # 225
```

Important Note

$$1^3 + 2^3 + \dots + n^3 = (n(n+1)/2)^2 = (1+2+\dots+n)^2$$

Ye interesting mathematical property hai - but hum general Nth power ke liye solve kar rahe hain.

Problem 3: Print Numbers Whose Nth Power $\leq X$

Problem: 1 se start karte hue, un numbers ko print karo jinki Nth power X se chhoti ya barabar ho.

Input: X = 20, N = 2

Output:

1 ($1^2 = 1$)
2 ($2^2 = 4$)
3 ($3^2 = 9$)
4 ($4^2 = 16$)

5 print nahi hoga kyunki $5^2=25 > 20$

Solution Approach

Loop chalao, jab tak power $\leq$ X, print karte jao.

Python Code

```
def print_valid_numbers(x, n):  
    """Print numbers whose nth power <= x"""  
    i = 1  
    while True:  
        power = i ** n  
        if power > x:  
            break  
        print(f"{i} (^{n} = {power})")  
        i += 1  
  
# Test  
print_valid_numbers(20, 2)  
print("----")  
print_valid_numbers(100, 3)
```

Line by Line

- `i = 1` : 1 se start

- Infinite loop (break condition andar hai)
- `power = i ** n` : Power calculate ki
- `if power > x: break` : Agar limit cross, loop se bahar
- Print karo, phir `i += 1` se next number

Time Complexity: $O(X^{1/n})$ - basically, kitne numbers ki power $\leq X$

Space Complexity: $O(1)$

Common Mistake

`while power <= x`: condition while loop mein rakhna tempting hai, lekin tab `power` pehle define karna padega. Ya `while True` simple hai.

Problem 4: Perfect Power Check

Problem: Check karo ki kya koi number, kisi integer ki Nth power hai?

Input:

- `num = 64, n = 3` → True ($4^3 = 64$)
- `num = 64, n = 2` → True ($8^2 = 64$)
- `num = 65, n = 2` → False

Python Code

```
def is_perfect_power(num, n):  
    """Check if num is nth power of some integer"""  
    if num < 1:  
        return False, None  
  
    i = 1  
    while True:  
        power = i ** n
```

```
    if power == num:
        return True, i
    if power > num:
        return False, None
    i += 1

# Test
print(is_perfect_power(64, 2))    # (True, 8)
print(is_perfect_power(64, 3))    # (True, 4)
print(is_perfect_power(65, 2))    # (False, None)
print(is_perfect_power(1, 5))     # (True, 1)
```

Time Complexity: $O(\text{num}^{(1/n)})$

Space Complexity: $O(1)$

Problem 5: Find One Combination (Recursion Intro)

Problem: Ek aisa combination dhundho (koi bhi ek) jiska sum exactly X ho using unique Nth powers.

Input: X = 10, N = 2

Output: [1, 3] (kyunki $1^2 + 3^2 = 10$)

Solution Approach

Pehli baar recursion use karenge! Hum ek combination dhundh rahe hain, saare nahi.

Python Code

```
def find_one_combination(x, n, i, current_combo):
    """
    Find ONE combination that sums to x
    Returns: True if found, False otherwise
    """
```

```

"""
# Base: Found!
if x == 0:
    return True

# Base: Cannot proceed
if i**n > x:
    return False

# Try including i
current_combo.append(i)
if find_one_combination(x - i**n, n, i + 1, current_combo):
    return True    # Found with i included
current_combo.pop() # Backtrack - i se nahi ban raha

# Try skipping i
if find_one_combination(x, n, i + 1, current_combo):
    return True    # Found without i

return False    # No combination found

# Test
combo = []
if find_one_combination(10, 2, 1, combo):
    print("Found:", combo)    # [1, 3]
else:
    print("No combination exists")

```

Line by Line

- **Line 7-8:** Agar $x=0$, matlab sum exactly mil gaya - return True
- **Line 11-12:** Agar current number ki power x se zyada, aage kuch nahi milega - return False
- **Line 15-19:** Current number ko include karo
 - `append(i)` - add to current combination
 - Recursive call with reduced x
 - Agar True mila - done!
 - Agar False - `pop()` karo (backtrack)
- **Line 22-24:** Current number skip karo
 - Bina modify kiye aage check karo

- **Line 26:** Kuch nahi mila

Dry Run: X=10, N=2, start with i=1

```

Call 1: find_one(10,2,1,[]) → combo=[]
  Try include 1: combo=[1], call find_one(9,2,2,[1])
    Call 2: find_one(9,2,2,[1])
      Try include 2: combo=[1,2], call find_one(5,2,3,[1,2])
        Call 3: find_one(5,2,3,[1,2])
          i=3, 3²=9 > 5 → return False
        Backtrack: pop 2, combo=[1]
      Try skip 2: call find_one(9,2,3,[1])
        Call 4: find_one(9,2,3,[1])
          i=3, 3²=9
          Try include 3: combo=[1,3], call find_one(0,2,4,[1,3])
            Call 5: find_one(0,2,4,[1,3])
              x=0 → return True!
            Found! Return True
          Return True
        Return True
      Return True
    Return True → combo = [1, 3]

```

Time Complexity: $O(2^M)$ worst case, jahan M = number of valid powers

Space Complexity: $O(M)$ for recursion stack + combination list

Problem 6: Count All Ways (The Actual Problem!)

Problem: Count karo kitne ways hain X ko unique N th powers ke sum se banane ke.

Input: $X = 10, N = 2$

Output: 1 (sirf $1^2 + 3^2$)

Input: $X = 100, N = 2$

Output: 3 ($6^2+8^2, 1^2+3^2+4^2+5^2+7^2, 10^2$)

Python Code (Final Solution)

```
def count_ways(x, n, i=1):
    """
    Count number of ways to represent x as sum of unique nth powers
    starting from i
    """
    # Base case 1: Exact sum achieved
    if x == 0:
        return 1

    # Base case 2: Current power exceeds remaining sum
    if i ** n > x:
        return 0

    # Option 1: Include i^n
    include = count_ways(x - i**n, n, i + 1)

    # Option 2: Skip i
    skip = count_ways(x, n, i + 1)

    return include + skip

# Test
print(count_ways(10, 2))    # 1
print(count_ways(13, 2))    # 1
print(count_ways(100, 2))   # 3
print(count_ways(100, 3))   # 1
```

Line by Line Explanation

- **Line 1:** Function with default parameter `i=1` (start from 1)
- **Line 6-7:** `if x == 0` - Agar exactly sum achieve ho gaya, ye ek valid way hai. Return 1.
- **Line 10-11:** `if i**n > x` - Agar current number ki power remaining sum se zyada hai, aage ke sab numbers bhi zyada denge. Return 0.
- **Line 14:** Include karo - `x - i**n` remaining ke saath, `i+1` se start karo
- **Line 17:** Skip karo - `x` same, `i+1` se start
- **Line 19:** Dono choices ke results ka sum return

Dry Run: X=10, N=2

```

count_ways(10, 2, 1)
├─ include 1: count_ways(9, 2, 2)
|   ├─ include 2: count_ways(5, 2, 3)
|   |   └─  $3^2=9 > 5 \rightarrow$  return 0
|   |   └─ total = 0
|   └─ skip 2: count_ways(9, 2, 3)
|       ├─ include 3: count_ways(0, 2, 4)  $\rightarrow$  return 1
|       └─ skip 3: count_ways(9, 2, 4)  $\rightarrow 4^2=16>9 \rightarrow$  return 0
|       └─ total = 1 + 0 = 1
|   └─ total from 2 = 0 + 1 = 1
└─ skip 1: count_ways(10, 2, 2)
    ├─ include 2: count_ways(6, 2, 3)
    |   └─  $3^2=9 > 6 \rightarrow$  return 0
    |   └─ total = 0
    └─ skip 2: count_ways(10, 2, 3)
        ├─ include 3: count_ways(1, 2, 4)  $\rightarrow 4^2=16>1 \rightarrow$  return 0
        └─ skip 3: count_ways(10, 2, 4)  $\rightarrow 16>10 \rightarrow$  return 0
        └─ total = 0
    └─ total from 2 = 0 + 0 = 0
└─ Total ways = 1 + 0 = 1

```

Time Complexity: $O(2^M)$ worst case

Jahan $M = \text{floor}(X^{(1/N)})$ = number of valid integers

For $X=100$, $N=2$: $M=10$, worst case $2^{10} = 1024$ operations

For $X=1000$, $N=2$: $M \approx 31$, worst case ~ 2 billion (but pruning reduces drastically)

Space Complexity: $O(M)$ for recursion stack**Common Mistakes**

1. **Base case order:** Pehle $x==0$ check karo, phir $i**n > x$. Kyun? Agar dono true hain to $x==0$ pehle catch hona chahiye.
2. **i+1 vs i:** Hamesha $i+1$ use karo taaki unique numbers hi use hon.
3. **Return type:** Integer return karo, True/False nahi.

Problem 7: Print All Combinations

Problem: Saare valid combinations print karo.

Input: X = 100, N = 2

Output:

```
[6, 8]
[1, 3, 4, 5, 7]
[10]
```

Python Code

```
def print_combinations(x, n, i=1, current_combo=None):
    """Print all combinations that sum to x"""
    if current_combo is None:
        current_combo = []

    if x == 0:
        print(current_combo)
        return

    if i ** n > x:
        return

    # Include i
    current_combo.append(i)
    print_combinations(x - i*n, n, i + 1, current_combo)
    current_combo.pop() # Backtrack

    # Skip i
    print_combinations(x, n, i + 1, current_combo)

# Test
print("Combinations for X=100, N=2:")
print_combinations(100, 2)
```

Important Note: Mutable Default Parameter

current_combo=None kyun use kiya instead of current_combo=[] ?

Kyunki Python mein mutable default arguments function define hote waqt ek baar create hote hain, aur har call mein same object use hota hai. Isse bugs aate hain.

```
# Galat:
def func(lst=[]): # Same list har call mein use hogi!
    ...

# Sahi:
def func(lst=None):
    if lst is None:
        lst = []
```

Problem 8: Smallest Combination (Minimum Numbers)

Problem: Sabse chhoti combination dhundho (minimum numbers wali).

Input: X = 100, N = 2

Output: [10] (sirf 1 number)

Python Code

```
def find_smallest_combo(x, n, i=1, current=None):
    """Find combination with minimum numbers"""
    if current is None:
        current = []

    if x == 0:
        return current[:] # Return copy of current combination

    if i ** n > x:
        return None # No valid combination
```

```

# Include i
current.append(i)
include_result = find_smallest_combo(x - i*n, n, i + 1, current)
current.pop()

# Skip i
skip_result = find_smallest_combo(x, n, i + 1, current)

# Compare and return the smaller one
if include_result is None and skip_result is None:
    return None
if include_result is None:
    return skip_result
if skip_result is None:
    return include_result

# Return the one with fewer elements
if len(include_result) <= len(skip_result):
    return include_result
return skip_result

# Test
result = find_smallest_combo(100, 2)
print("Smallest:", result) # [10]

```

Problem 9: Largest Combination (Maximum Numbers)

Problem: Sabse badi combination dhundho (maximum numbers wali).

Input ke liye, include first strategy se badi combinations pehle milti hain. Actually, “largest” ka matlab maximum count of numbers.

Python Code

```

def find_largest_combo(x, n, i=1):
    """Find combination with maximum numbers"""
    if x == 0:

```

```

        return []

    if i ** n > x:
        return None

    # Include i
    include_result = find_largest_combo(x - i**n, n, i + 1)
    if include_result is not None:
        return [i] + include_result

    # Skip i (only if include didn't work, to get max size we prefer includes)
    skip_result = find_largest_combo(x, n, i + 1)

    # Actually, we should try both and compare lengths
    # Let's do it properly
    include = find_largest_combo(x - i**n, n, i + 1)
    if include is not None:
        include = [i] + include

    skip = find_largest_combo(x, n, i + 1)

    if include is None and skip is None:
        return None
    if include is None:
        return skip
    if skip is None:
        return include
    if len(include) >= len(skip):
        return include
    return skip

# Test
result = find_largest_combo(100, 2)
print("Largest:", result) # Might be [1, 3, 4, 5, 7] or similar

```

Problem 10: Original HackerRank Problem

Problem: Complete function jo number of ways return kare.

Yeh wahi Problem 6 hai, lekin ab hum isse completely samajhte hain.

Final Clean Code

```
#!/bin/python3

import sys

def powerSum(X, N):
    """
    Complete the powerSum function.
    Returns number of ways to represent X as sum of unique Nth powers.
    """
    def helper(remaining, power, start):
        # Base: Found a valid way
        if remaining == 0:
            return 1

        # Base: Can't proceed further
        if start ** power > remaining:
            return 0

        # Include current number
        include = helper(remaining - start**power, power, start + 1)

        # Skip current number
        skip = helper(remaining, power, start + 1)

        return include + skip

    return helper(X, N, 1)

if __name__ == "__main__":
    X = int(input().strip())
    N = int(input().strip())
    result = powerSum(X, N)
    print(result)
```

HackerRank Specific Notes

1. Input format: Pehli line X, doosri line N
2. Constraints: $1 \leq X \leq 1000$, $2 \leq N \leq 10$
3. Ye constraints recursion ke liye safe hain

Chapter Summary Table

Problem	Concept	Complexity
Sum of Squares	Loop	$O(N)$
Sum of Cubes	Loop	$O(N)$
Print Valid Numbers	While + Break	$O(X^{(1/N)})$
Perfect Power Check	While loop	$O(X^{(1/N)})$
Find One Combo	Recursion + Backtracking	$O(2^M)$
Count All Ways	Recursion (Pure)	$O(2^M)$ pruned
Print All Combos	Recursion + List	$O(2^M)$
Smallest Combo	Recursion + Compare	$O(2^M)$
Largest Combo	Recursion + Compare	$O(2^M)$
HackerRank Solution	Final recursive	$O(2^M)$ pruned

Ab aap ready hain original problem ko confidently solve karne ke liye!

Chapter 9: Understanding The Main Recursive Function

Iss chapter mein hum apne main recursive function ko line-by-line, parameter-by-parameter samjhenge.

The Function Signature

```
def ways(x, n, i=1):
```

Parameter x (Remaining Sum)

Kya hai? Kitna sum aur banana hai (target remaining).

Shuru mein: $x = X$ (original target)

Har step par: Agar number include kiya to x kam hoga, nahi to same rahega.

Example:

```
Original X = 10
12=1 include karne ke baad:  $x = 10 - 1 = 9$ 
22=4 include na karne par:  $x = 9$  (same)
32=9 include karne par:  $x = 9 - 9 = 0 \rightarrow \text{SUCCESS!}$ 
```

Real-life: Jaise aapke paas ₹100 hain. Kuch khareedne ke baad bache hue paise = x .

Parameter n (Power)

Kya hai? Kis power ka use karna hai (2 for squares, 3 for cubes, etc.)

Kya ye change hota hai? KABHI NAHI! Pure recursion mein n constant rehta hai.

Kyun? Problem definition mein power fixed hai. Agar $X=100$, $N=2$, to hum sirf squares use karenge - kabhi cubes nahi.

Common Mistake: Kuch students n ko change kar dete hain. Ye galat hai!

```
# GALAT:
ways(x, n-1, i+1) # N ko mat chhedo!
```

```
# SAHI:
ways(x - i**n, n, i+1) # N same, sirf i change
```

Parameter i (Current Number)

Kya hai? Kaun sa number currently consider kar rahe hain.

Shuru mein: $i = 1$ (natural numbers 1 se start)

Har step par: $i + 1$ (agla number)

Kyun $i=1$ se start?

- Natural numbers 1, 2, 3, ... se start hote hain
- 0 include karna usually allowed nahi (aur $0^n = 0$ jo infinite loop de sakta hai)
- 1 se start karna safe aur problem requirement hai

Kyun i change hota hai?

- Taaki unique numbers use ho (1 baar use kiya to dobara nahi)
- Agar i change nahi kiya to infinite recursion (ek hi number baar-baar)

Function Body Deep Dive

```
def ways(x, n, i=1):
    # Part 1: Base Cases
    if x == 0:
        return 1

    if i ** n > x:
        return 0

    # Part 2: Recursive Cases
    include = ways(x - i**n, n, i + 1)
    skip = ways(x, n, i + 1)

    # Part 3: Combine Results
    return include + skip
```


Part 1: Base Cases (Rukne Ki Conditions)

Base Case 1: `if x == 0: return 1`

Question: "Return 1" kyun? "Return True" kyun nahi?

Answer: Hum **count** kar rahe hain ways ka. Jab $x=0$, matlab ek valid way mil gaya. Isliye 1 return karte hain (1 way found at this leaf).

```
# Agar return True karte:
if x == 0:
    return True # Ye batata hai "mil gaya", lekin kitne ways? Pata nahi

# Count karne ke liye:
if x == 0:
    return 1 # "Ek way mil gaya"
```

Real-life: Jaise aap coin toss mein head count karte ho. Har head = 1 count. Aise hi har valid way = 1 count.

Base Case 2: `if i**n > x: return 0`

Logic: Agar current number ki power remaining sum se zyada hai, to:

- Current number include nahi kar sakte (sum exceed ho jayega)
- Aage ke numbers bhi include nahi kar sakte (kyunki i badhta hai, to i^n bhi badhega)

Isliye is branch mein koi valid way nahi milega. Return 0.

Example: $x=5, i=3, n=2$

- $3^2 = 9 > 5$
- Na 3 le sakte hain, na 4, na 5...
- Return 0

Part 2: Recursive Calls

Call 1: Include Current Number

```
include = ways(x - i**n, n, i + 1)
```

Breakdown:

- $x - i^n$: Current number ki power remaining sum se subtract ki (number include kar liya)
- n : Power same
- $i + 1$: Agle number par jao (unique numbers ke liye)

Meaning: “Maine i ko choose kar liya. Ab remaining $(x - i^n)$ ko $i+1$ se aage ke numbers se banana hai.”

Call 2: Skip Current Number

```
skip = ways(x, n, i + 1)
```

Breakdown:

- x : Same remaining sum (kuch subtract nahi kiya)
- n : Power same
- $i + 1$: Agle number par jao

Meaning: “Maine i ko skip kar diya. Ab poora x $i+1$ se aage ke numbers se banana hai.”

Part 3: Combine Results

```
return include + skip
```

Total ways = (ways jisme i include hai) + (ways jisme i skip hai)

Kyun add? Kyunki ye do mutually exclusive possibilities hain. Har combination ya to i ko include karegi ya skip.

Visual:

Total ways for (x, i)
 |— Ways with i included: include

└─ Ways with i skipped: skip
Total = include + skip

Parameter Flow Visualization

```
ways(10, 2, 1)
  |
  x=10, n=2, i=1
  |
  |--- include: ways(10-1, 2, 1+1) = ways(9, 2, 2)
  |   |
  |   x=9, n=2, i=2
  |   |
  |   |--- include: ways(9-1, 2, 2+1) = ways(8, 2, 3)
  |   |   |
  |   |   ...
  |   |--- skip: ways(9, 2, 3)
  |   ...
  |
  |--- skip: ways(10, 2, 2)
      |
      x=10, n=2, i=2
      ...
```

Notice: n hamesha 2 rehta hai! Sirf x aur i change hote hain.

Why i+1 and Not i+n?

Question: Agle number par jaane ke liye i+1 kyun, i+n kyun nahi?

Answer: Hum consecutive natural numbers consider kar rahe hain: 1, 2, 3, 4, ...

Agar i+n karte:

```
ways(x, n, i+n) # i=1, n=2: 1, 3, 5, 7... (odd numbers only!)
```

Ye numbers 2, 4, 6 miss kar dega. Aur $2^2 = 4$ bhi important ho sakta hai.

Why Not Use `i` Without Increment?

```
# GALAT - Infinite recursion!  
def ways(x, n, i):  
    include = ways(x - i*n, n, i) # i same raha!  
    ...
```

Agar `i` change nahi kiya:

- Hamesha same number consider hoga
- `x` kabhi 0 nahi hoga (ya negative)
- Infinite recursion → Stack Overflow!

Common Questions

Q: Default parameter `i=1` kyun?

A: Taaki user ko `ways(10, 2)` call karna pade, `ways(10, 2, 1)` nahi. Starting number hamesha 1 hai, to default rakhna convenient hai.

Q: `n` constant kyun hai?

A: Problem definition: hum ek specific power (2nd, 3rd, etc.) ke numbers use kar rahe hain. Jaise agar problem hai "sum of squares", to sirf squares, cubes nahi.

Q: Kya hum `i=0` se start kar sakte hain?

A: Technically $0^n = 0$ (for $n > 0$). Isse fayda nahi, infinite loop ka risk hai, aur natural numbers conventionally 1 se start hote hain.

Chapter Summary

Parameter	Meaning	Changes?
x	Remaining sum to achieve	Decreases when include
n	Power (constant)	NEVER
i	Current number	Always i+1

Key Insight: Function har number ke liye do choices explore karta hai (include/skip), unique numbers ensure karne ke liye i badhata hai, aur base cases se rukna jaanta hai.

Chapter 10: Base Conditions (Deep Dive)

Base conditions hi recursion ki jaan hain. Inke bina recursion infinite loop mein chali jayegi. Iss chapter mein hum dono base conditions ko microscopically examine karenge.

Base Case 1: `if x == 0: return 1`

Yeh Condition Kyun?

Jab `x == 0`, matlab humne exactly target sum achieve kar liya.

Real-life: Jaise aapne exactly ₹100 ka saman khareed liya - mission accomplished!

Return 1 Kyun? Return True Kyun Nahi?

```
# Option A: Return 1 (Correct)
if x == 0:
    return 1
```

```
# Option B: Return True (Wrong for counting)
```

```
if x == 0:
    return True
```

Reason: Hum **count** kar rahe hain ways ka.

Caller function ye karta hai:

```
include = ways(...) # Expect integer
skip = ways(...)    # Expect integer
return include + skip # Integer addition
```

Agar hum True return karte:

```
include = True # Boolean
skip = False   # Boolean
return True + False # Kya hoga? (Actually Python mein 1+0=1, lekin logically
```

Python mein `True + False = 1`, lekin ye **semantically galat** hai. Hum count kar rahe hain, to numbers return karo.

Proof With Small Example

X=4, N=2: Sirf $2^2=4$ ek way hai.

```
def ways_count(x, n, i):
    if x == 0:
        return 1 # Counting: 1 way
    if i**n > x:
        return 0 # Counting: 0 ways
    return ways_count(x - i**n, n, i+1) + ways_count(x, n, i+1)
```

Tree:

```
ways(4,2,1) → 12=1
├─ include: ways(3,2,2) → 22=4>3 → 0
└─ skip: ways(4,2,2) → 22=4
    ├─ include: ways(0,2,3) → return 1 ✓
    └─ skip: ways(4,2,3) → 32=9>4 → 0
Total: 0 + (1 + 0) = 1 ✓ Correct!
```

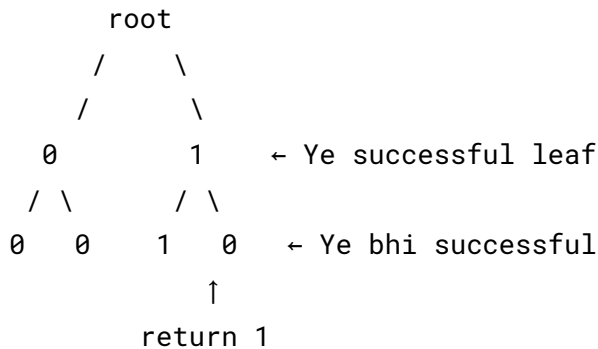
Agar return True karte:

```
# True = 1, False = 0 in Python arithmetic
# But let's say we return True/False
include = ways(...) # True or False
skip = ways(...)    # True or False
return include + skip # Bool + Bool → Int (but confusing)
```

Mathematical Meaning

return 1 ka matlab: "Is leaf node par ek solution mila."

Tree ke saare successful leaves ke 1's ka sum = total ways.



Total = 1 + 1 = 2 ways

Kya $x < 0$ Check Karna Chahiye?

Hamara code $x < 0$ explicitly check nahi karta. Kyun?

```
if i ** n > x:    # Ye condition  $x < i^n$  check karti hai
    return 0
```

Agar hum sirf positive values hi subtract karte hain ($x - i**n$), to x kabhi negative ho sakta hai?

Haan! Example: $x=5, i=3, n=2$

- $3^2 = 9$

- $x - i^{**n} = 5 - 9 = -4$ (negative!)

To kya $x < 0$ ka base case add karna chahiye?

```
if x < 0:
    return 0
```

Mere recommendation: Add kar do! Safety ke liye.

```
def ways(x, n, i):
    if x == 0:
        return 1
    if x < 0:                # Extra safety
        return 0
    if i ** n > x:
        return 0
    # ...
```

Lekin original code mein ye zaroori nahi kyunki $i^{**n} > x$ condition negative x ko bhi handle karegi (negative $< i^n$ for positive i^n).

Base Case 2: if $i^{**n} > x$: return 0

Yeh Condition Kyun?

Ye **pruning** hai. Agar current number ki power remaining sum se zyada hai, to:

1. Current number include nahi kar sakte
2. Aage ke numbers ki power aur zyada hogi
3. Is branch mein kabhi solution nahi milega

Real-life: Aapke paas ₹10 bache hain. Agla item ₹50 ka hai. Na wo le sakte hain, na usse mehenge items.

Mathematical Justification

For positive integers, if $a < b$:

- $a^n < b^n$ (for $n > 0$)

So if $i^n > x$, then:

- $(i+1)^n > i^n > x$
- $(i+2)^n > (i+1)^n > i^n > x$
- And so on...

No future number's power will ever be $\leq x$.

Why Return 0?

Kyunki is branch mein 0 ways hain. Koi solution nahi milega.

```
return 0 # "Yahan kuch nahi mila"
```

Caller ke paas jaake:

```
include = ways(x, n, i+1) # Returns 0
# include = 0, matlab include wali branch se kuch nahi mila
```

Example Demonstration

$X=10, N=2$:

```
ways(10, 2, 4)
Check:  $4^2 = 16$ 
 $16 > 10$ ? YES → return 0
```

Is branch ki zaroorat hi nahi explore karne ki!

Without this condition:

```
# Function kabhi khatam nahi hoga
# i badhta rahega: 4, 5, 6, ... infinity
# Kabhi  $x=0$  nahi hoga kyunki sab 16, 25, 36... ( $all > 10$ )
```

Is Condition Ki Placement

Order of checks (IMPORTANT):

1. $x == 0$ → Success
2. $x < 0$ → Failure (optional but recommended)
3. $i^n > x$ → Failure (pruning)

Kyun ye order? Agar $x=0$ hai, to success return karna chahiye chahe i kuch bhi ho.

Example:

```
ways(0, 2, 100)
# x=0 already achieved
#  $100^2 > 0$ , lekin humein farak nahi padta
# Return 1 (way found!)
```

Agar hum pehle $i^n > x$ check karein:

```
if  $100^{**2} > 0$ : # True!
    return 0 # Galat!  $x=0$  hone par 1 return karna chahiye tha
```

Base Cases Ke Beech Ka Relation

Dono base cases complementary hain:

Condition	Meaning	Return
$x == 0$	Perfect sum achieved	1 (success)
$i^n > x$	Can't proceed	0 (failure)

Together they ensure:

1. Kabhi infinite loop nahi hoga
2. Har valid path count hoga
3. Invalid paths ignore honge

Edge Cases Testing

Edge Case 1: X=1, N=2

```
ways(1, 2, 1)
├─ 12=1, include: ways(0,2,2) → x=0, return 1 ✓
└─ skip: ways(1,2,2) → 22=4>1, return 0
Total: 1
```

✓ Correct: $1^2 = 1$

Edge Case 2: X=2, N=2

```
ways(2, 2, 1)
├─ include 1: ways(1,2,2) → 22=4>1, return 0
└─ skip 1: ways(2,2,2) → 22=4>2, return 0
Total: 0
```

✓ Correct: $1^2=1$, need 1 more, lekin agla $2^2=4>1$. No way.

Edge Case 3: X=4, N=2

```
ways(4, 2, 1)
├─ include 1: ways(3,2,2) → 22=4>3, return 0
└─ skip 1: ways(4,2,2)
    ├─ include 2: ways(0,2,3) → return 1
    └─ skip 2: ways(4,2,3) → 32=9>4, return 0
Total: 1
```

✓ Correct: $2^2 = 4$

Edge Case 4: X=100, N=3

Power of 4 = 64, power of 5 = 125 > 100. So consider numbers 1-4.

Tree will find: $1^3+2^3+3^3+4^3 = 1+8+27+64 = 100 \rightarrow 1$ way.

Common Mistake: Missing Base Case

```
# GALAT - Missing  $i^n > x$  check
def ways(x, n, i):
    if x == 0:
        return 1
    # Missing: if  $i^n > x$ : return 0

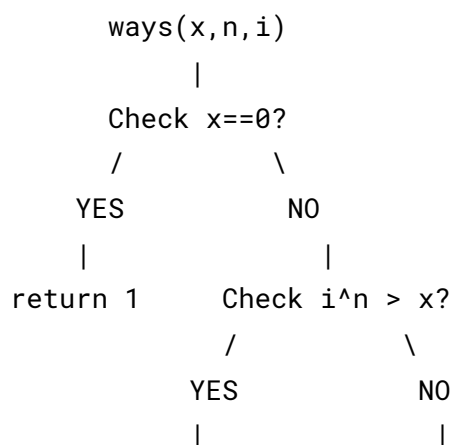
    include = ways(x - i**n, n, i+1)
    skip = ways(x, n, i+1) # i keeps increasing forever!
    return include + skip
```

Isme i badhta rahega, kabhi base case hit nahi hoga. Stack overflow!

Common Mistake: Wrong Return Value

```
# GALAT
if x == 0:
    return True # Should return 1
if i**n > x:
    return False # Should return 0
```

Visual: Base Cases in Recursion Tree



```
return 0    Make recursive
           calls
```

Chapter Summary

- **$x == 0 \rightarrow \text{return } 1$** : Ek valid way mil gaya, count karo
 - **$i^n > x \rightarrow \text{return } 0$** : Is branch mein kuch nahi milega, pruning
 - Dono base cases mil kar recursion ko safe banate hain
 - Order matters: pehle success check, phir failure
 - Bina base cases ke recursion kabhi khatam nahi hogi
-

Chapter 11: Two Recursive Calls (Deep Understanding)

Is chapter mein hum samjhenge ki do recursive calls kyun, aur har call ka kya meaning hai.

The Two Calls

```
include = ways(x - i**n, n, i + 1)    # Call 1
skip = ways(x, n, i + 1)              # Call 2
```

Call 1: Include Current Number

```
include = ways(x - i**n, n, i + 1)
```

Breakdown

Part	Value	Meaning
$x - i^{**}n$	Remaining - current power	Current number ko sum mein add kar diya
n	Same	Power constant
$i + 1$	Next number	Unique numbers ke liye aage badho

Meaning (English + Hinglish)

"Maine ye number choose kar liya. Ab bache hue sum ko agle numbers se pura karo."

Real-Life Example

Aap shopping kar rahe ho, budget ₹100:

- Item 1 (₹30) khareed liya
- Ab bache hue ₹70 se aage ki shopping karo
- Agla item 2 se shuru karo (item 1 dobara mat khareedo)

Visualization

Include karte waqt:

```

| Current State: |
| x = 10, i = 1  |
|               |
| Decision: Include  $1^2 = 1$  |
|               |
| New State:     |
| x = 9, i = 2   |
| "9 ko 2 se aage ke |
| numbers se banao" |

```

Code Flow

```
# Jab include call hota hai:  
include = ways(9, 2, 2) # x kam ho gaya, i badh gaya  
# Ye function ab explore karega:  
# - Include  $2^2=4$ : ways(5, 2, 3)  
# - Skip 2: ways(9, 2, 3)
```

Why $x - i^{**n}$?

Kyunki humne current number ki power sum mein use kar li. Ab target sum kam ho gaya.

```
Original target: 10  
Used:  $1^2 = 1$   
Remaining target:  $10 - 1 = 9$ 
```

Why $i + 1$?

Taaki same number dobara use na ho (uniqueness).

```
Current i=1  
Next i=2 (1+1)  
1 ko dobara consider nahi karenge
```

Call 2: Skip Current Number

```
skip = ways(x, n, i + 1)
```

Breakdown

Part	Value	Meaning
x	Same	Kuch subtract nahi kiya
n	Same	Power constant

Part	Value	Meaning
$i + 1$	Next number	Agle number par jao

Meaning

"Maine ye number skip kar diya. Same target agle number se pura karo."

Real-Life Example

Shopping:

- Item 1 (₹30) ko ignore kiya
- Abhi bhi ₹100 bache hain
- Agle item (item 2) se shopping shuru karo

Visualization

Skip karte waqt:

```

| Current State: |
| x = 10, i = 1 |
|               |
| Decision: Skip 1² |
|               |
| New State: |
| x = 10, i = 2 |
| "10 ko 2 se aage ke |
| numbers se banao" |

```

Why x Same?

Humne current number use nahi kiya, to target same raha.

Why $i + 1$?

Current number skip kar diya, ab agle number par focus karo. Aur current number ko dubara consider nahi karna (taaki infinite loop na ho).

Dono Calls Together

```
include = ways(x - i**n, n, i + 1)
skip = ways(x, n, i + 1)
return include + skip
```

Mathematical Logic

Total ways to make sum x from numbers $\{i, i+1, i+2, \dots\}$:

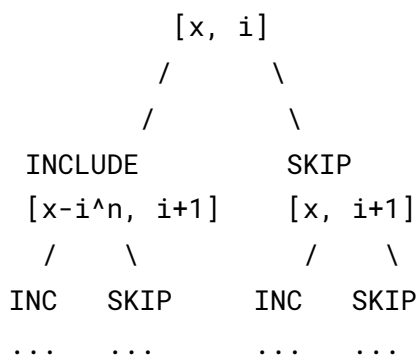
= (Ways that include i) + (Ways that exclude i)

= $\text{ways}(x - i^n, i+1) + \text{ways}(x, i+1)$

Ye partition complete hai:

- Har combination ya to i ko include karegi ya skip
- Dono sets disjoint hain (koi combination dono mein nahi)
- Dono sets ka union = all possible combinations

Visual: Binary Decision Tree



Har level par do choices. Total paths = $2^{\text{(number of levels)}}$.

Detailed Trace: X=10, N=2

```

ways(10, 2, 1) # Start
|
├ include = ways(9, 2, 2) # 1²=1 include kiya
| |
| └ include = ways(5, 2, 3) # 2²=4 include kiya
| | |
| | └ check: 3²=9 > 5 → return 0
| |
| └ skip = ways(9, 2, 3) # 2 skip kiya
|   |
|   └ include = ways(0, 2, 4) # 3²=9 include kiya → return 1
|     └ skip = ways(9, 2, 4) # 4²=16>9 → return 0
|       return 1 + 0 = 1
| return 0 + 1 = 1
|
└ skip = ways(10, 2, 2) # 1 skip kiya
  |
  └ include = ways(6, 2, 3) # 2²=4 include kiya
    |
    └ check: 3²=9 > 6 → return 0
    |
    └ skip = ways(10, 2, 3) # 2 skip kiya
      |
      └ include = ways(1, 2, 4) # 3²=9 include kiya
        |
        └ check: 4²=16 > 1 → return 0
        |
        └ skip = ways(10, 2, 4) # 4²=16 > 10 → return 0
          return 0 + 0 = 0
        return 0 + 0 = 0

```

Total = 1 + 0 = 1

Why Two Calls Exactly?

Question: Teen calls kyun nahi? Ya ek call?

Answer: Har number ke liye sirf **do** possibilities hain:

1. Lena hai (include)
2. Nahi lena hai (skip)

“Thoda lena” ya “aadha lena” possible nahi hai. Power ya to poori loge ya nahi loge.

Counter Example: Agar Teen Calls Hote

```
# Galat approach:
include_full = ways(x - i**n, n, i + 1)
include_half = ways(x - i**n/2, n, i + 1) # Makes no sense!
skip = ways(x, n, i + 1)
```

Power values fixed hain. $2^2 = 4$, aap 4 ya 0 le sakte ho, 2 nahi le sakte.

Why Not Call With Same i?

```
# Galat:
include = ways(x - i**n, n, i) # i same raha!
```

Isse kya hoga?

```
ways(10, 2, 1)
├ include: ways(9, 2, 1) # i still 1
| └ include: ways(8, 2, 1) # i still 1
| | └ include: ways(7, 2, 1) # i still 1...
| | | ... infinite!
```

Hamesha $i+1$ karo taaki aage badho aur kabhi base case hit ho.

Why Not $i+2$ or $i+n$?

```
# Galat:  $i+2$ 
skip = ways(x, n, i + 2) # Numbers skip ho jayenge
```

Example: $i=1, n=2$

- $i+2$ karte to: 1, 3, 5, 7... (2 miss ho jayega)
- $2^2 = 4$ important ho sakta hai

```
# Galat: i+n
skip = ways(x, n, i + n)
# For n=2: 1, 3, 5... (even numbers miss)
# For n=3: 1, 4, 7... (many numbers miss)
```

Hamesha $i+1$ use karo - har natural number consider karo.

The Addition Logic

```
return include + skip
```

Kyun add? Kyunki:

```
Total valid combinations
= Combinations WITH current number
+ Combinations WITHOUT current number
```

Ye addition tabhi valid hai jab dono sets completely disjoint hon. Aur hain bhi - ek combination mein ya to i hai ya nahi, dono nahi ho sakta.

Chapter Summary

Call	Parameters	Meaning
include	$(x-i^n, n, i+1)$	"Ye number le liya, aage badho"
skip	$(x, n, i+1)$	"Ye number skip kiya, aage badho"

- Dono calls complement hain - cover all possibilities
- i hamesha $i+1$ hota hai for uniqueness

- n kabhi change nahi hota
- Results add hote hain for total count

Chapter 12: Complete Dry Runs

Ab hum teen important examples ka complete dry run karenge. Har recursive call, har return value trace karenge.

Dry Run 1: X=10, N=2

Available: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16(>10 \text{ stop})$

Complete Trace

```
CALL 1: ways(10, 2, 1)
├─ Check:  $x=10 \neq 0$ ,  $1^2=1 \leq 10$ 
├─ Call 2: ways(9, 2, 2) [INCLUDE 1]
│   ├─ Check:  $x=9 \neq 0$ ,  $2^2=4 \leq 9$ 
│   └─ Call 3: ways(5, 2, 3) [INCLUDE 2]
│       └─ Check:  $x=5 \neq 0$ ,  $3^2=9 > 5$ 
│           └─ RETURN 0
│   └─ Call 4: ways(9, 2, 3) [SKIP 2]
│       └─ Check:  $x=9 \neq 0$ ,  $3^2=9 \leq 9$ 
│           └─ Call 5: ways(0, 2, 4) [INCLUDE 3]
│               └─ Check:  $x=0$ 
│                   └─ RETURN 1 ✓ WAY FOUND!
│   └─ Call 6: ways(9, 2, 4) [SKIP 3]
│       └─ Check:  $4^2=16 > 9$ 
│           └─ RETURN 0
│   └─ Return:  $1 + 0 = 1$ 
└─ Return:  $0 + 1 = 1$ 
├─ Call 7: ways(10, 2, 2) [SKIP 1]
└─ Check:  $x=10 \neq 0$ ,  $2^2=4 \leq 10$ 
```

```

|   |─ Call 8: ways(6, 2, 3)  [INCLUDE 2]
|   |   |─ Check:  $3^2=9>6$ 
|   |   └─ RETURN 0
|   |─ Call 9: ways(10, 2, 3) [SKIP 2]
|   |   |─ Check:  $3^2=9\leq 10$ 
|   |   |─ Call 10: ways(1, 2, 4) [INCLUDE 3]
|   |   |   |─ Check:  $4^2=16>1$ 
|   |   |   └─ RETURN 0
|   |   |─ Call 11: ways(10, 2, 4) [SKIP 3]
|   |   |   |─ Check:  $4^2=16>10$ 
|   |   |   └─ RETURN 0
|   |   └─ Return:  $0 + 0 = 0$ 
|   └─ Return:  $0 + 0 = 0$ 
└─ FINAL Return:  $1 + 0 = 1$ 

```

Summary

Total Ways = 1

Combination: $1^2 + 3^2 = 1 + 9 = 10$

Dry Run 2: X=17, N=2

Available: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16$, $5^2=25(>17 \text{ stop})$

Skeleton Trace (Important Calls Only)

```

ways(17, 2, 1)
|─ INCLUDE 1: ways(16, 2, 2)
|   |─ INCLUDE 2: ways(12, 2, 3)
|   |   |─ INCLUDE 3: ways(3, 2, 4) →  $4^2=16>3 \rightarrow 0$ 
|   |   └─ SKIP 3: ways(12, 2, 4)
|   |       |─ INCLUDE 4: ways(-4, 2, 5) → Actually check:  $4^2=16\leq 12?$ 
|   |       |   Wait:  $4^2=16$ ,  $12-16=-4$ 
|   |       |   ways(-4, 2, 5): Check  $5^2=25>(-4) \rightarrow 0$ 
|   |       |   (Need  $x<0$  check for safety)
|   |       └─ SKIP 4: ways(12, 2, 5) →  $5^2=25>12 \rightarrow 0$ 

```

```

|   |   Return 0
|   |   Return 0
|   └─ SKIP 2: ways(16,2,3)
|       └─ INCLUDE 3: ways(7,2,4) →  $4^2=16>7$  → 0
|       └─ SKIP 3: ways(16,2,4)
|           └─ INCLUDE 4: ways(0,2,5) → return 1 ✓ ( $4^2=16$ )
|           └─ SKIP 4: ways(16,2,5) →  $5^2=25>16$  → 0
|       Return 1
|   Return 1
|   Return 0 + 1 = 1
└─ SKIP 1: ways(17,2,2)
    └─ INCLUDE 2: ways(13,2,3)
        └─ INCLUDE 3: ways(4,2,4)
            └─  $4^2=16>4$  → 0
            └─ SKIP 4: ways(4,2,5) →  $25>4$  → 0
            Return 0
        └─ SKIP 3: ways(13,2,4)
            └─ INCLUDE 4: ways(-3,...) → 0
            └─ SKIP 4: ways(13,2,5) → 0
            Return 0
        Return 0
    └─ SKIP 2: ways(17,2,3)
        └─ INCLUDE 3: ways(8,2,4) →  $4^2=16>8$  → 0
        └─ SKIP 3: ways(17,2,4)
            └─ INCLUDE 4: ways(1,2,5) →  $25>1$  → 0
            └─ SKIP 4: ways(17,2,5) →  $25>17$  → 0
            Return 0
        Return 0
    Return 0
Return 0

```

Total = 1 + 0 = 1

Way found: $1^2 + 4^2 = 1 + 16 = 17$

Verification

Kya 17 ko doosre tarike se bana sakte hain squares se?

- $2^2+3^2=4+9=13$ X
- $3^2+?$ $9+?=17$, need 8 which is not square
- 4^2+1^2 already counted

Answer: 1 way ✓

Dry Run 3: $X=100$, $N=2$

Available: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16$, $5^2=25$, $6^2=36$, $7^2=49$, $8^2=64$, $9^2=81$, $10^2=100$

Ye bada example hai, isliye sirf successful paths trace karenge.

Success Path 1: $10^2 = 100$

```
ways(100, 2, 1)
SKIP 1: ways(100, 2, 2)
SKIP 2: ways(100, 2, 3)
...
SKIP 9: ways(100, 2, 10)
INCLUDE 10: ways(0, 2, 11) → RETURN 1 ✓
```

Success Path 2: $6^2 + 8^2 = 36 + 64 = 100$

```
ways(100, 2, 1)
SKIP 1: ways(100, 2, 2)
...
SKIP 5: ways(100, 2, 6)
INCLUDE 6: ways(64, 2, 7)
SKIP 7: ways(64, 2, 8)
INCLUDE 8: ways(0, 2, 9) → RETURN 1 ✓
```

Success Path 3: $1^2+3^2+4^2+5^2+7^2 = 1+9+16+25+49 = 100$

```
ways(100, 2, 1)
INCLUDE 1: ways(99, 2, 2)
SKIP 2: ways(99, 2, 3)
INCLUDE 3: ways(90, 2, 4)
INCLUDE 4: ways(74, 2, 5)
```



```
INCLUDE 5: ways(49,2,6)
SKIP 6: ways(49,2,7)
INCLUDE 7: ways(0,2,8) → RETURN 1 ✓
```

Total Count

3 ways found.

Chapter Summary Table

X	N	Ways	Combinations
10	2	1	1^2+3^2
17	2	1	1^2+4^2
100	2	3	$10^2, 6^2+8^2, 1^2+3^2+4^2+5^2+7^2$
100	3	1	$1^3+2^3+3^3+4^3$
13	2	1	2^2+3^2

Chapter 13: Visualization (Recursion Tree Diagrams)

Iss chapter mein hum ASCII art se complete recursion trees draw karenge.

How To Read Our Trees

```
Status indicators:
✓ = Success (return 1)
✗ = Failure (return 0)
```



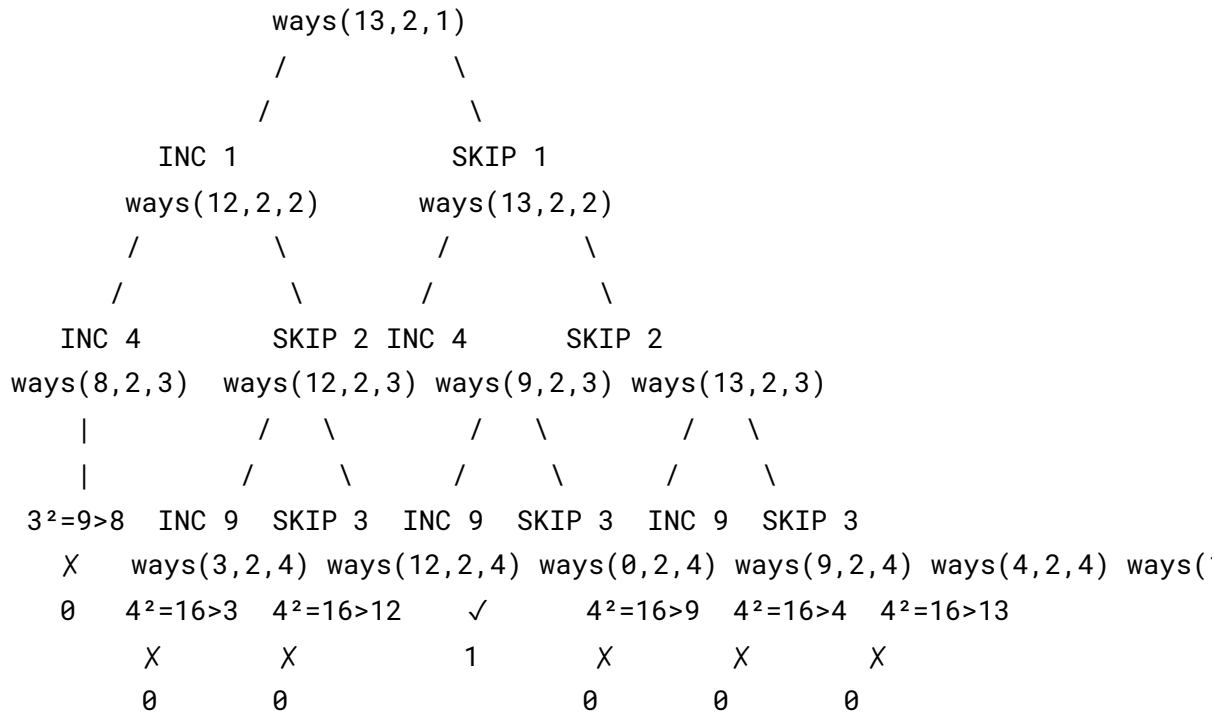
90/237

$3^2=9>5$ = Check: $3^2=9 > \text{remaining}=5 \rightarrow \text{Stop}$

✓ = Base case $x=0$ hit, return 1

X = Base case $i^n>x$ hit, return 0

Simplified Tree: X=13, N=2



Result: Sirf ek path successful: SKIP 1 → SKIP 2 → INC 3 → ✓

Combination: $2^2 + 3^2 = 4 + 9 = 13$

Tree Visualization For Understanding Flow

Depth-First Traversal Order

1. ways(10,2,1) ← Start (root)
2. ways(9,2,2) ← Go left (include)
3. ways(5,2,3) ← Go left (include)

```

4. return 0           ← Leaf, go back
5. ways(9,2,3)        ← Go right (skip)
6. ways(0,2,4)        ← Go left (include)
7. return 1 ✓         ← Success leaf!
8. ways(9,2,4)        ← Go right (skip)
9. return 0           ← Leaf
10. return 1          ← Combine children
11. return 1          ← Combine children
12. ways(10,2,2)      ← Go right from root
... continue ...

```

Visual Stack Representation

Stack grows downward (each level = one function call)

```

Level 1: ways(10,2,1) - calculating include...
Level 2:  ways(9,2,2) - calculating include...
Level 3:   ways(5,2,3) → returns 0
Level 2:   ways(9,2,2) - got include=0, calculating skip...
Level 3:   ways(9,2,3) - calculating include...
Level 4:    ways(0,2,4) → returns 1 ✓
Level 3:    ways(9,2,3) - got include=1, calculating skip...
Level 4:    ways(9,2,4) → returns 0
Level 3:    ways(9,2,3) → returns 1+0=1
Level 2:    ways(9,2,2) → returns 0+1=1
Level 1: ways(10,2,1) - got include=1, calculating skip...
...

```

Success Paths Visualization: X=100, N=2

Tree itna bada hai ki pura draw nahi kar sakte. Sirf success paths dikhate hain:

Path 1 (Direct):

```

ways(100,2,1)
└─ SKIP 1

```

```

└─ SKIP 2
  └─ ... (skip 3-9)
    └─ SKIP 9
      └─ INCLUDE 10: ways(0,2,11) → ✓

```

Combination: [10]

Path 2:

```

ways(100,2,1)
└─ SKIP 1
  └─ ...
    └─ SKIP 5
      └─ INCLUDE 6: ways(64,2,7)
        └─ SKIP 7
          └─ INCLUDE 8: ways(0,2,9) → ✓

```

Combination: [6, 8]

Path 3:

```

ways(100,2,1)
└─ INCLUDE 1: ways(99,2,2)
  └─ SKIP 2: ways(99,2,3)
    └─ INCLUDE 3: ways(90,2,4)
      └─ INCLUDE 4: ways(74,2,5)
        └─ INCLUDE 5: ways(49,2,6)
          └─ SKIP 6: ways(49,2,7)
            └─ INCLUDE 7: ways(0,2,8) → ✓

```

Combination: [1, 3, 4, 5, 7]

Chapter Summary

- Recursion tree har possible path dikhata hai

- Tree DFS order mein traverse hota hai
 - Left child = include, right child = skip
 - Leaves par base cases handle hote hain
 - Successful leaves return 1, contribute to total
 - Tree size = $O(2^M)$ where M = valid numbers count
-

Chapter 14: Common Mistakes (Aur Unse Kaise Bachen)

Iss chapter mein hum common mistakes dekhenge jo beginners karte hain, aur unhe kaise avoid karein.

Mistake 1: Wrong Base Condition Order

Galat Code:

```
def ways(x, n, i):  
    if i**n > x:          # Pehle ye check  
        return 0  
    if x == 0:           # Phir ye  
        return 1
```

Problem:

Jab $x=0$ aur i kuch bhi ho, $i**n > 0$ true ho sakta hai!

Example: `ways(0, 2, 5)`

- $5^2 = 25 > 0 \rightarrow$ return 0
- Lekin $x=0$ already achieved tha! Return 1 hona chahiye.

Sahi Code:

```
def ways(x, n, i):  
    if x == 0:          # Pehle success check  
        return 1  
    if i**n > x:        # Phir failure check  
        return 0
```

Mistake 2: Infinite Recursion

Galat Code:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    include = ways(x - i**n, n, i)    # i same!  
    skip = ways(x, n, i)              # i same!  
    return include + skip
```

Problem:

i kabhi change nahi hota. Function baar-baar same parameters se call hota hai. Infinite loop!

Sahi:

```
include = ways(x - i**n, n, i + 1)    # i badhao  
skip = ways(x, n, i + 1)             # i badhao
```

Mistake 3: Wrong Increment (i+n or i+2)

Galat Code:

```
include = ways(x - i**n, n, i + n)
```

Problem:

Agar n=2: 1, 3, 5, 7... (even numbers miss)

Agar n=3: 1, 4, 7, 10... (many numbers miss)

Sahi:

```
include = ways(x - i**n, n, i + 1)    # Hamesha i+1
```

Mistake 4: Changing n

Galat Code:

```
include = ways(x - i**n, n-1, i+1)    # n change kar diya
```

Problem:

Power change ho gayi. Problem requirement: fixed power use karo.

Sahi:

```
include = ways(x - i**n, n, i+1)    # n constant
```

Mistake 5: Not Understanding Return Values

Galat:

```
if x == 0:  
    return True    # Boolean return
```

Problem:

Caller expect karta hai integer taaki add kar sake.

```
include + skip    # Agar True/False hain to unexpected behavior
```

Sahi:

```
if x == 0:  
    return 1      # Integer
```

Mistake 6: Missing Base Case

Galat:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    # Missing: if i**n > x: return 0  
    include = ways(x - i**n, n, i+1)  
    skip = ways(x, n, i+1)  
    return include + skip
```

Problem:

i badhta rahega, kabhi rukega nahi. Stack overflow error.

Sahi:

Add if `i**n > x`: return 0

Mistake 7: Using Loops Inside Recursive Function

Galat:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    total = 0  
    for j in range(i, int(x**(1/n))+1):    # Loop!  
        total += ways(x - j**n, n, j+1)  
    return total
```

Ye technically kaam karega (aur optimized bhi hai!), lekin hum recursion seekh rahe hain. Pehle simple recursion samjho, optimization baad mein.

Mistake 8: Mutable Default Arguments

Galat:

```
def print_combos(x, n, i=1, combo=[]):    # Mutable default!  
    if x == 0:  
        print(combo)  
        return  
    combo.append(i)  
    print_combos(x - i**n, n, i+1, combo)  
    combo.pop()  
    print_combos(x, n, i+1, combo)
```

Problem:

Python mein `combo=[]` sirf ek baar create hota hai. Multiple calls same list use karenge!

Sahi:

```
def print_combos(x, n, i=1, combo=None):  
    if combo is None:  
        combo = []  
    # ... rest of code
```

Mistake 9: Not Handling $x < 0$

Galat:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    include = ways(x - i**n, n, i+1)  
    # ...
```

Problem:

Agar $x - i**n$ negative ho jaye, to $i**n > x$ true hoga (positive > negative). Lekin ye implicit hai. Explicit check better:

Sahi:

```
def ways(x, n, i):  
    if x == 0:  
        return 1
```

```
if x < 0:                # Explicit negative check
    return 0
if i**n > x:
    return 0
```

Mistake 10: Copy-Paste Errors

Common copy-paste error:

```
include = ways(x - i**n, n, i+1)
skip = ways(x - i**n, n, i+1)    # Copy-paste! Galat!
```

Skip call mein x same hona chahiye, x - i**n nahi.

Sahi:

```
include = ways(x - i**n, n, i+1)
skip = ways(x, n, i+1)           # x unchanged
```

Mistake 11: Wrong Starting Index

Galat:

```
def powerSum(X, N):
    return ways(X, N, 0)    # 0 se start
```

Problem:

$0^2 = 0$, jo infinite loop de sakta hai. Aur 0 include karne ka koi fayda nahi.

Sahi:

```
def powerSum(X, N):  
    return ways(X, N, 1)    # 1 se start
```

Mistake 12: Printing Instead of Returning

Galat:

```
if x == 0:  
    print("Found")  
    return    # Returns None!
```

Sahi:

```
if x == 0:  
    return 1    # Integer return
```

Checklist: Avoid Mistakes

Check	Question
✓	Base case $x==0$ pehle check hora hai?
✓	Base case $i**n > x$ hai?
✓	i hamesha $i+1$ ho raha hai?
✓	n change nahi ho raha?
✓	Return integer ho raha hai?

Check	Question
✓	Default argument mutable to nahi?
✓	Starting index 1 hai?

Chapter 15: Interview Questions (50 Questions)

Conceptual Questions (1-10)

Q1: “Sum of Nth Powers” problem kya hai? Simple words mein samjhao.

Answer Humein count karna hai ki kitne tareeke hain jisse hum unique natural numbers ki Nth powers (jaise squares, cubes) ko jod kar exactly X bana sakte hain. Har number sirf ek baar use kar sakte hain.

Q2: Hum is problem mein loops use kyun nahi kar sakte efficiently?

Answer Kyunki humein nahi pata ki kitne numbers ki zaroorat padegi. Kabhi 2 numbers se sum ban jata hai, kabhi 7 se. Fixed nested loops unknown depth ke liye kaam nahi karte. Recursion naturally variable depth handle karta hai.

Q3: Recursion aur Backtracking mein kya difference hai?

Answer Recursion: Function khud ko call karta hai. Backtracking: Try karo, agar fail to undo karo, doosra option try karo. Backtracking usually recursion use karta hai, lekin har recursion backtracking nahi hai. Is problem mein backtracking ho rahi hai kyunki hum include/skip try kar rahe hain aur fail hone par wapas aa rahe hain.

Q4: Base case $x == 0$ par 1 kyun return karte hain, True nahi?

Answer Hum count kar rahe hain number of ways. Har valid way ek count hai. 1 return karne se caller function mein `include + skip` integer addition properly kaam karta hai. True return karne se semantically galat hai, chahe Python mein $\text{True}=1$ ho.

Q5: Parameter i ko $i+1$ kyun karte hain, i same kyun nahi rehne dete?

Answer Taaki: (1) Unique numbers use ho (ek number dobara consider na ho), (2) Infinite recursion na ho. Agar i same rahega to function baar-baar same number consider karega aur kabhi base case hit nahi karega.

Q6: Is problem ki time complexity kya hai? Samjhao.

Answer Worst case: $O(2^M)$ jahan $M = \text{floor}(X^{(1/N)})$ = number of valid integers. Example: $X=100$, $N=2$, $M=10$, worst case ~ 1024 nodes. Lekin pruning ($i^n > x$) ki wajah se actual nodes bahut kam hote hain. Tree kaafi chhota rehta hai.

Q7: Agar hum same number ko multiple times use kar sakte to kya change hota?

Answer Phir $i+1$ ki jagah i use karte (same number repeat allowed). Ye "unbounded knapsack" type problem ho jaati. Code mein change: `ways(x-i^n, n, i)` aur `ways(x, n, i+1)`.

Q8: Decision tree mein pruning kya hai? Is problem mein kahan use hota hai?

Answer Pruning = unnecessary branches ko explore karne se pehle hi kaat dena. Is problem mein: `if i**n > x: return 0` pruning hai. Kyunki agar current power remaining sum se zyada hai, to aage ke sab numbers ki power aur zyada hogi. Koi solution nahi milega.

Q9: Kya hum DP (Dynamic Programming) se is problem ko optimize kar sakte hain?

Answer Haan, memoization se. `(x, i)` pair ke results cache kar sakte hain. Lekin constraints ($X \leq 1000$, $N \geq 2$) ke liye recursion without memoization bhi fast enough hai. DP advanced topic hai, pehle recursion mastery zaroori hai.

Q10: Call stack kya hai? Recursion mein iski kya bhoomika hai?

Answer Call stack memory mein ek data structure hai jo function calls ko track karta hai. Har recursive call stack mein push hoti hai, return hone par pop. Is problem mein maximum stack depth $\approx X^{(1/N)}$ hogi, jo safe hai.

Output Prediction Questions (11-20)

Q11: Output batao

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    return ways(x-i**n, n, i+1) + ways(x, n, i+1)  
  
print(ways(1, 2, 1))
```

Answer: $1 \cdot 1^2 = 1$, x becomes 0, return 1. Only one way: [1].

Q12: Output batao

```
print(ways(2, 2, 1))
```

Answer: $0 \cdot 1^2 = 1 \rightarrow$ remaining 1, next $2^2 = 4 > 1$. Skip 1 $\rightarrow 2^2 = 4 > 2$. No way.

Q13: Output batao


```
print(ways(4, 2, 1))
```

Answer: 1 Only $2^2=4$. ($1^2+?$ need 3, not a square)

Q14: Output batao

```
print(ways(5, 2, 1))
```

Answer: 1 $1^2+2^2=1+4=5$. One way.

Q15: Output batao

```
print(ways(9, 2, 1))
```

Answer: 1 $3^2=9$. ($1^2+2^2+?=5+?$ need $4=2^2$ but 2 already used. Actually $1^2+2^2+?=5$, need 4 but $2^2=4$ would be duplicate if already used? Check: $1^2+2^2=5$, need 4, but next would be $3^2=9>4$. So only $3^2=9$.)

Q16: Output batao

```
print(ways(8, 3, 1))
```

Answer: 1 $2^3=8$. One way.

Q17: Output batao

```
def f(x, n, i):
    if x == 0: return 1
    if i**n > x: return 0
    return f(x, n, i+1)

print(f(10, 2, 1))
```

Answer: 0 Ye function sirf skip karta hai, include nahi. Kabhi kuch sum nahi banega (jab tak $x=0$ start na ho).

Q18: Output batao

```
def f(x, n, i):  
    if x == 0: return 1  
    if i**n > x: return 0  
    return f(x - i**n, n, i+1)  
  
print(f(5, 2, 1))
```

Answer: 1 Sirf include karta hai, skip nahi. $1^2+2^2=5 \rightarrow$ success. Agar combination skip karna zaroori hota to galat answer deta.

Q19: Output batao

```
def ways(x, n, i):  
    if x == 0: return 1  
    if x < 0: return 0  
    if i**n > x: return 0  
    return ways(x-i**n, n, i+1) + ways(x, n, i+1)  
  
print(ways(25, 2, 1))
```

Answer: 2 Ways: $3^2+4^2=25$, $5^2=25$. Two ways.

Q20: Output batao

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    return ways(x-i**n, n, i) + ways(x, n, i+1)  
  
print(ways(4, 2, 1))
```

Answer: Infinite recursion / Stack overflow `ways(x-i\*\*n, n, i)` mein i same rehta hai, kabhi base case hit nahi hoga agar $x \neq 0$. Infinite loop.

Debugging Questions (21-30)

Q21: Bug find karo

```
def powerSum(X, N):
    def helper(x, n, i):
        if i**n > x:
            return 0
        if x == 0:
            return 1
        return helper(x-i**n, n, i+1) + helper(x, n, i+1)
    return helper(X, N, 0)
```

Answer Do bugs: 1. Base case order galat - `i\*\*n > x` pehle check ho raha, `x==0` baad mein. `helper(0,2,100)` galat answer dega. 2. Starting index 0 - $0^n = 0$, fayda nahi, aur recursion issues create kar sakta hai. 1 se start karo.

Q22: Ye code kya galat hai?

```
def ways(x, n, i=1):
    if x == 0:
        return True
    if i**n > x:
        return False
    return ways(x-i**n, n, i+1) + ways(x, n, i+1)
```

Answer Return type mismatch. `True + False` Python mein 1 deta hai (True=1, False=0), lekin semantically galat hai. Integers return karo: 1 aur 0.

Q23: Bug find karo

```
def ways(x, n, i):
    if x == 0:
        return 1
    if i**n > x:
        return 0
    include = ways(x - i**n, n, i+n)
```

```
skip = ways(x, n, i+1)
return include + skip
```

Answer `i+n` galat hai. Agar n=2: 1,3,5,7... skip ho jayenge 2,4,6... Numbers miss honge. `i+1` hona chahiye.

Q24: Recursion kabhi khatam kyun nahi hoti?

```
def ways(x, n, i):
    if x == 0:
        return 1
    include = ways(x - i*n, n, i+1)
    skip = ways(x, n, i)
    return include + skip
```

Answer Skip call mein `i` same hai, increment nahi ho raha. Isse infinite recursion hogi skip branch mein. `ways(x, n, i+1)` hona chahiye.

Q25: Output predict karo aur bug batao

```
def ways(x, n, i=1, memo={}):
    if x == 0: return 1
    if i*n > x: return 0
    if (x,i) in memo: return memo[(x,i)]
    memo[(x,i)] = ways(x-i*n, n, i+1) + ways(x, n, i+1)
    return memo[(x,i)]

print(ways(10,2))
print(ways(100,2))
```

Answer Pehla call sahi dega (1). Doosra call galat de sakta hai kyunki `memo={}` mutable default argument hai - pehli call ka data doosri call mein bhi rahega! Har call ke liye fresh memo chahiye.

Coding Questions (31-45)

Q31: Power sum function likho jo recursive ho.

```
def powerSum(X, N):  
    def helper(rem, n, start):  
        if rem == 0:  
            return 1  
        if start**n > rem:  
            return 0  
        return helper(rem-start**n, n, start+1) + helper(rem, n, start+1)  
    return helper(X, N, 1)
```

Q32: Saare valid combinations print karne wala function likho.

```
def printAllCombos(X, N):  
    def backtrack(rem, n, start, combo):  
        if rem == 0:  
            print(combo)  
            return  
        if start**n > rem:  
            return  
        combo.append(start)  
        backtrack(rem-start**n, n, start+1, combo)  
        combo.pop()  
        backtrack(rem, n, start+1, combo)  
    backtrack(X, N, 1, [])
```

Q33: Aise combinations count karo jahan exactly K numbers use ho.

```
def countWithK(X, N, K):  
    def helper(rem, n, start, count):  
        if rem == 0 and count == K:  
            return 1  
        if rem < 0 or count > K:  
            return 0  
        if start**n > rem:  
            return 0  
        return (helper(rem-start**n, n, start+1, count+1) +
```

```

        helper(rem, n, start+1, count))
    return helper(X, N, 1, 0)

```

Q34: Minimum kitne numbers chahiye X banane ke liye?

```

def minNumbers(X, N):
    def helper(rem, n, start):
        if rem == 0:
            return 0
        if start**n > rem:
            return float('inf')
        include = 1 + helper(rem-start**n, n, start+1)
        skip = helper(rem, n, start+1)
        return min(include, skip)
    ans = helper(X, N, 1)
    return ans if ans != float('inf') else -1

```

Q35: Check karo ki kya X ko represent kar sakte hain (boolean return).

```

def canRepresent(X, N):
    def helper(rem, n, start):
        if rem == 0:
            return True
        if start**n > rem:
            return False
        return helper(rem-start**n, n, start+1) or helper(rem, n, start+1)
    return helper(X, N, 1)

```

Logic Questions (46-50)

Q46: Agar N=2 aur X=1000 ho to maximum recursion depth kitni hogi?

Answer Maximum $i = \text{floor}(\text{sqrt}(1000)) \approx 31$. So maximum depth ≈ 31 calls. Bahut safe hai (Python default recursion limit ~ 1000).

Q47: Is problem mein duplicate combinations count kyun nahi hote?

Answer Kyunki hum numbers ko strictly increasing order mein consider karte hain ($i, i+1, i+2, \dots$). Agar hum 1^2+3^2 combination lenge to 1 pehle, phir 3. 3^2+1^2 kabhi consider nahi hoga kyunki 3 ke baad hum 4,5,... dekhenge, 1 nahi. Isliye har combination exactly ek baar count hota hai.

Q48: Kya ye problem subset sum problem jaisi hai? Similarities/Differences batao.

Answer Haan, exactly subset sum problem hai! Numbers ki jagah unki Nth powers ka set hai. Difference: Yahan numbers dynamic hain (i^n calculate karte hain) bajay predefined set ke. Pruning bhi natural hai ($i^n > x$).

Q49: Kya hum greedy approach se solve kar sakte hain? Kyun/Kyun nahi?

Answer Nahi. Greedy (sabse bada pehle lo) kaam nahi karega. Example: $X=10, N=2$. Greedy: $3^2=9$, remaining $1=1^2$. Ye to kaam kar gaya. Lekin $X=18$: Greedy $4^2=16$, remaining 2, no square. Lekin solution hai: 3^2+3^2 ? No, unique. Actually $18 = 3^2+?$ No. 18 has no solution. Another example: $X=12$, greedy $3^2=9$, remaining 3 (no square). But 2^2+2^2 ? No unique. $12 = 2^2+?$ No. No solution. But subset sum jaise problems mein greedy generally fail hota hai.

Q50: Explain how pruning reduces the search space.

Answer `if  $i^{**n} > x$ : return 0` pruning hai. Ye check karta hai ki agar current number ki power remaining sum se zyada hai, to aage ke sab numbers (jo aur bade hain) ki power bhi zyada hogi. Poori branch ko bina explore kiye discard kar dete hain. Isse tree ka size drastically kam ho jata hai. Example: $X=100, N=2$ mein $i=11$ ke baad sab branches automatic prune ho jaati hain.

Chapter 16: Practice Problems (50 Problems)

Easy Problems (1-20)

1. **Sum of first N squares:** $1^2+2^2+\dots+N^2$ recursively nikalna
 2. **Sum of first N cubes:** $1^3+2^3+\dots+N^3$ recursively
 3. **Print numbers 1 to N** using recursion
 4. **Print numbers N to 1** using recursion
 5. **Factorial** using recursion
 6. **Power function:** a^b calculate karo recursively
 7. **Sum of digits** of a number recursively
 8. **Count digits** in a number recursively
 9. **Check if number is perfect square:** recursive approach
 10. **Print all squares $\leq X$:** recursive version
 11. **Print all cubes $\leq X$:** recursive
 12. **Is X a perfect power?:** Check if $X = a^N$ for some a
 13. **Find a such that $a^N = X$,** if exists
 14. **Count numbers whose Nth power $\leq X$**
 15. **Print first K numbers** whose Nth power $\leq X$
 16. **Sum of Nth powers** from 1 to K
 17. **Maximum a** such that $a^N \leq X$
 18. **Print valid base numbers** for given X and N
 19. **Check if X can be written as sum of two squares** (not necessarily unique)
 20. **Count ways to write X as sum of TWO unique squares**
-

Medium Problems (21-40)

21. **Count ways to represent X as sum of unique squares** (N=2 fixed)
22. **Count ways to represent X as sum of unique cubes** (N=3 fixed)

23. **Print all ways to represent X as sum of unique squares**
 24. **Print all ways to represent X as sum of unique cubes**
 25. **Find the combination with minimum count of numbers**
 26. **Find the combination with maximum count of numbers**
 27. **Check if there exists a combination with exactly K numbers**
 28. **Count combinations with exactly K numbers**
 29. **Find the combination with the smallest largest number**
 30. **Find the combination with the largest smallest number**
 31. **Sum of Nth powers - memoized version** (with caching)
 32. **Check if X can be represented using only odd numbers' powers**
 33. **Check if X can be represented using only even numbers' powers**
 34. **Count ways using numbers starting from M** (not necessarily 1)
 35. **Check if representation exists without using number K**
 36. **Count ways without using number K**
 37. **Find if X can be represented as sum of exactly TWO Nth powers**
 38. **Count combinations where sum of base numbers is also X** (sum of bases = X)
 39. **Return the lexicographically smallest combination** (as list of numbers)
 40. **Return the lexicographically largest combination**
-

Hard Problems (41-50)

41. **Power Sum with bounded K:** Count ways using AT MOST K numbers
 42. **Power Sum with repetition allowed:** Each number can be used multiple times
 43. **Power Sum with both squares and cubes:** Numbers can be raised to EITHER 2 or 3
 44. **Find X that can be represented in maximum ways** for given N and range
 45. **Power sum for multiple targets:** Given list of X values, solve for each
 46. **Minimum X that can be represented in exactly W ways** for given N
 47. **Power sum with constraints:** Some numbers are forbidden
 48. **Count ways modulo M** (for large results)
 49. **Power sum DP solution:** Bottom-up approach
 50. **Generalized power sum:** Given custom set of numbers, count subset sum equal to X
-

Solutions for Practice Problems (Selected Few)

Problem 19: Check if X can be written as sum of two squares

```
def sum_of_two_squares(x):  
    for a in range(1, int(x**0.5) + 1):  
        b_sq = x - a*a  
        b = int(b_sq**0.5)  
        if b*b == b_sq and b != a: # b!=a for uniqueness  
            return True, (a, b)  
    return False, None
```

Problem 25: Minimum count combination

```
def min_count_combo(X, N):  
    def helper(rem, n, start):  
        if rem == 0:  
            return 0 # 0 more numbers needed  
        if start**n > rem:  
            return float('inf')  
        include = 1 + helper(rem - start**n, n, start + 1)  
        skip = helper(rem, n, start + 1)  
        return min(include, skip)  
  
    ans = helper(X, N, 1)  
    return ans if ans != float('inf') else -1
```

Problem 41: At most K numbers

```
def count_at_most_K(X, N, K):  
    def helper(rem, n, start, count):  
        if rem == 0:  
            return 1  
        if count >= K:  
            return 0  
        if start**n > rem:  
            return 0  
        include = helper(rem - start**n, n, start + 1, count + 1)  
        skip = helper(rem, n, start + 1, count)
```

```
    return include + skip
return helper(X, N, 1, 0)
```

Problem 49: DP Solution

```
def power_sum_dp(X, N):
    # Generate all valid powers
    powers = []
    i = 1
    while i**N <= X:
        powers.append(i**N)
        i += 1

    # DP table: dp[i][j] = ways to make sum j using first i powers
    dp = [[0]*(X+1) for _ in range(len(powers)+1)]
    dp[0][0] = 1

    for i in range(1, len(powers)+1):
        for j in range(X+1):
            # Skip current power
            dp[i][j] = dp[i-1][j]
            # Include current power (if possible)
            if j >= powers[i-1]:
                dp[i][j] += dp[i-1][j - powers[i-1]]

    return dp[len(powers)][X]
```

Chapter 17: Cheat Sheet (One Page Summary)

The Power Sum Problem - Complete Cheat Sheet

Problem Statement

Count ways to represent X as sum of **unique** Nth powers of natural numbers.

Function Template

```
def ways(remaining, power, current_num):
    # BASE CASES (order matters!)
    if remaining == 0:          # ✓ Found a way
        return 1
    if current_num**power > remaining: # X Can't proceed
        return 0

    # RECURSIVE CASES
    include = ways(remaining - current_num**power, power, current_num + 1)
    skip = ways(remaining, power, current_num + 1)

    return include + skip
```

Key Rules

Rule	Explanation
i+1 always	For uniqueness, never i, i+n, i+2
n constant	Power never changes
Return integers	1 for success, 0 for failure
Base case order	$x==0$ BEFORE $i^n > x$
Start from i=1	Not 0 ($0^n=0$ causes issues)

Common Mistakes

- ✗ return True/False instead of 1/0
- ✗ i not incrementing (infinite recursion)
- ✗ Wrong base case order
- ✗ Mutable default arguments `def func(lst=[])`

- ❌ Missing $i^n > x$ check

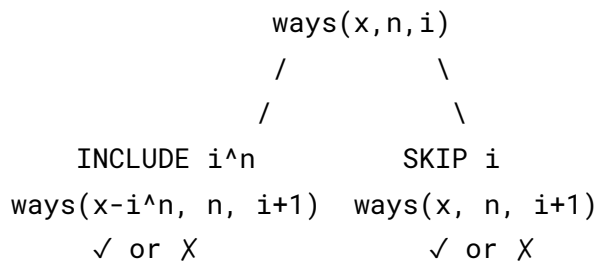
Complexity

- **Time:** $O(2^M)$ worst, where $M = \lfloor X^{(1/N)} \rfloor$ (pruning makes it much smaller)
- **Space:** $O(M)$ for recursion stack

Memory Tricks

- **"IPS"** Rule: **I**ncrement, **P**ower-constant, **S**tart-from-1
- **Base Case order:** **S**uccess before **F**ailure (SBF)
- **Two choices:** **I**nclude or **S**kip (I-S)

Decision Tree Pattern



Backtracking Template (for printing combinations)

```

def backtrack(rem, n, i, combo):
    if rem == 0: print(combo); return
    if i**n > rem: return
    combo.append(i)                # CHOOSE
    backtrack(rem - i**n, n, i+1, combo) # EXPLORE
    combo.pop()                    # UNCHOOSE
    backtrack(rem, n, i+1, combo)    # EXPLORE OTHER

```

Sample Input/Output

X	N	Ways	Combinations
10	2	1	1^2+3^2
13	2	1	2^2+3^2
25	2	2	$3^2+4^2, 5^2$
100	2	3	$6^2+8^2, 1^2+3^2+4^2+5^2+7^2, 10^2$
100	3	1	$1^3+2^3+3^3+4^3$

Quick Debug Checklist

- [] Is $x==0$ checked first?
- [] Is $i*n > x$ checked?
- [] Is i incremented by 1?
- [] Is n unchanged?
- [] Return type is integer?
- [] Starting $i = 1$?

When to Use What

- **Counting ways:** Simple recursion (no list needed)
- **Printing combinations:** Backtracking with list
- **Finding one solution:** Return True/False, stop on first
- **Optimization (min/max):** Track best, compare at leaves

Congratulations!

Aapne **Sum of Nth Powers of Unique Natural Numbers** problem ko zero se hero level tak seekh liya!

Aap ab kar sakte hain:

- Problem ko real-life examples se samajhna
- Recursion tree draw karna
- Dry run karna
- Code likhna
- Debug karna
- Interview questions answer karna
- Similar problems solve karna

Yaad rakho: Practice makes perfect. Jitna zyada dry run karoge, utna better samajh aaega.

Happy Coding! 🚀 # Sum of Nth Powers of Unique Natural Numbers

Ek Complete Hinglish Guide Book

Chapter 1: Problem Ko Samajhna (Understanding the Problem)

Real-Life Story Se Shuru Karte Hain

Maan lo aapke paas ek bag hai. Aapko us bag mein kuch kitabein rakhni hain. Condition ye hai ki:

- Har kitaab ka weight perfect square hona chahiye (jaise 1 kg, 4 kg, 9 kg, 16 kg...)
- Total weight exactly 10 kg hona chahiye
- Har kitaab ka weight alag-alag hona chahiye (duplicate allowed nahi)

Ab aap sochiye - kaun kaun si kitabein chunenge?

Possible combinations:

- $1^2 + 3^2 = 1 + 9 = 10$ ✓ (1 aur 3 chuna)
- $1^2 + 2^2 + \dots$ (ye 10 se kam ya zyada ho jayega)

Yehi basic idea hai “**Sum of Nth Powers**” problem ka!

Problem Statement (Simple Words Mein)

Humein batana hai ki **kitne different ways** hain jisme hum **unique natural numbers** ki **Nth power** ko jod kar **exactly X** bana sakte hain.

Keywords Ko Samjhte Hain:

1. Sum of Nth Powers

“Sum” matlab jod. “Nth Power” matlab kisi number ko N baar multiply karna.

Examples:

- Sum of 2nd powers (squares): $1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$
- Sum of 3rd powers (cubes): $1^3 + 2^3 = 1 + 8 = 9$

2. Unique Natural Numbers

- **Natural Numbers:** 1, 2, 3, 4, 5, ... (0 include nahi hota generally, lekin yahan 1 se start)
- **Unique:** Har number ek baar hi use kar sakte hain. Agar 2^2 use kiya to dobara 2^2 use nahi kar sakte.

Iska matlab: $\{1^2, 1^2, 2^2\}$ ✗ GALAT (1 do baar aaya)

Iska matlab: $\{1^2, 2^2, 3^2\}$ ✓ SAHI

3. Target Number X

Yeh woh number hai jo humein exactly achieve karna hai. Jaise $X=10$ ka matlab humein powers ka sum exactly 10 banana hai.

4. Power N

Yeh batata hai ki kis power ka use karna hai.

- $N=2$: squares use karenge ($1^2, 2^2, 3^2, 4^2, \dots$)
- $N=3$: cubes use karenge ($1^3, 2^3, 3^3, \dots$)
- $N=4$: fourth powers ($1^4, 2^4, 3^4, \dots$)

5. Ways (Tareeke)

Different combinations kitne possible hain. Order matter nahi karta.

$\{1^2, 3^2\}$ aur $\{3^2, 1^2\}$ same way consider hoga.

Detailed Examples Se Samjhte Hain

Example 1: $X = 10, N = 2$

Hindi: Humein 10 banana hai squares ke sum se, unique numbers use karte hue.

Possible squares: $1^2=1, 2^2=4, 3^2=9, 4^2=16$ ($16 > 10$, to 4 se aage nahi jayenge)

Check karte hain combinations:

- Sirf $1^2 = 1 \neq 10$ ✗
- Sirf $2^2 = 4 \neq 10$ ✗
- Sirf $3^2 = 9 \neq 10$ ✗
- $1^2 + 2^2 = 1+4 = 5 \neq 10$ ✗
- $1^2 + 3^2 = 1+9 = 10$ ✓ **MIL GAYA!** (Way 1)
- $2^2 + 3^2 = 4+9 = 13 \neq 10$ ✗
- $1^2 + 2^2 + 3^2 = 1+4+9 = 14 \neq 10$ ✗

Answer: 1 way ($1^2 + 3^2$)

Example 2: $X = 13, N = 2$

Squares: 1, 4, 9, 16(>13, stop)

Check karte hain:

- $1^2 + 2^2 + 3^2 = 1+4+9 = 14 > 13$ ✗
- $2^2 + 3^2 = 4+9 = 13$ ✓ **WAY 1**
- $1^2 + ? =$ sochiye... $1^2 + 2^2 = 5$, $1^2 + 3^2 = 10$, $1^2 + 4^2 = 17$... nahi ban raha

Kya $1^2 + 2^2 +$ kuch aur? $1+4=5$, $13-5=8$, kya 8 kisi ka perfect square hai? 2.8^2 ... nahi.

Answer: 1 way ($2^2 + 3^2$)

Example 3: X = 100, N = 2

Ye thoda bada example hai. Squares: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100

Possible ways:

- $10^2 = 100$ ✓ (Way 1)
- $6^2 + 8^2 = 36 + 64 = 100$ ✓ (Way 2)
- $1^2 + 3^2 + 4^2 + 5^2 + 7^2 = 1+9+16+25+49 = 100$ ✓ (Way 3)

Aur bhi combinations ho sakti hain! Isko manual count karna mushkil hai - isliye humein programming ki zaroorat hai.

Example 4: X = 100, N = 3

Cubes: $1^3=1$, $2^3=8$, $3^3=27$, $4^3=64$, $5^3=125(>100, \text{stop})$

Check karein:

- $4^3 = 64 \neq 100$ ✗
- $3^3 + 4^3 = 27+64 = 91 \neq 100$ ✗
- $2^3 + 3^3 + 4^3 = 8+27+64 = 99 \neq 100$ ✗
- $1^3 + 2^3 + 3^3 + 4^3 = 1+8+27+64 = 100$ ✓ **WAY 1**

Answer: 1 way ($1^3 + 2^3 + 3^3 + 4^3$)

Visual Representation

Maan lo aapke paas numbers hain: 1, 2, 3, 4, ...

Har number ke saath ek choice hai: **lena hai ya nahi lena hai**

Number 1: [Lena] ya [Nahi Lena]

Number 2: [Lena] ya [Nahi Lena]

Number 3: [Lena] ya [Nahi Lena]

...

Har decision ke baad aage badhte hain. Isse **Decision Tree** kehte hain.

Is problem mein:

- **Lena** = current number ki power ko sum mein add karo
- **Nahi Lena** = skip karo, agle number par jao

Problem Ki Key Points (Summary)

1. **Input:** Do numbers - X (target sum) aur N (power)
2. **Output:** Kitne ways hain X ko unique natural numbers ki Nth powers ke sum ke roop mein likhne ke
3. **Rules:**
 - Har number sirf ek baar use kar sakte hain (unique)
 - Powers ko jodna hai
 - Sum exactly X ke barabar hona chahiye
 - Natural numbers 1 se start hote hain (kabhi kabhi 0 se, lekin yahan 1 se)
 - Order matter nahi karta

Exercise (Aap Try Karein)

Pen-paper le kar ye solve karein:

1. X = 5, N = 2 (Hint: squares 1,4 use karo)
2. X = 25, N = 2
3. X = 9, N = 3
4. X = 12, N = 2

Answers check karne ke baad aage badhenge.

Is chapter mein humne problem ko real-life example se samjha. Aage badhne se pehle ensure karein ki aap "unique natural numbers", "Nth power", aur "ways" ka concept clear hai.

Chapter 2: Powers Ko Samajhna (Understanding Powers)

Power Kya Hai? (What is Power?)

Jab hum kisi number ko baar-baar multiply karte hain, use **power** kehte hain.

Real-Life Analogy: Paper Fold Karna

Sochiye aapke paas ek paper hai. Usse fold karne par:

- 1 fold: 2 layers ($2^1 = 2$)
- 2 folds: 4 layers ($2^2 = 2 \times 2 = 4$)
- 3 folds: 8 layers ($2^3 = 2 \times 2 \times 2 = 8$)
- N folds: 2^N layers

Har baar aap previous result ko 2 se multiply kar rahe ho. Yehi power ka concept hai.

Mathematical Definition

a^n ka matlab: a ko n baar multiply karo

$$a^n = a \times a \times a \times \dots \times a \text{ (n times)}$$

Examples:

- $2^2 = 2 \times 2 = 4$ (2 ka square)
- $2^3 = 2 \times 2 \times 2 = 8$ (2 ka cube)
- $3^2 = 3 \times 3 = 9$
- $3^3 = 3 \times 3 \times 3 = 27$
- $5^4 = 5 \times 5 \times 5 \times 5 = 625$

Terminology (Naam Yaad Rakho)

- **Base (a):** Jis number ki power nikalni hai
- **Exponent/Power (n):** Kitni baar multiply karna hai
- **Result (a^n):** Final value

Jaise 2^3 mein:

- Base = 2
- Exponent = 3
- Result = 8

Different Powers Ki Samajh

Square (Power 2 - N=2)

Kisi number ka square = number $\times$ number

$$\begin{aligned}1^2 &= 1 \times 1 = 1 \\2^2 &= 2 \times 2 = 4 \\3^2 &= 3 \times 3 = 9 \\4^2 &= 4 \times 4 = 16 \\5^2 &= 5 \times 5 = 25 \\6^2 &= 36 \\7^2 &= 49 \\8^2 &= 64 \\9^2 &= 81 \\10^2 &= 100\end{aligned}$$

Square numbers: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, ...

Visual: Agar aapke paas 3×3 ka grid hai, to usme total 9 cells hain. Isliye $3^2 = 9$.

Cube (Power 3 - $N=3$)

Number $\times$ number $\times$ number

$$1^3 = 1 \times 1 \times 1 = 1$$

$$2^3 = 2 \times 2 \times 2 = 8$$

$$3^3 = 3 \times 3 \times 3 = 27$$

$$4^3 = 4 \times 4 \times 4 = 64$$

$$5^3 = 5 \times 5 \times 5 = 125$$

Cube numbers: 1, 8, 27, 64, 125, 216, 343, 512, 729, 1000

Visual: Agar aapke paas $3 \times 3 \times 3$ ka cube hai, usme 27 chhote cubes hain. Isliye $3^3 = 27$.

Higher Powers

$$N=4: 1^4=1, 2^4=16, 3^4=81, 4^4=256$$

$$N=5: 1^5=1, 2^5=32, 3^5=243$$

Jaise N badhta hai, values bahut tezi se badhti hain. Ye observation important hai - isi wajah se humara search space chhota rehta hai.

Python Mein Powers Calculate Karna

Python mein power nikalne ke **do popular tarike** hain:

Method 1: ****** Operator

```
# ** operator use karna
result = 2 ** 3    # 23 = 8
print(result)      # Output: 8
```

```

square = 5 ** 2    # 52 = 25
cube = 5 ** 3      # 53 = 125

# Dynamic power
base = 3
power = 4
result = base ** power    # 34 = 81
print(result)

```

Kab use karein? Jab aapko integer powers chahiye aur simple code likhna hai. `**` operator seedha-saada hai aur fast hai.

Method 2: math.pow() Function

```

import math

result = math.pow(2, 3)    # 23 = 8.0
print(result)              # Output: 8.0 (float!)

square = math.pow(5, 2)    # 25.0
print(square)

```

Dono Mein Difference (Important!)

Feature	** Operator	math.pow()
Return Type	Integer (agar dono integer)	Hamesha Float
Speed	Thoda fast	Thoda slow
Import	Zaroorat nahi	import math karna padega
Use case	Simple integer powers	Scientific calculations

Hamari problem ke liye konsa use karein?

```

# Ye better hai:
value = i ** n    # Integer return karega

```

```
# Ye avoid karein:
value = math.pow(i, n) # Float return karega, comparison mein problem
```

Kyun? Kyunki hum integer comparisons kar rahe hain:

```
if i**n == x:    # Integer comparison - safe
    ...

if math.pow(i, n) == x:    # Float comparison - risky (0.1 + 0.2 != 0.3 type is
    ...
```

Built-in pow() Function (Teesra Option)

```
result = pow(2, 3)    # 8
result = pow(2, 3, 10) # 23 % 10 = 8 (modulo ke saath)
```

Recommendation: Hamari problem mein `**` operator use karein. Simple, fast, aur integer return karta hai.

Powers Ki Properties Jo Humein Kaam Aayengi

Property 1: Monotonically Increasing

Jaise-jaise base badhta hai, power ki value bhi badhti hai.

Agar $a < b$, to $a^n < b^n$ (for positive numbers)

```
12 = 1
22 = 4    (4 > 1) ✓
32 = 9    (9 > 4) ✓
42 = 16   (16 > 9) ✓
```

Is property ki wajah se hum ek point ke baad search rok sakte hain.

Property 2: Power Ki Value Bahut Tezi Se Badhti Hai

For N=2: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100
For N=3: 1, 8, 27, 64, 125, 216, 343, 512, 729, 1000
For N=4: 1, 16, 81, 256, 625, 1296
For N=5: 1, 32, 243, 1024

Notice karo: N jitna bada, values utni tezi se badhti hain. Iska matlab humara recursion tree utna chhota hoga.

Practice Examples (Python Mein)

Example 1: Sabhi Squares Print Karo Jo X Se Chhote Ya Barabar Hon

```
def print_squares_upto(x):  
    """X se chhote ya barabar sabhi perfect squares print karo"""  
    i = 1  
    while i*i <= x:    # i*i is same as i**2  
        print(f"{i}^2 = {i*i}")  
        i += 1  
  
# Test  
print("Squares up to 20:")  
print_squares_upto(20)
```

Output:

```
1^2 = 1  
2^2 = 4  
3^2 = 9  
4^2 = 16
```

Example 2: Check If Number Is Perfect Power

```
def is_perfect_power(num, n):  
    """Check if num is nth power of some integer"""  
    # Brute force approach  
    i = 1  
    while i**n <= num:
```

```

        if i**n == num:
            return True, i
        i += 1
    return False, None

# Test
print(is_perfect_power(64, 2))    # True, 8 (82=64)
print(is_perfect_power(64, 3))    # True, 4 (43=64)
print(is_perfect_power(10, 2))    # False, None

```

Example 3: Generate Powers List

```

def generate_powers(x, n):
    """List of nth powers <= x"""
    powers = []
    i = 1
    while True:
        power = i ** n
        if power > x:
            break
        powers.append(power)
        i += 1
    return powers

# Test
print(generate_powers(50, 2))    # [1, 4, 9, 16, 25, 36, 49]
print(generate_powers(50, 3))    # [1, 8, 27]
print(generate_powers(100, 3))   # [1, 8, 27, 64]

```

Common Mistakes With Powers

Mistake 1: Power aur Multiplication Ko Confuse Karna

```

# Galat
2^3 = ? # Python mein ^ XOR operator hai, power nahi!

```

```
# Sahi
2 ** 3 = 8
```

Mistake 2: Zero Power

```
5 ** 0 = 1    # Koi bhi number ki power 0 = 1
10 ** 0 = 1
100 ** 0 = 1
```

Mistake 3: Negative Numbers

```
(-2) ** 2 = 4    # Negative × Negative = Positive
(-2) ** 3 = -8    # Odd power: negative hi rehta hai
```

Hamari problem mein sirf positive numbers se kaam hai, to ye tension nahi leni.

Problem-Specific Power Understanding

Hamari problem mein:

- Hum 1 se shuru karte hain (1^n , 2^n , 3^n , ...)
- Jab kisi number ki power X se badi ho jaye, hum wahan ruk jaate hain
- Sirf unique numbers use kar sakte hain, to ek number ki power ek hi baar use hogi

```
# Maximum number jiski power humein consider karni hai
max_num = int(X ** (1/N)) # N-th root of X
# Ya loop se:
i = 1
while i**N <= X:
    # i**N ko consider karo
    i += 1
```

Chapter Summary

- Power ka matlab: number ko baar-baar multiply karna
- ****** operator simple aur efficient hai integers ke liye
- Powers monotonically badhti hain - isliye search space limited hai
- Higher N ke liye search space aur chhota ho jata hai

Aage badhne se pehle: Ensure karein ki aap comfortably powers calculate kar sakte hain aur samajhte hain ki kyun hum sirf limited numbers consider karte hain.

Chapter 3: Programmer Ki Tarah Sochna (Thinking Like a Programmer)

Programming Sirf Code Likhna Nahi Hai

Programming = **Problem Solving + Code Writing**

Pehle dimaag mein solution clear hona chahiye, phir code likhna easy ho jata hai.

Real-Life Analogy: Ghar Banana

Jab ghar banta hai:

1. Pehle **architect sochta hai** - kitne rooms, kahan kitchen, kahan bathroom
2. Phir **blueprint** (naksha) banata hai
3. Phir **construction** shuru hoti hai

Aise hi programming mein:

1. Pehle **problem samjho**
2. Phir **solution ka structure socho** (algorithm design karo)
3. Phir **code likho**

Bina soche code likhna = bina naksha ke ghar banana = disaster!

Step-by-Step Thinking Process

Step 1: Problem Ko Break Karo (Divide and Conquer)

Badi problem ko chhoti-chhoti problems mein todo.

Hamari Problem: Kitne ways hain X ko unique Nth powers ke sum se banane ke?

Break Down:

1. Sabse pehle - kaun kaun si powers available hain?
2. Har power ke liye decision - leni hai ya nahi?
3. Sum track karo - exactly X achieve karna hai
4. Unique numbers maintain karo

Step 2: Choices Identify Karo

Har decision point par kya-kya choices hain?

Maan lo current number = 1, N = 2:

- **Choice 1:** $1^2 = 1$ ko sum mein add karo, phir aage badho
- **Choice 2:** 1^2 ko skip karo, seedha aage badho

Yehi choices har number ke liye repeat hongii.

Step 3: Constraints Identify Karo (Rukna Kab Hai?)

Kab rukna hai decisions lena?

- **Success:** Sum exactly X ke barabar ho jaye → Ek way mil gaya!
- **Failure 1:** Sum X se zyada ho jaye → Ye path galat hai, wapas jao
- **Failure 2:** Current number ki power X se zyada ho → Aage ke sab numbers bhi zyada denge, wapas jao
- **Failure 3:** Saare numbers check ho gaye, sum X nahi bana → Koi way nahi mila

Manual Simulation (Pen-Paper Exercise)

Problem: $X = 10$, $N = 2$

Available Powers (jo 10 se chhoti ya barabar hain):

$$1^2 = 1$$

$$2^2 = 4$$

$$3^2 = 9$$

($4^2 = 16 > 10$, isliye 4 aur usse aage nahi lenge)

Ab Systematic Exploration

Starting: sum = 0, number = 1

Number 1 (power=1):

```

├─ Include 1: sum=1, next number=2
|   ├─ Include 2: sum=1+4=5, next=3
|   |   ├─ Include 3: sum=5+9=14 > 10 X (over, backtrack)
|   |   └─ Skip 3: sum=5, next=4 ( $4^2=16 >$  remaining 5, stop)
|   └─ Skip 2: sum=1, next=3
|       └─ Include 3: sum=1+9=10 ✓ FOUND!
|       └─ Skip 3: sum=1, next=4 (stop)
└─ Skip 1: sum=0, next=2
    ├─ Include 2: sum=4, next=3
    |   ├─ Include 3: sum=4+9=13 > 10 X
    |   └─ Skip 3: sum=4 (stop)
    └─ Skip 2: sum=0, next=3
        └─ Include 3: sum=9, next=4 (stop)
            └─ Skip 3: sum=0 (stop)
  
```

Way found: $1^2 + 3^2 = 10$

Is systematic exploration ko **Recursion Tree** ya **Decision Tree** kehte hain.

Programmer Ki Soch Ka Template

Har problem ke liye ye pattern follow karo:

1. State Define Karo

“State” matlab - kahan ho abhi? Kya-kya information hai?

Hamari problem mein state:

- `remaining_sum` : X mein se kitna baki hai (ya current sum kitna hai)
- `current_num` : Kaun sa number consider kar rahe hain
- `power_values` : N decide ho chuka hai, change nahi hoga

2. Choices Per State

Har state se kahan-kahan ja sakte hain?

- `State(remaining, num)` se:
 - Agar num^N include karein: `State(remaining - num^N, num+1)`
 - Agar skip karein: `State(remaining, num+1)`

3. Base Cases (Rukne Ki Conditions)

Kab aur kaise rukna hai?

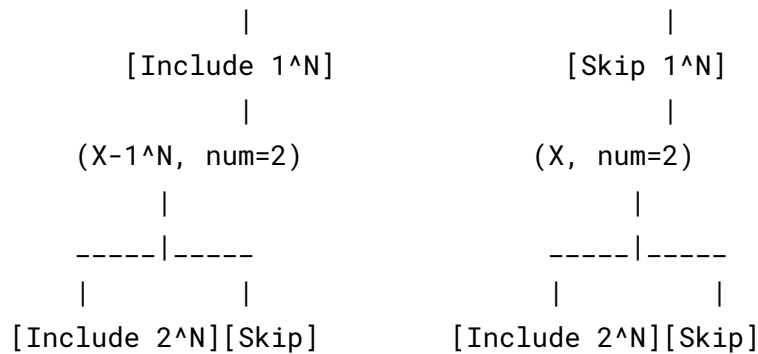
- `remaining == 0` : Ek valid way mil gaya $\rightarrow$ return 1
- `num^N > remaining` : Aage nahi ja sakte $\rightarrow$ return 0
- `remaining < 0` : Already zyada ho gaya $\rightarrow$ return 0

Visual Representation: Decision Tree Sochna

```

      (remaining=X, num=1)
        |
-----|-----

```



Yahi tree grow hota hai jab tak ya to sum 0 ho jaye (success) ya num^N remaining se bada ho jaye (failure).

Transition From Thinking To Code

Sochne ke baad code likhne ka structure:

```

def solve(remaining, n, current_num):
    # Base cases (rukna kab hai)
    if remaining == 0:    # Target achieve!
        return 1
    if current_num**n > remaining: # Aage nahi ja sakte
        return 0

    # Recursive cases (choices explore karo)
    # Choice 1: Include current number
    include = solve(remaining - current_num**n, n, current_num + 1)

    # Choice 2: Skip current number
    skip = solve(remaining, n, current_num + 1)

    # Total ways = include ke ways + skip ke ways
    return include + skip
  
```

Exercise: Sochne Ki Practice

In problems ko bina code likhe, sirf pen-paper se solve karo:

1. **X = 5, N = 2**: Kitne ways hain? Tree draw karo.
2. **X = 13, N = 2**: Kitne ways hain? Tree draw karo.
3. **X = 9, N = 3**: Kitne ways hain? Tree draw karo.

Hint har problem ke liye:

1. Pehle available powers list karo
2. Phir systematically har combination try karo
3. Valid combinations count karo

Important Thinking Habits

Habit 1: Small Examples Se Shuru Karo

Bade X, N ke liye seedha code mat likho. Pehle chhote examples manually solve karo.

Habit 2: Edge Cases Socho

- $X = 1, N = 2$: $1^2 = 1 \rightarrow 1$ way
- $X = 2, N = 2$: $1^2 = 1$, need 1 more lekin next $2^2=4 > 2 \rightarrow 0$ ways
- $X = 0, N = \text{anything}$: Kya karna hai? (Hint: empty sum = 0)

Habit 3: Pattern Pehchano

Notice karo ki problem structure same rehta hai, sirf X aur N change hote hain. Isliye ek general solution possible hai.

Chapter Summary

- **Socho pehle, code baad mein**
- Problem ko chhote pieces mein break karo
- Choices aur constraints clearly identify karo
- Pen-paper se manually solve karke logic samjho

- Phir code mein translate karo

Key Takeaway: Programmer ka dimaag algorithm sochta hai, syntax baad ki baat hai.

Chapter 4: Simple Loops Kyun Kaam Nahi Kareng

Introduction: Loops Ki Limitations

Bahut saare beginners sochte hain - "Loops se sab kuch ho jata hai!" Lekin aisa nahi hai. Kuch problems aisi hain jahan loops fail ho jate hain ya bahut complicated ho jate hain.

Hamari problem aisi hi hai. Samjhte hain kyun.

Approach 1: Simple For Loop (Fail)

Koshish: Ek variable sum karte jao

```
def power_sum_loop1(x, n):  
    total = 0  
    ways = 0  
    for i in range(1, int(x**(1/n)) + 1):  
        total += i**n  
        if total == x:  
            ways += 1  
    return ways
```

Kyun fail? Ye sirf consecutive numbers ka sum check karega ($1^2 + 2^2 + 3^2 \dots$). Ye check nahi karega ki kuch numbers skip karke bhi sum ban sakta hai.

Example: $X = 10, N = 2$

- Loop: $1^2=1$, $1^2+2^2=5$, $1^2+2^2+3^2=14$ (10 kabhi aaya hi nahi)
- Actual answer: $1^2 + 3^2 = 10$ (2 skip karna pada)

Loop ne skip karna consider hi nahi kiya!

Approach 2: Nested Loops (Partial Solution)

Socho agar hum jante hain ki maximum 3 numbers tak use honge:

```
# Assume maximum 3 numbers needed
def power_sum_nested(x, n):
    ways = 0
    limit = int(x**(1/n)) + 1

    for i in range(1, limit):
        for j in range(i+1, limit):
            for k in range(j+1, limit):
                if i**n + j**n + k**n == x:
                    ways += 1
                print(f"Found: {i}**{n} + {j}**{n} + {k}**{n}")

    return ways
```

Kyun fail?

1. **Kitne nested loops lagaye?** Pata nahi kitne numbers lagenge. Kabhi 2, kabhi 5, kabhi 10. Fixed nested loops nahi likh sakte.
2. **Readability:** 10 nested loops likhna pagalpan hai.
3. **Maintainability:** Agar kuch change karna ho to?

Approach 3: While Loop With Backtracking Logic (Complicated)

```
def power_sum_while(x, n):
    ways = 0
    current_nums = [] # Stack banakar try karte hain
    current_sum = 0
```

```
i = 1
```

```
while i**n <= x or len(current_nums) > 0:  
    # Bahut complex logic likhna padega  
    # Kab push karna, kab pop karna, kab i increment karna  
    # Ye basically recursion reinvent kar raha hai  
    pass  
return ways
```

Ye possible hai lekin:

- Code bahut complex ho jayega
- Bug-prone hoga
- Recursion isko naturally handle karta hai

Fundamental Problem: Unknown Depth

Hamari problem ka fundamental challenge ye hai:

Humein nahi pata ki kitne numbers ki zaroorat padegi.

Kabhi 2 numbers se sum ban jayega:

- $1^2 + 3^2 = 10$

Kabhi 4 numbers lagenge:

- $1^3 + 2^3 + 3^3 + 4^3 = 100$

Kabhi 7 numbers lagenge:

- $1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 = 140$

Loops ke liye humein pehle se pata hona chahiye ki kitni depth tak jana hai. Recursion is problem ko naturally solve karta hai.

Visualization: Loop vs Recursion

Loop ka structure (Fixed depth):

```
for i1:
  for i2:
    for i3:
      # Fixed 3-level deep
```

Recursion ka structure (Variable depth):

```
check(num=1):
  check(num=2):
    check(num=3):
      check(num=4): # Jitni zaroorat utni depth
      ...
```

Recursion automatically manage karta hai ki kitni depth mein jana hai.

Comparison Table

Feature	Loops	Recursion
Depth known?	Required	Not required
Code complexity	High for unknown depth	Low
Readability	Poor for nested	Clear logic
Bug-prone	Yes (complex logic)	Less (simple logic)
Natural fit	For fixed iterations	For branching choices

When Loops Work (Counter Example)

Loops tab best hain jab:

- Fixed number of iterations ho
- Linear ya simple nested structure ho
- State maintain karna simple ho

Example jahan loop perfect hai:

```
# Sum of first N squares (fixed range, known iterations)
def sum_first_n_squares(n):
    total = 0
    for i in range(1, n+1):
        total += i**2
    return total
```

Transition To Recursion

Ab samajh aa gaya hoga ki humein recursion ki zaroorat kyun hai:

1. **Unknown depth** of choices
2. **Branching decisions** (include/skip)
3. **Need to backtrack** (try different combinations)
4. **Clean, readable code**

Agle chapter mein hum recursion ko bilkul basic se seekhenge, bilkul aise jaise pehli baar seekh rahe hon.

Chapter Summary

- Simple loops fixed number of elements consider karte hain, combinations nahi
- Nested loops tabhi kaam karte hain jab depth pehle se pata ho
- Hamari problem mein depth unknown hai - kitne bhi numbers lag sakte hain
- Recursion is problem ke liye natural solution hai

Remember: Right tool for the right job. Loops galat nahi hain, bas ye problem loops ke liye suitable nahi hai.

Chapter 5: Recursion - Bilkul Basic Se (Introduction to Recursion)

Recursion Kya Hai? (Real-Life Analogy)

Analogy 1: Russian Dolls (Matryoshka)

Sochiye aapke paas Russian dolls hain. Ek badi doll ke andar chhoti doll, uske andar aur chhoti... aise kai layers.

Jab aap dolls kholte hain:

1. Sabse badi doll kholo
2. Andar se chhoti doll nikalo
3. Usse kholo
4. Aur chhoti niklegi
5. ...aise chalta rehta hai
6. Jab last doll (sabse chhoti) aati hai - woh khulti nahi, solid hai

Yehi recursion hai! Har doll same structure hai (ek doll jo khulti hai), lekin chhoti hoti jaati hai. End mein ek doll aati hai jo “base case” hai.

Analogy 2: Countdown

Socho aap 5 se 1 tak countdown kar rahe ho:

- 5... (ruk kar socho: ab 4 se countdown karo)
- 4... (ruk kar socho: ab 3 se countdown karo)
- 3... (ruk kar socho: ab 2 se countdown karo)
- 2... (ruk kar socho: ab 1 se countdown karo)
- 1... (ye last hai, ruk jao)
- 2 done
- 3 done

4 done

5 done

Har step par same kaam ho raha hai: number bolo, phir chhote number ke liye same process repeat karo. Jab 1 aaye, ruk jao.

Recursion Ki Simple Definition

Recursion: Jab ek function khud ko call kare.

Ya aur simply: Ek aina jo aine ke saamne rakha ho - infinite reflections, lekin ek point par rukna zaroori hai.

Technical Definition:

Recursion ek programming technique hai jisme ek function apne aap ko chhote input ke saath call karta hai jab tak ek stopping condition (base case) na reach ho jaye.

Pehla Recursive Function (5 Minutes Mein)

Dekhte hain sabse simple recursive function:

```
def countdown(n):  
    """  
    Countdown karo n se 1 tak  
    """  
    # Base Case: Rukne ki condition  
    if n == 0:  
        print("Go!")  
        return  
  
    # Recursive Case  
    print(n)  
    countdown(n - 1)    # Khud ko chhote input se call  
    print(f"Returning from n={n}")
```



```
# Call the function  
countdown(5)
```

Output:

```
5  
4  
3  
2  
1  
Go!  
Returning from n=1  
Returning from n=2  
Returning from n=3  
Returning from n=4  
Returning from n=5
```

Kya Ho Raha Hai? (Line By Line)

1. `countdown(5)` call hua

- `n=5`, base case nahi ($5 \neq 0$)
- Print "5"
- `countdown(4)` call karo

2. `countdown(4)` ab execute ho raha hai

- `n=4`, base case nahi
- Print "4"
- `countdown(3)` call karo

3. `countdown(3)` :

- `n=3`
- Print "3"
- `countdown(2)` call karo

4. `countdown(2)` :

- $n=2$
- Print "2"
- `countdown(1)` call karo

5. `countdown(1)` :

- $n=1$
- Print "1"
- `countdown(0)` call karo

6. `countdown(0)` :

- **BASE CASE!** $n=0$
- Print "Go!"
- Return (function khatam, neech wale ko control wapas)

7. Control `countdown(1)` mein wapas:

- Print "Returning from $n=1$ "
- Function khatam

8. Control `countdown(2)` mein wapas:

- Print "Returning from $n=2$ "
- Function khatam

... aur aise hi 5 tak wapas aata hai.

Recursion Ke Do Zaroori Parts

Part 1: Base Case (Rukna Kab Hai?)

Ye sabse important part hai! Bina base case ke recursion infinite ho jayegi.

```
if n == 0:    # Base case
    return    # Stop condition
```

Real-life mein: Jab Russian doll ki last, solid doll mile - ruk jao, usse mat kholo.

Without base case:

```
def infinite_countdown(n):
    print(n)
    infinite_countdown(n-1) # Kabhi nahi rukega!
```

Part 2: Recursive Case (Problem Chhoti Karna)

Har recursive call mein problem thodi chhoti honi chahiye.

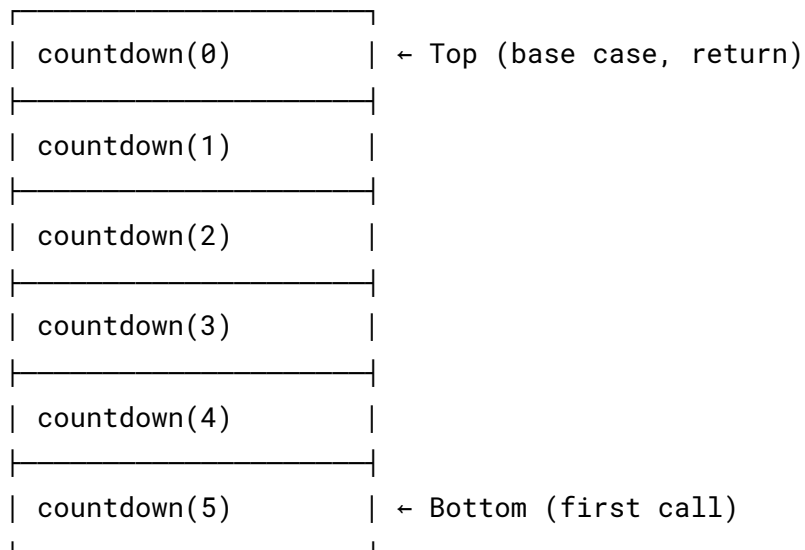
```
countdown(n-1) # Problem size kam ho raha hai (n → n-1)
```

Agar problem size kam nahi hua to base case kabhi reach nahi hoga.

Recursion Ka Execution Flow (Call Stack)

Jab function khud ko call karta hai, har call memory mein stack ki tarah store hoti hai.

Stack (neeche se upar):



Flow:

1. Calls stack mein add hoti jaati hain (push)
2. Base case tak jaate hain
3. Phir return hote hain (pop)
4. Har return ke saath stack se ek call remove hoti hai

Recursion Jo Value Return Kare (Important!)

Ab tak ka example sirf print kar raha tha. Lekin recursion aksar value return karta hai:

```
def sum_till_n(n):  
    """  
    1 + 2 + 3 + ... + n calculate karo recursively  
    """  
  
    # Base case  
    if n == 1:  
        return 1  
  
    # Recursive case  
    smaller_sum = sum_till_n(n - 1)    # Chhote problem ka answer  
    return n + smaller_sum            # Usme current number add karo  
  
# Usage  
print(sum_till_n(5))    # 15 (1+2+3+4+5)
```

Execution:

```
sum_till_n(5)  
= 5 + sum_till_n(4)  
= 5 + (4 + sum_till_n(3))  
= 5 + (4 + (3 + sum_till_n(2)))  
= 5 + (4 + (3 + (2 + sum_till_n(1))))  
= 5 + (4 + (3 + (2 + 1)))  
= 5 + (4 + (3 + 3))  
= 5 + (4 + 6)  
= 5 + 10  
= 15
```

Recursion Ke Fayde (For Our Problem)

1. **Code simple aur readable:** Include/skip ka logic clearly dikhta hai
2. **Unknown depth handle:** Automatically jitni depth chahiye utni le leta hai
3. **Backtracking natural:** Return karte waqt automatically previous state mein aate hain
4. **Every combination explore:** Tree structure naturally sab kuch cover karta hai

Recursion Ke Nuksan (Awareness)

1. **Memory:** Har call stack mein store hoti hai
2. **Speed:** Function calls ka overhead hota hai
3. **Stack overflow:** Bahut deep recursion stack overflow de sakti hai

Lekin hamari problem ke liye depth limited hai (kyunki powers jaldi badhti hain), isliye safe hai.

Important Rule: Recursion Design Karte Waqt

Rule 1: Maan Lo Recursion Kaam Karega

Jab `sum_till_n(n-1)` likhte hain, to maan lo ki ye sahi answer la dega. "How" ki tension mat lo.

Isse **Leap of Faith** kehte hain. Jaise induction hypothesis maths mein.

Rule 2: Base Case Perfect Rakho

Base case hi recursion ko infinite hone se bachata hai. Isko thoroughly test karo.

Rule 3: Problem Size Definitely Kam Ho

Har call mein ensure karo ki input base case ki taraf move kar raha hai.

Hamari Problem Ke Liye Recursion Structure

Ab samjhte hain ki hamari problem mein recursion kaise lagega:

```
def ways(x, n, current_num):  
    # BASE CASES  
    if x == 0:          # Target achieve kar liya  
        return 1  
    if current_num**n > x:  # Aage ki sab values badi hongi  
        return 0  
  
    # RECURSIVE CASES (Do choices)  
    # Choice 1: Current number include karo  
    include = ways(x - current_num**n, n, current_num + 1)  
  
    # Choice 2: Current number skip karo  
    skip = ways(x, n, current_num + 1)  
  
    # Total ways  
    return include + skip
```

Is structure ko detail mein aage ke chapters mein samjhenge.

Practice: Simple Recursion Problems

Problem 1: Factorial

```
def factorial(n):  
    """  
    n! = n × (n-1) × (n-2) × ... × 1  
    Base case: 0! = 1, 1! = 1  
    """  
    if n <= 1:      # Base case  
        return 1  
    return n * factorial(n - 1)
```

```
# Test
print(factorial(5))    # 120
```

Problem 2: Print Numbers from 1 to N

```
def print_1_to_n(n):
    """1 se n tak print karo"""
    if n == 0:
        return
    print_1_to_n(n - 1)    # Pehle 1 to n-1 print karo
    print(n)              # Phir n print karo

print_1_to_n(5)          # 1 2 3 4 5
```

Problem 3: Sum of Digits

```
def sum_of_digits(n):
    """123 ka sum = 1+2+3 = 6"""
    if n == 0:
        return 0
    return (n % 10) + sum_of_digits(n // 10)

print(sum_of_digits(123))    # 6
```

Chapter Summary

- Recursion = Function khud ko call kare chhote input ke saath
- Do parts: Base case (stop) + Recursive case (continue with smaller problem)
- Call stack automatically manage hota hai
- Leap of Faith rakho - maan lo recursion kaam karega
- Hamari problem ke liye perfect fit hai

Ab aap recursion ki basic samajh rakhate hain. Agle chapter mein hum recursion tree banayenge!

Chapter 6: Recursion Tree (Decision Tree)

Tree Kya Hai? (Computer Science Mein)

Tree ek structure hai jisme:

- Ek **root** (starting point) hota hai
- Root se **branches** (shaakhayein) nikalti hain
- Har branch ke end mein ya to **leaf** hota hai ya aur branches

Real-life mein jaise:

- Family tree (dada se shuru, phir unke bacche, phir unke bacche...)
- Folder structure (main folder, uske andar folders, aur andar...)
- Decision making (kya karein? option A, option B...)

Recursion Tree (Decision Tree) Kya Hai?

Recursion tree ek visual representation hai jo dikhata hai ki recursive function kaise different choices explore karta hai.

Hamari problem mein:

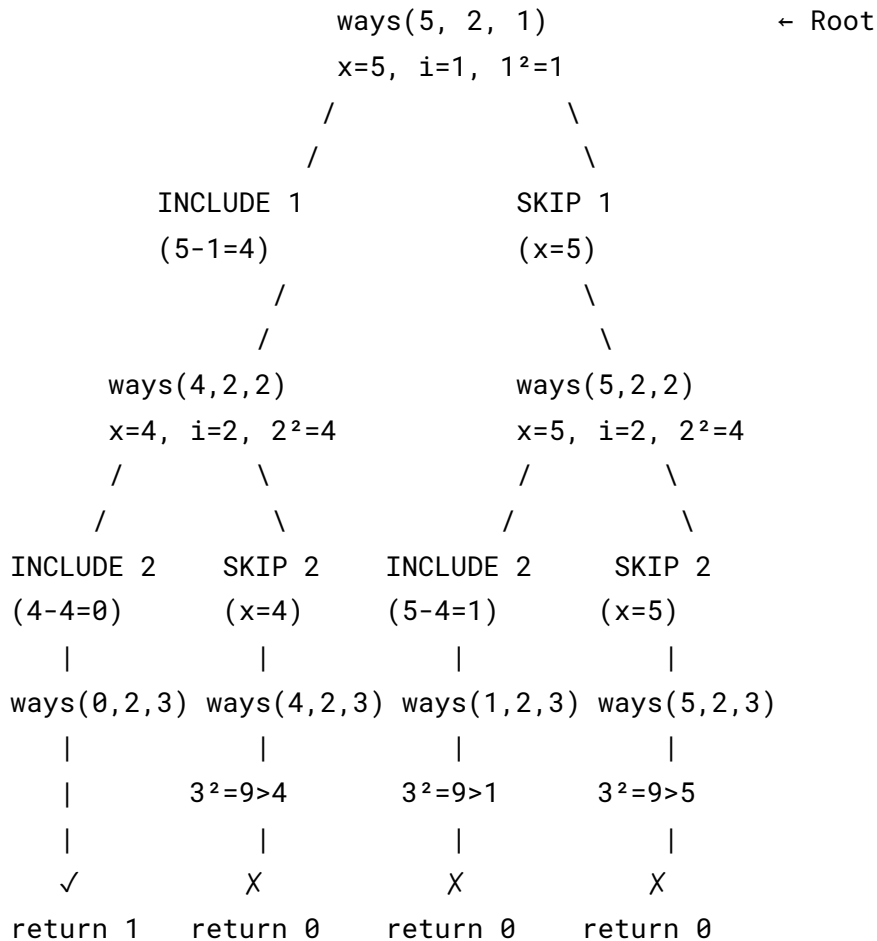
- Har **node** ek function call represent karta hai: `ways(x, n, i)`
- Har **branch** ek choice hai: "include" ya "skip"
- Har **leaf** ek base case hai: ya to success (return 1) ya failure (return 0)

Simple Example Se Tree Samjho

Problem: **X = 5, N = 2**

Powers available: $1^2=1$, $2^2=4$, $3^2=9$ ($9>5$, to 3 consider nahi karenge)

Tree Drawing



Tree Ko Padhna

1. **Root se start:** ways(5, 2, 1) - Humein 5 banana hai squares se, 1 se shuru
2. **Do choices:** 1 ko include karein ya skip karein
3. **Left branch (Include 1):** ways(4, 2, 2) - Ab 4 banana hai, 2 se shuru
4. **Wahan bhi do choices:** 2²=4 include karo ya skip
 - Include 2: ways(0, 2, 3) - **SUCCESS!** sum=0, return 1
 - Skip 2: ways(4, 2, 3) - 3²=9>4, aage nahi ja sakte, return 0
5. **Right branch (Skip 1):** ways(5, 2, 2) - 5 banana hai, 2 se shuru
 - Include 2: ways(1, 2, 3) - 3²=9>1, return 0
 - Skip 2: ways(5, 2, 3) - 3²=9>5, return 0

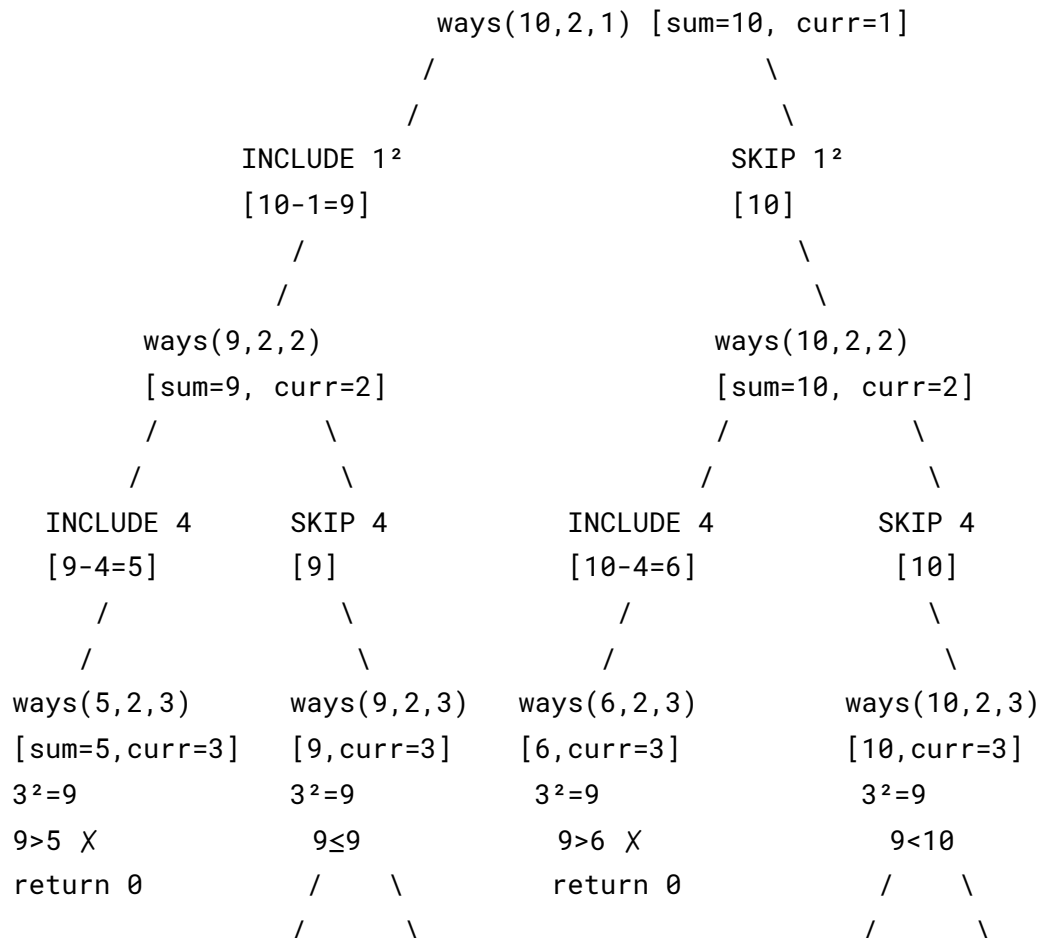
Result: 1 way (1²+2²=1+4=5)

Simple tree draw karne ka tarika:

- |— Left Child (Include)
- └— Right Child (Skip)

-  matlab left branch
-  matlab last branch (right)

Available: $1^2=1$, $2^2=4$, $3^2=9$



INCLUDE 9	SKIP 9	INCLUDE 9	SKIP 9
[9-9=0]	[9]	[10-9=1]	[10]
ways(0,2,4)	ways(9,2,4)	ways(1,2,4)	ways(10,2,4)
	4 ² =16>9		4 ² =16>10
✓	X	X	X
return 1	return 0	return 0	return 0

Ways Found:

1. Path: Include 1 → Skip 2 → Include 3 = $1^2 + 3^2 = 1 + 9 = 10$ ✓
2. Aur koi? Tree check karo... nahi. Total = 1 way.

Important Tree Concepts

1. Tree Ki Depth

Depth = root se leaf tak ki maximum length.

Is example mein depth = 4 (root se include-include... ya skip-skip...)

Tree depth limited hai kyunki:

- Har level par i badhta hai
- Kabhi na kabhi $i^2 > x$ ho jata hai
- Us point par branch ruk jaati hai

2. Branching Factor

Har node se kitni branches nikalti hain.

Hamari problem mein: **2 branches** (include/skip)

3. Leaf Nodes

Jahan decision khatam ho jaye.

Do type ke leaves:

- **Success leaf:** $x=0$ (target achieve) $\rightarrow$ return 1
- **Failure leaf:** $i^n > x$ (aage nahi ja sakte) $\rightarrow$ return 0

4. Path

Root se leaf tak ka complete route.

Har path ek unique combination hai.

- Include decisions ka set = ek combination
- Jaise path: Include 1, Skip 2, Include 3 $\rightarrow$ combination {1, 3}

Visualization: Tree Traversal Order

Function kis order mein nodes visit karta hai (Depth-First Search - DFS):

```
Order:  1
        / \
       2  5
      / \
     3  4
```

Visit order: 1, 2, 3, 4, 5

(Pehle left subtree poori explore karo, phir right)

Hamare example mein:

1. Root: ways(10,2,1)
2. Include 1: ways(9,2,2)
3. Include 2: ways(5,2,3) $\rightarrow$ return 0
4. Skip 2: ways(9,2,3)
5. Include 3: ways(0,2,4) $\rightarrow$ return 1 ✓
6. Skip 3: ways(9,2,4) $\rightarrow$ return 0

7. Back to root, Skip 1: ways(10,2,2)
8. Include 2: ways(6,2,3) → return 0
9. Skip 2: ways(10,2,3)
10. Include 3: ways(1,2,4) → return 0
11. Skip 3: ways(10,2,4) → return 0

Exercise: Draw Trees Yourself

Exercise 1: X=3, N=2

Available powers: $1^2=1$

Tree draw karo. (Hint: sirf 1 ka square use kar sakte hain)

Exercise 2: X=9, N=3

Available: $1^3=1$, $2^3=8$

Tree draw karo aur batao kitne ways hain.

Exercise 3: X=4, N=2

Available: $1^2=1$, $2^2=4$

Tree draw karo.

Answers

Exercise 1: X=3, N=2

ways(3, 2, 1)

└─ Include 1: ways(2, 2, 2) → $2^2=4 > 2$ → return 0

└─ Skip 1: ways(3, 2, 2) → $2^2=4 > 3$ → return 0

Total = 0 ways

Exercise 2: X=9, N=3

```

ways(9,3,1)
├─ Include 1³: ways(8,3,2)
|   └─ Include 2³: ways(0,3,3) → return 1 ✓
|   └─ Skip 2: ways(8,3,3) → 3³=27>8 → return 0
└─ Skip 1: ways(9,3,2)
    └─ Include 2³: ways(1,3,3) → 3³=27>1 → return 0
    └─ Skip 2: ways(9,3,3) → 27>9 → return 0
Total = 1 way (1³ + 2³ = 9)

```

Exercise 3: X=4, N=2

```

ways(4,2,1)
├─ Include 1²: ways(3,2,2) → 2²=4>3 → return 0
└─ Skip 1: ways(4,2,2)
    └─ Include 2²: ways(0,2,3) → return 1 ✓
    └─ Skip 2: ways(4,2,3) → 3²=9>4 → return 0
Total = 1 way (2² = 4)

```

Chapter Summary

- Recursion tree har function call aur uski choices ko visual karta hai
- Har path ek unique combination hai
- Leaves par base cases handle hote hain
- Tree DFS order mein traverse hota hai (left se right)
- Depth limited hai kyunki powers jaldi badhti hain

Tree ko samajhna = Recursion ko samajhna. Agle chapter mein backtracking seekhenge!

Chapter 7: Backtracking (Explore, Undo, Continue)

Backtracking Kya Hai?

Real-Life Analogy 1: Bhool Bhulaiya (Maze)

Socho aap ek maze mein hain. Aapko exit dhundhna hai:

1. Aap ek path choose karte hain
2. Us path par chalte hain
3. Agar exit mil gaya - great!
4. Agar dead end (band rasta) mila - **wapas aao** (backtrack)
5. Phir doosra path try karo

Yehi hai backtracking! Try, fail, wapaa aao, doosra try karo.

Real-Life Analogy 2: Tala Aur Chaabi

Aapke paas 5 chaabiyaa hain, ek tala kholna hai:

1. Pehli chaabi try ki - nahi khula (backtrack - chaabi wapaa rakho)
2. Doosri try ki - nahi khuli (backtrack)
3. Teesri try ki - khul gaya! ✓

Har galat chaabi ke baad “undo” karte hain aur next try karte hain.

Real-Life Analogy 3: Shopping

Aapke paas ₹100 hain. Aapko exactly ₹100 ka saman khareedna hai:

1. Ek item uthaya (₹30) - bag mein daala
2. Doosra item uthaya (₹40) - bag mein daala (total ₹70)
3. Teesra item (₹50) - ₹120 ho jayenge, budget se zyada
 - **Backtrack:** Teesra item wapaa rakho
4. Choutha item (₹30) - exactly ₹100! ✓

Backtracking vs Simple Recursion

Simple Recursion	Backtracking
Ek hi direction mein jaata hai	Multiple paths try karta hai
Wapas nahi aata	Galat path se wapas aata hai
Ek solution	All possible solutions explore
Example: Factorial	Example: Maze, Sudoku, Our Problem

Hamari problem backtracking isliye hai kyunki:

- Hum combinations try karte hain
- Agar sum zyada ho jaye to “undo” karte hain
- Doosri combinations try karte hain
- **All possible ways** count karte hain

Backtracking Ki Teen Steps

Step 1: CHOOSE (Chuno)

Ek decision lo. Current number ko include karo ya skip karo.

Step 2: EXPLORE (Aage Badho)

Decision ke saath aage ki possibilities explore karo.

`ways(remaining - power, n, next_num)` ya `ways(remaining, n, next_num)`

Step 3: UNCHOOSE / BACKTRACK (Wapas Aao)

Agar path sahi nahi to wapas aao aur doosra path try karo.

Ye automatically hota hai jab function return karta hai!

Important: Hamari problem mein “unchoose” explicit nahi karna padta kyunki hum koi global state modify nahi kar rahe. Parameters pass kar rahe hain to wapas aane par

automatically previous state mil jaati hai.

Backtracking Ka Magic (Code Mein)

```
def ways(x, n, i):
    # Base cases...

    # CHOOSE: Include i
    # EXPLORE with included i
    include_result = ways(x - i*n, n, i + 1)
    # Backtrack automatically! (x wapas wahi hai, kyunki local variable tha)

    # CHOOSE: Skip i
    # EXPLORE with skipped i
    skip_result = ways(x, n, i + 1)
    # Backtrack automatically!

    return include_result + skip_result
```

Notice karo: Humne kuch “undo” nahi kiya explicitly. Kyun?

- $x - i*n$ ek new value hai jo function call ko pass hui
- Original x ki value change nahi hui
- Jab function return karta hai, original state intact hai

Ye hi backtracking ka sabse clean form hai!

Backtracking With Explicit State (Jab Zaroorat Ho)

Kabhi kabhi humein explicit state maintain karni padti hai. Jaise agar hum combinations print karwana chahein:

```
def find_combinations(x, n, i, current_combo):
    """
    current_combo: list of numbers chosen so far
    """
    if x == 0:
```

```

    print("Found:", current_combo) # Print the combination
    return 1

if i**n > x:
    return 0

ways_count = 0

# CHOOSE: Include i
current_combo.append(i) # Add to combination
ways_count += find_combinations(x - i**n, n, i+1, current_combo)
current_combo.pop() # UNCHOOSE: Remove (BACKTRACK!)

# CHOOSE: Skip i
ways_count += find_combinations(x, n, i+1, current_combo)
# No need to unchoose because we didn't modify state

return ways_count

# Call
combos = []
find_combinations(10, 2, 1, combos)

```

Yahan append aur pop explicitly backtracking kar rahe hain.

Visual: Backtracking in Action

X=10, N=2 ke liye backtracking flow:

Start: [] (empty combination), remaining=10

Step 1: Try including 1

Combo: [1], remaining=9

Step 2: Try including 2

Combo: [1,2], remaining=5

Step 3: Try including 3

Combo: [1,2,3], remaining=-4 → FAIL (negative)

BACKTRACK: Remove 3 → [1,2], remaining=5

Step 4: Try skipping 3

Combo: [1,2], remaining=5

```
Next is 4:  $4^2=16 > 5 \rightarrow$  FAIL  
BACKTRACK: Remove 2  $\rightarrow [1]$ , remaining=9  
Step 5: Try skipping 2  
Combo:  $[1]$ , remaining=9  
Step 6: Try including 3  
Combo:  $[1,3]$ , remaining=0  $\rightarrow$  SUCCESS!  $\checkmark$   
BACKTRACK: Remove 3  $\rightarrow [1]$ , remaining=9  
Step 7: Try skipping 3  
Combo:  $[1]$ , remaining=9  
Next is 4:  $16 > 9 \rightarrow$  FAIL  
BACKTRACK: Remove 1  $\rightarrow []$ , remaining=10  
Step 8: Try skipping 1  
Combo:  $[]$ , remaining=10  
... (continue exploring starting from 2)
```

Backtracking Ki Important Properties

1. Systematic Exploration

Har possibility exactly ek baar explore hoti hai. Kuch miss nahi hota.

2. State Restoration

Backtrack karte waqt state wapas wahi aa jaati hai. Jaise kuch hua hi nahi.

3. Pruning (Kaant-Chhaant)

Agar pata chal jaye ki aage kuch nahi milega, to turant wapas aa jao.

Hamari problem mein: `if i**n > x: return 0` - ye pruning hai!

4. Completeness

Agar solution exist karta hai to backtracking zaroor dhundhegi.

Cricket Team Selection Analogy

Socho aapko 11 players ki team chunni hai 15 players mein se:

```
Player 1: [Select] ya [Reject]
Player 2: [Select] ya [Reject]
Player 3: [Select] ya [Reject]
...
Jab 11 select ho jayein: Team complete! ✓
Jab remaining players + selected < 11: Impossible, backtrack X
```

Hamari Problem Mein Backtracking

Hamara code backtracking naturally implement karta hai:

```
def ways(x, n, i):
    # Pruning: Agar current number ki power hi x se zyada
    if i**n > x:
        return 0    # Backtrack! Is path mein kuch nahi milega

    # Success: Target achieve
    if x == 0:
        return 1    # Mil gaya!

    # Explore both choices
    # Choice 1: Include (state automatically restored on return)
    include = ways(x - i**n, n, i + 1)

    # Choice 2: Skip (state intact)
    skip = ways(x, n, i + 1)

    return include + skip
```

Yahan backtracking automatic hai kyunki:

- x ki value change nahi hoti (immutable integer)
- x - i**n ek new value function ko pass hoti hai

- Return hone par purani value wapas mil jaati hai

Chapter Summary

- **Backtracking** = Try, Check, If fail then Undo and Try Next
- Teen steps: Choose → Explore → Unchoose
- Automatically backtrack jab function parameters pass ho (immutable types)
- Explicit backtrack needed for mutable types (lists, etc.)
- Hamari problem natural backtracking example hai
- Pruning se unnecessary exploration avoid hoti hai

Backtracking samajh aane ke baad, ab hum actual problem solve karne ki taraf badhenge!

Chapter 8: Building Towards The Solution (Practice Problems)

Iss chapter mein hum step-by-step 10 practice problems solve karenge. Har problem pehli se thodi mushkil hogi. End tak aate-aate aap original HackerRank problem solve karne ke liye ready ho jaoge.

Har problem ke liye:

1. Problem statement
 2. Solution approach (soch)
 3. Dry run
 4. Python code
 5. Line by line explanation
 6. Complexity analysis
 7. Common mistakes
-

Problem 1: Sum of Squares (Simple Loop Version)

Problem: 1 se N tak ke sabhi numbers ke squares ka sum nikalo.

Input: N = 5

Output: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1+4+9+16+25 = 55$

Solution Approach

Simple loop se kar sakte hain - abhi recursion ki zaroorat nahi.

Python Code

```
def sum_of_squares(n):  
    """Sum of squares from 1 to n"""  
    total = 0  
    for i in range(1, n + 1):  
        total += i ** 2  
    return total  
  
# Test  
print(sum_of_squares(5))    # 55  
print(sum_of_squares(3))    # 14  
print(sum_of_squares(10))   # 385
```

Line by Line

- Line 1-2: Function define kiya
- Line 3: total = 0 - sum store karne ke liye variable
- Line 4: for i in range(1, n+1) - 1 se n tak loop
- Line 5: total += i**2 - current number ka square add karo
- Line 6: Final sum return

Time Complexity: O(N)

Space Complexity: O(1)

Problem 2: Sum of Cubes (Simple Loop)

Problem: 1 se N tak ke cubes ka sum nikalo.

Input: N = 3

Output: $1^3 + 2^3 + 3^3 = 1+8+27 = 36$

Python Code

```
def sum_of_cubes(n):  
    total = 0  
    for i in range(1, n + 1):  
        total += i ** 3  
    return total  
  
# Test  
print(sum_of_cubes(3))    # 36  
print(sum_of_cubes(5))    # 225
```

Important Note

$$1^3 + 2^3 + \dots + n^3 = (n(n+1)/2)^2 = (1+2+\dots+n)^2$$

Ye interesting mathematical property hai - but hum general Nth power ke liye solve kar rahe hain.

Problem 3: Print Numbers Whose Nth Power $\leq X$

Problem: 1 se start karte hue, un numbers ko print karo jinki Nth power X se chhoti ya barabar ho.

Input: X = 20, N = 2

Output:

1 ($1^2 = 1$)
2 ($2^2 = 4$)
3 ($3^2 = 9$)
4 ($4^2 = 16$)

5 print nahi hoga kyunki $5^2=25 > 20$

Solution Approach

Loop chalao, jab tak power $\leq$ X, print karte jao.

Python Code

```
def print_valid_numbers(x, n):  
    """Print numbers whose nth power <= x"""  
    i = 1  
    while True:  
        power = i ** n  
        if power > x:  
            break  
        print(f"{i} (^{n} = {power})")  
        i += 1  
  
# Test  
print_valid_numbers(20, 2)  
print("----")  
print_valid_numbers(100, 3)
```

Line by Line

- `i = 1` : 1 se start

- Infinite loop (break condition andar hai)
- `power = i ** n` : Power calculate ki
- `if power > x: break` : Agar limit cross, loop se bahar
- Print karo, phir `i += 1` se next number

Time Complexity: $O(X^{1/n})$ - basically, kitne numbers ki power $\leq X$

Space Complexity: $O(1)$

Common Mistake

`while power <= x`: condition while loop mein rakhna tempting hai, lekin tab `power` pehle define karna padega. Ya `while True` simple hai.

Problem 4: Perfect Power Check

Problem: Check karo ki kya koi number, kisi integer ki Nth power hai?

Input:

- `num = 64, n = 3` → True ($4^3 = 64$)
- `num = 64, n = 2` → True ($8^2 = 64$)
- `num = 65, n = 2` → False

Python Code

```
def is_perfect_power(num, n):
    """Check if num is nth power of some integer"""
    if num < 1:
        return False, None

    i = 1
    while True:
        power = i ** n
```

```
    if power == num:
        return True, i
    if power > num:
        return False, None
    i += 1

# Test
print(is_perfect_power(64, 2))    # (True, 8)
print(is_perfect_power(64, 3))    # (True, 4)
print(is_perfect_power(65, 2))    # (False, None)
print(is_perfect_power(1, 5))     # (True, 1)
```

Time Complexity: $O(\text{num}^{(1/n)})$

Space Complexity: $O(1)$

Problem 5: Find One Combination (Recursion Intro)

Problem: Ek aisa combination dhundho (koi bhi ek) jiska sum exactly X ho using unique Nth powers.

Input: X = 10, N = 2

Output: [1, 3] (kyunki $1^2 + 3^2 = 10$)

Solution Approach

Pehli baar recursion use karenge! Hum ek combination dhundh rahe hain, saare nahi.

Python Code

```
def find_one_combination(x, n, i, current_combo):
    """
    Find ONE combination that sums to x
    Returns: True if found, False otherwise
    """
```

```

"""
# Base: Found!
if x == 0:
    return True

# Base: Cannot proceed
if i**n > x:
    return False

# Try including i
current_combo.append(i)
if find_one_combination(x - i**n, n, i + 1, current_combo):
    return True    # Found with i included
current_combo.pop() # Backtrack - i se nahi ban raha

# Try skipping i
if find_one_combination(x, n, i + 1, current_combo):
    return True    # Found without i

return False    # No combination found

# Test
combo = []
if find_one_combination(10, 2, 1, combo):
    print("Found:", combo)    # [1, 3]
else:
    print("No combination exists")

```

Line by Line

- **Line 7-8:** Agar $x=0$, matlab sum exactly mil gaya - return True
- **Line 11-12:** Agar current number ki power x se zyada, aage kuch nahi milega - return False
- **Line 15-19:** Current number ko include karo
 - `append(i)` - add to current combination
 - Recursive call with reduced x
 - Agar True mila - done!
 - Agar False - `pop()` karo (backtrack)
- **Line 22-24:** Current number skip karo
 - Bina modify kiye aage check karo

- **Line 26:** Kuch nahi mila

Dry Run: X=10, N=2, start with i=1

```

Call 1: find_one(10,2,1,[]) → combo=[]
  Try include 1: combo=[1], call find_one(9,2,2,[1])
    Call 2: find_one(9,2,2,[1])
      Try include 2: combo=[1,2], call find_one(5,2,3,[1,2])
        Call 3: find_one(5,2,3,[1,2])
          i=3, 3²=9 > 5 → return False
        Backtrack: pop 2, combo=[1]
      Try skip 2: call find_one(9,2,3,[1])
        Call 4: find_one(9,2,3,[1])
          i=3, 3²=9
          Try include 3: combo=[1,3], call find_one(0,2,4,[1,3])
            Call 5: find_one(0,2,4,[1,3])
              x=0 → return True!
            Found! Return True
          Return True
        Return True
      Return True
    Return True → combo = [1, 3]

```

Time Complexity: $O(2^M)$ worst case, jahan M = number of valid powers

Space Complexity: $O(M)$ for recursion stack + combination list

Problem 6: Count All Ways (The Actual Problem!)

Problem: Count karo kitne ways hain X ko unique N th powers ke sum se banane ke.

Input: $X = 10, N = 2$

Output: 1 (sirf $1^2 + 3^2$)

Input: $X = 100, N = 2$

Output: 3 ($6^2+8^2, 1^2+3^2+4^2+5^2+7^2, 10^2$)

Python Code (Final Solution)

```
def count_ways(x, n, i=1):
    """
    Count number of ways to represent x as sum of unique nth powers
    starting from i
    """
    # Base case 1: Exact sum achieved
    if x == 0:
        return 1

    # Base case 2: Current power exceeds remaining sum
    if i ** n > x:
        return 0

    # Option 1: Include i^n
    include = count_ways(x - i**n, n, i + 1)

    # Option 2: Skip i
    skip = count_ways(x, n, i + 1)

    return include + skip

# Test
print(count_ways(10, 2))    # 1
print(count_ways(13, 2))    # 1
print(count_ways(100, 2))   # 3
print(count_ways(100, 3))   # 1
```

Line by Line Explanation

- **Line 1:** Function with default parameter `i=1` (start from 1)
- **Line 6-7:** `if x == 0` - Agar exactly sum achieve ho gaya, ye ek valid way hai. Return 1.
- **Line 10-11:** `if i**n > x` - Agar current number ki power remaining sum se zyada hai, aage ke sab numbers bhi zyada denge. Return 0.
- **Line 14:** Include karo - `x - i**n` remaining ke saath, `i+1` se start karo
- **Line 17:** Skip karo - `x` same, `i+1` se start
- **Line 19:** Dono choices ke results ka sum return

Dry Run: X=10, N=2

```

count_ways(10, 2, 1)
├─ include 1: count_ways(9, 2, 2)
|   ├─ include 2: count_ways(5, 2, 3)
|   |   └─  $3^2=9 > 5 \rightarrow$  return 0
|   |   └─ total = 0
|   └─ skip 2: count_ways(9, 2, 3)
|       ├─ include 3: count_ways(0, 2, 4)  $\rightarrow$  return 1
|       └─ skip 3: count_ways(9, 2, 4)  $\rightarrow 4^2=16>9 \rightarrow$  return 0
|       └─ total = 1 + 0 = 1
|   └─ total from 2 = 0 + 1 = 1
└─ skip 1: count_ways(10, 2, 2)
    ├─ include 2: count_ways(6, 2, 3)
    |   └─  $3^2=9 > 6 \rightarrow$  return 0
    |   └─ total = 0
    └─ skip 2: count_ways(10, 2, 3)
        ├─ include 3: count_ways(1, 2, 4)  $\rightarrow 4^2=16>1 \rightarrow$  return 0
        └─ skip 3: count_ways(10, 2, 4)  $\rightarrow 16>10 \rightarrow$  return 0
        └─ total = 0
    └─ total from 2 = 0 + 0 = 0
└─ Total ways = 1 + 0 = 1

```

Time Complexity: $O(2^M)$ worst case

Jahan $M = \text{floor}(X^{(1/N)})$ = number of valid integers

For $X=100$, $N=2$: $M=10$, worst case $2^{10} = 1024$ operations

For $X=1000$, $N=2$: $M \approx 31$, worst case ~ 2 billion (but pruning reduces drastically)

Space Complexity: $O(M)$ for recursion stack

Common Mistakes

1. **Base case order:** Pehle $x==0$ check karo, phir $i**n > x$. Kyun? Agar dono true hain to $x==0$ pehle catch hona chahiye.
2. **i+1 vs i:** Hamesha $i+1$ use karo taaki unique numbers hi use hon.
3. **Return type:** Integer return karo, True/False nahi.

Problem 7: Print All Combinations

Problem: Saare valid combinations print karo.

Input: X = 100, N = 2

Output:

```
[6, 8]
[1, 3, 4, 5, 7]
[10]
```

Python Code

```
def print_combinations(x, n, i=1, current_combo=None):
    """Print all combinations that sum to x"""
    if current_combo is None:
        current_combo = []

    if x == 0:
        print(current_combo)
        return

    if i ** n > x:
        return

    # Include i
    current_combo.append(i)
    print_combinations(x - i*n, n, i + 1, current_combo)
    current_combo.pop() # Backtrack

    # Skip i
    print_combinations(x, n, i + 1, current_combo)

# Test
print("Combinations for X=100, N=2:")
print_combinations(100, 2)
```

Important Note: Mutable Default Parameter

current_combo=None kyun use kiya instead of current_combo=[] ?

Kyunki Python mein mutable default arguments function define hote waqt ek baar create hote hain, aur har call mein same object use hota hai. Isse bugs aate hain.

```
# Galat:
def func(lst=[]): # Same list har call mein use hogi!
    ...

# Sahi:
def func(lst=None):
    if lst is None:
        lst = []
```

Problem 8: Smallest Combination (Minimum Numbers)

Problem: Sabse chhoti combination dhundho (minimum numbers wali).

Input: X = 100, N = 2

Output: [10] (sirf 1 number)

Python Code

```
def find_smallest_combo(x, n, i=1, current=None):
    """Find combination with minimum numbers"""
    if current is None:
        current = []

    if x == 0:
        return current[:] # Return copy of current combination

    if i ** n > x:
        return None # No valid combination
```



```

# Include i
current.append(i)
include_result = find_smallest_combo(x - i*n, n, i + 1, current)
current.pop()

# Skip i
skip_result = find_smallest_combo(x, n, i + 1, current)

# Compare and return the smaller one
if include_result is None and skip_result is None:
    return None
if include_result is None:
    return skip_result
if skip_result is None:
    return include_result

# Return the one with fewer elements
if len(include_result) <= len(skip_result):
    return include_result
return skip_result

# Test
result = find_smallest_combo(100, 2)
print("Smallest:", result) # [10]

```

Problem 9: Largest Combination (Maximum Numbers)

Problem: Sabse badi combination dhundho (maximum numbers wali).

Input ke liye, include first strategy se badi combinations pehle milti hain. Actually, “largest” ka matlab maximum count of numbers.

Python Code

```

def find_largest_combo(x, n, i=1):
    """Find combination with maximum numbers"""
    if x == 0:

```

```

        return []

    if i ** n > x:
        return None

    # Include i
    include_result = find_largest_combo(x - i**n, n, i + 1)
    if include_result is not None:
        return [i] + include_result

    # Skip i (only if include didn't work, to get max size we prefer includes)
    skip_result = find_largest_combo(x, n, i + 1)

    # Actually, we should try both and compare lengths
    # Let's do it properly
    include = find_largest_combo(x - i**n, n, i + 1)
    if include is not None:
        include = [i] + include

    skip = find_largest_combo(x, n, i + 1)

    if include is None and skip is None:
        return None
    if include is None:
        return skip
    if skip is None:
        return include
    if len(include) >= len(skip):
        return include
    return skip

# Test
result = find_largest_combo(100, 2)
print("Largest:", result) # Might be [1, 3, 4, 5, 7] or similar

```

Problem 10: Original HackerRank Problem

Problem: Complete function jo number of ways return kare.

Yeh wahi Problem 6 hai, lekin ab hum isse completely samajhte hain.

Final Clean Code

```
#!/bin/python3

import sys

def powerSum(X, N):
    """
    Complete the powerSum function.
    Returns number of ways to represent X as sum of unique Nth powers.
    """
    def helper(remaining, power, start):
        # Base: Found a valid way
        if remaining == 0:
            return 1

        # Base: Can't proceed further
        if start ** power > remaining:
            return 0

        # Include current number
        include = helper(remaining - start**power, power, start + 1)

        # Skip current number
        skip = helper(remaining, power, start + 1)

        return include + skip

    return helper(X, N, 1)

if __name__ == "__main__":
    X = int(input().strip())
    N = int(input().strip())
    result = powerSum(X, N)
    print(result)
```

HackerRank Specific Notes

1. Input format: Pehli line X, doosri line N
2. Constraints: $1 \leq X \leq 1000$, $2 \leq N \leq 10$
3. Ye constraints recursion ke liye safe hain

Chapter Summary Table

Problem	Concept	Complexity
Sum of Squares	Loop	$O(N)$
Sum of Cubes	Loop	$O(N)$
Print Valid Numbers	While + Break	$O(X^{(1/N)})$
Perfect Power Check	While loop	$O(X^{(1/N)})$
Find One Combo	Recursion + Backtracking	$O(2^M)$
Count All Ways	Recursion (Pure)	$O(2^M)$ pruned
Print All Combos	Recursion + List	$O(2^M)$
Smallest Combo	Recursion + Compare	$O(2^M)$
Largest Combo	Recursion + Compare	$O(2^M)$
HackerRank Solution	Final recursive	$O(2^M)$ pruned

Ab aap ready hain original problem ko confidently solve karne ke liye!

Chapter 9: Understanding The Main Recursive Function

Iss chapter mein hum apne main recursive function ko line-by-line, parameter-by-parameter samjhenge.

The Function Signature

```
def ways(x, n, i=1):
```

Parameter x (Remaining Sum)

Kya hai? Kitna sum aur banana hai (target remaining).

Shuru mein: $x = X$ (original target)

Har step par: Agar number include kiya to x kam hoga, nahi to same rahega.

Example:

```
Original X = 10
12=1 include karne ke baad:  $x = 10 - 1 = 9$ 
22=4 include na karne par:  $x = 9$  (same)
32=9 include karne par:  $x = 9 - 9 = 0 \rightarrow \text{SUCCESS!}$ 
```

Real-life: Jaise aapke paas ₹100 hain. Kuch khareedne ke baad bache hue paise = x .

Parameter n (Power)

Kya hai? Kis power ka use karna hai (2 for squares, 3 for cubes, etc.)

Kya ye change hota hai? KABHI NAHI! Pure recursion mein n constant rehta hai.

Kyun? Problem definition mein power fixed hai. Agar $X=100$, $N=2$, to hum sirf squares use karenge - kabhi cubes nahi.

Common Mistake: Kuch students n ko change kar dete hain. Ye galat hai!

```
# GALAT:
ways(x, n-1, i+1) # N ko mat chhedo!
```

```
# SAHI:
ways(x - i**n, n, i+1) # N same, sirf i change
```

Parameter i (Current Number)

Kya hai? Kaun sa number currently consider kar rahe hain.

Shuru mein: $i = 1$ (natural numbers 1 se start)

Har step par: $i + 1$ (agla number)

Kyun $i=1$ se start?

- Natural numbers 1, 2, 3, ... se start hote hain
- 0 include karna usually allowed nahi (aur $0^n = 0$ jo infinite loop de sakta hai)
- 1 se start karna safe aur problem requirement hai

Kyun i change hota hai?

- Taaki unique numbers use ho (1 baar use kiya to dobara nahi)
- Agar i change nahi kiya to infinite recursion (ek hi number baar-baar)

Function Body Deep Dive

```
def ways(x, n, i=1):
    # Part 1: Base Cases
    if x == 0:
        return 1

    if i ** n > x:
        return 0

    # Part 2: Recursive Cases
    include = ways(x - i**n, n, i + 1)
    skip = ways(x, n, i + 1)

    # Part 3: Combine Results
    return include + skip
```

Part 1: Base Cases (Rukne Ki Conditions)

Base Case 1: `if x == 0: return 1`

Question: "Return 1" kyun? "Return True" kyun nahi?

Answer: Hum **count** kar rahe hain ways ka. Jab $x=0$, matlab ek valid way mil gaya. Isliye 1 return karte hain (1 way found at this leaf).

```
# Agar return True karte:
if x == 0:
    return True # Ye batata hai "mil gaya", lekin kitne ways? Pata nahi

# Count karne ke liye:
if x == 0:
    return 1 # "Ek way mil gaya"
```

Real-life: Jaise aap coin toss mein head count karte ho. Har head = 1 count. Aise hi har valid way = 1 count.

Base Case 2: `if i**n > x: return 0`

Logic: Agar current number ki power remaining sum se zyada hai, to:

- Current number include nahi kar sakte (sum exceed ho jayega)
- Aage ke numbers bhi include nahi kar sakte (kyunki i badhta hai, to i^n bhi badhega)

Isliye is branch mein koi valid way nahi milega. Return 0.

Example: $x=5, i=3, n=2$

- $3^2 = 9 > 5$
- Na 3 le sakte hain, na 4, na 5...
- Return 0

Part 2: Recursive Calls

Call 1: Include Current Number

```
include = ways(x - i**n, n, i + 1)
```

Breakdown:

- $x - i^n$: Current number ki power remaining sum se subtract ki (number include kar liya)
- n : Power same
- $i + 1$: Agle number par jao (unique numbers ke liye)

Meaning: “Maine i ko choose kar liya. Ab remaining $(x - i^n)$ ko $i+1$ se aage ke numbers se banana hai.”

Call 2: Skip Current Number

```
skip = ways(x, n, i + 1)
```

Breakdown:

- x : Same remaining sum (kuch subtract nahi kiya)
- n : Power same
- $i + 1$: Agle number par jao

Meaning: “Maine i ko skip kar diya. Ab poora x $i+1$ se aage ke numbers se banana hai.”

Part 3: Combine Results

```
return include + skip
```

Total ways = (ways jisme i include hai) + (ways jisme i skip hai)

Kyun add? Kyunki ye do mutually exclusive possibilities hain. Har combination ya to i ko include karegi ya skip.

Visual:

```
Total ways for (x, i)
|— Ways with i included: include
```


└─ Ways with i skipped: skip
Total = include + skip

Parameter Flow Visualization

```
ways(10, 2, 1)
  |
  x=10, n=2, i=1
  |
  |--- include: ways(10-1, 2, 1+1) = ways(9, 2, 2)
  |   |
  |   x=9, n=2, i=2
  |   |
  |   |--- include: ways(9-1, 2, 2+1) = ways(8, 2, 3)
  |   |   |
  |   |   ...
  |   |--- skip: ways(9, 2, 3)
  |   ...
  |
  |--- skip: ways(10, 2, 2)
      |
      x=10, n=2, i=2
      ...
```

Notice: n hamesha 2 rehta hai! Sirf x aur i change hote hain.

Why i+1 and Not i+n?

Question: Agle number par jaane ke liye i+1 kyun, i+n kyun nahi?

Answer: Hum consecutive natural numbers consider kar rahe hain: 1, 2, 3, 4, ...

Agar i+n karte:

```
ways(x, n, i+n) # i=1, n=2: 1, 3, 5, 7... (odd numbers only!)
```

Ye numbers 2, 4, 6 miss kar dega. Aur $2^2 = 4$ bhi important ho sakta hai.

Why Not Use `i` Without Increment?

```
# GALAT - Infinite recursion!  
def ways(x, n, i):  
    include = ways(x - i*n, n, i) # i same raha!  
    ...
```

Agar `i` change nahi kiya:

- Hamesha same number consider hoga
- `x` kabhi 0 nahi hoga (ya negative)
- Infinite recursion → Stack Overflow!

Common Questions

Q: Default parameter `i=1` kyun?

A: Taaki user ko `ways(10, 2)` call karna pade, `ways(10, 2, 1)` nahi. Starting number hamesha 1 hai, to default rakhna convenient hai.

Q: `n` constant kyun hai?

A: Problem definition: hum ek specific power (2nd, 3rd, etc.) ke numbers use kar rahe hain. Jaise agar problem hai "sum of squares", to sirf squares, cubes nahi.

Q: Kya hum `i=0` se start kar sakte hain?

A: Technically $0^n = 0$ (for $n > 0$). Isse fayda nahi, infinite loop ka risk hai, aur natural numbers conventionally 1 se start hote hain.

Chapter Summary

Parameter	Meaning	Changes?
x	Remaining sum to achieve	Decreases when include
n	Power (constant)	NEVER
i	Current number	Always i+1

Key Insight: Function har number ke liye do choices explore karta hai (include/skip), unique numbers ensure karne ke liye i badhata hai, aur base cases se rukna jaanta hai.

Chapter 10: Base Conditions (Deep Dive)

Base conditions hi recursion ki jaan hain. Inke bina recursion infinite loop mein chali jayegi. Iss chapter mein hum dono base conditions ko microscopically examine karenge.

Base Case 1: `if x == 0: return 1`

Yeh Condition Kyun?

Jab `x == 0`, matlab humne exactly target sum achieve kar liya.

Real-life: Jaise aapne exactly ₹100 ka saman khareed liya - mission accomplished!

Return 1 Kyun? Return True Kyun Nahi?

```
# Option A: Return 1 (Correct)
if x == 0:
    return 1
```

```
# Option B: Return True (Wrong for counting)
```

```
if x == 0:
    return True
```

Reason: Hum **count** kar rahe hain ways ka.

Caller function ye karta hai:

```
include = ways(...) # Expect integer
skip = ways(...)    # Expect integer
return include + skip # Integer addition
```

Agar hum True return karte:

```
include = True # Boolean
skip = False   # Boolean
return True + False # Kya hoga? (Actually Python mein 1+0=1, lekin logically
```

Python mein `True + False = 1`, lekin ye **semantically galat** hai. Hum count kar rahe hain, to numbers return karo.

Proof With Small Example

X=4, N=2: Sirf $2^2=4$ ek way hai.

```
def ways_count(x, n, i):
    if x == 0:
        return 1 # Counting: 1 way
    if i**n > x:
        return 0 # Counting: 0 ways
    return ways_count(x - i**n, n, i+1) + ways_count(x, n, i+1)
```

Tree:

```
ways(4,2,1) → 12=1
├─ include: ways(3,2,2) → 22=4>3 → 0
└─ skip: ways(4,2,2) → 22=4
    ├─ include: ways(0,2,3) → return 1 ✓
    └─ skip: ways(4,2,3) → 32=9>4 → 0
Total: 0 + (1 + 0) = 1 ✓ Correct!
```

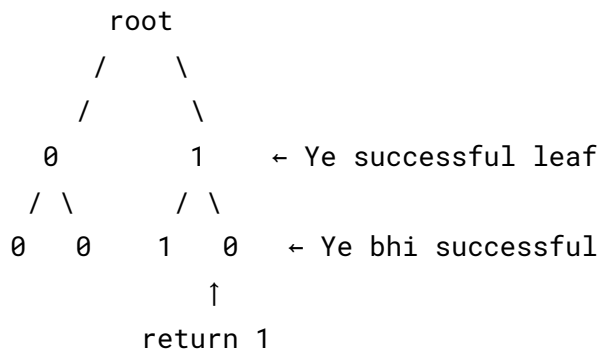
Agar return True karte:

```
# True = 1, False = 0 in Python arithmetic
# But let's say we return True/False
include = ways(...) # True or False
skip = ways(...)    # True or False
return include + skip # Bool + Bool → Int (but confusing)
```

Mathematical Meaning

return 1 ka matlab: "Is leaf node par ek solution mila."

Tree ke saare successful leaves ke 1's ka sum = total ways.



Total = 1 + 1 = 2 ways

Kya $x < 0$ Check Karna Chahiye?

Hamara code $x < 0$ explicitly check nahi karta. Kyun?

```
if i ** n > x:    # Ye condition  $x < i^n$  check karti hai
    return 0
```

Agar hum sirf positive values hi subtract karte hain ($x - i**n$), to x kabhi negative ho sakta hai?

Haan! Example: $x=5, i=3, n=2$

- $3^2 = 9$

- $x - i^{**n} = 5 - 9 = -4$ (negative!)

To kya $x < 0$ ka base case add karna chahiye?

```
if x < 0:
    return 0
```

Mere recommendation: Add kar do! Safety ke liye.

```
def ways(x, n, i):
    if x == 0:
        return 1
    if x < 0:                # Extra safety
        return 0
    if i ** n > x:
        return 0
    # ...
```

Lekin original code mein ye zaroori nahi kyunki $i^{**n} > x$ condition negative x ko bhi handle karegi (negative $< i^n$ for positive i^n).

Base Case 2: if $i^{**n} > x$: return 0

Yeh Condition Kyun?

Ye **pruning** hai. Agar current number ki power remaining sum se zyada hai, to:

1. Current number include nahi kar sakte
2. Aage ke numbers ki power aur zyada hogi
3. Is branch mein kabhi solution nahi milega

Real-life: Aapke paas ₹10 bache hain. Agla item ₹50 ka hai. Na wo le sakte hain, na usse mehenge items.

Mathematical Justification

For positive integers, if $a < b$:

- $a^n < b^n$ (for $n > 0$)

So if $i^n > x$, then:

- $(i+1)^n > i^n > x$
- $(i+2)^n > (i+1)^n > i^n > x$
- And so on...

No future number's power will ever be $\leq x$.

Why Return 0?

Kyunki is branch mein 0 ways hain. Koi solution nahi milega.

```
return 0 # "Yahan kuch nahi mila"
```

Caller ke paas jaake:

```
include = ways(x, n, i+1) # Returns 0
# include = 0, matlab include wali branch se kuch nahi mila
```

Example Demonstration

$X=10, N=2$:

```
ways(10, 2, 4)
Check:  $4^2 = 16$ 
 $16 > 10$ ? YES → return 0
```

Is branch ki zaroorat hi nahi explore karne ki!

Without this condition:

```
# Function kabhi khatam nahi hoga
# i badhta rahega: 4, 5, 6, ... infinity
# Kabhi  $x=0$  nahi hoga kyunki sab 16, 25, 36... ( $all > 10$ )
```

Is Condition Ki Placement

Order of checks (IMPORTANT):

1. $x == 0$ → Success
2. $x < 0$ → Failure (optional but recommended)
3. $i^n > x$ → Failure (pruning)

Kyun ye order? Agar $x=0$ hai, to success return karna chahiye chahe i kuch bhi ho.

Example:

```
ways(0, 2, 100)
# x=0 already achieved
#  $100^2 > 0$ , lekin humein farak nahi padta
# Return 1 (way found!)
```

Agar hum pehle $i^n > x$ check karein:

```
if 100**2 > 0: # True!
    return 0 # Galat! x=0 hone par 1 return karna chahiye tha
```

Base Cases Ke Beech Ka Relation

Dono base cases complementary hain:

Condition	Meaning	Return
$x == 0$	Perfect sum achieved	1 (success)
$i^n > x$	Can't proceed	0 (failure)

Together they ensure:

1. Kabhi infinite loop nahi hoga
2. Har valid path count hoga
3. Invalid paths ignore honge

Edge Cases Testing

Edge Case 1: X=1, N=2

```
ways(1, 2, 1)
├─ 12=1, include: ways(0,2,2) → x=0, return 1 ✓
└─ skip: ways(1,2,2) → 22=4>1, return 0
Total: 1
```

✓ Correct: $1^2 = 1$

Edge Case 2: X=2, N=2

```
ways(2, 2, 1)
├─ include 1: ways(1,2,2) → 22=4>1, return 0
└─ skip 1: ways(2,2,2) → 22=4>2, return 0
Total: 0
```

✓ Correct: $1^2=1$, need 1 more, lekin agla $2^2=4>1$. No way.

Edge Case 3: X=4, N=2

```
ways(4, 2, 1)
├─ include 1: ways(3,2,2) → 22=4>3, return 0
└─ skip 1: ways(4,2,2)
    ├─ include 2: ways(0,2,3) → return 1
    └─ skip 2: ways(4,2,3) → 32=9>4, return 0
Total: 1
```

✓ Correct: $2^2 = 4$

Edge Case 4: X=100, N=3

Power of 4 = 64, power of 5 = 125 > 100. So consider numbers 1-4.

Tree will find: $1^3+2^3+3^3+4^3 = 1+8+27+64 = 100 \rightarrow 1$ way.

Common Mistake: Missing Base Case

```
# GALAT - Missing  $i^n > x$  check
def ways(x, n, i):
    if x == 0:
        return 1
    # Missing: if  $i^n > x$ : return 0

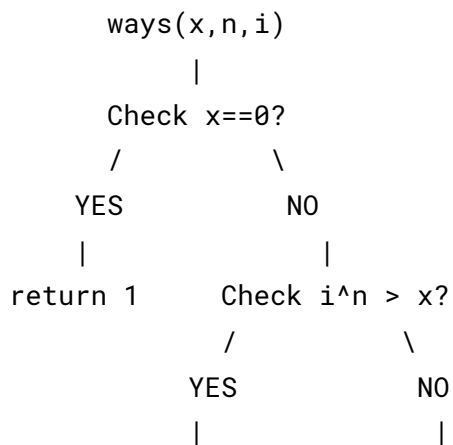
    include = ways(x - i**n, n, i+1)
    skip = ways(x, n, i+1) # i keeps increasing forever!
    return include + skip
```

Isme i badhta rahega, kabhi base case hit nahi hoga. Stack overflow!

Common Mistake: Wrong Return Value

```
# GALAT
if x == 0:
    return True # Should return 1
if i**n > x:
    return False # Should return 0
```

Visual: Base Cases in Recursion Tree



```
return 0    Make recursive
           calls
```

Chapter Summary

- **$x == 0 \rightarrow \text{return } 1$** : Ek valid way mil gaya, count karo
 - **$i^n > x \rightarrow \text{return } 0$** : Is branch mein kuch nahi milega, pruning
 - Dono base cases mil kar recursion ko safe banate hain
 - Order matters: pehle success check, phir failure
 - Bina base cases ke recursion kabhi khatam nahi hogi
-

Chapter 11: Two Recursive Calls (Deep Understanding)

Is chapter mein hum samjhenge ki do recursive calls kyun, aur har call ka kya meaning hai.

The Two Calls

```
include = ways(x - i**n, n, i + 1)    # Call 1
skip = ways(x, n, i + 1)              # Call 2
```

Call 1: Include Current Number

```
include = ways(x - i**n, n, i + 1)
```

Breakdown

Part	Value	Meaning
$x - i^{**}n$	Remaining - current power	Current number ko sum mein add kar diya
n	Same	Power constant
$i + 1$	Next number	Unique numbers ke liye aage badho

Meaning (English + Hinglish)

"Maine ye number choose kar liya. Ab bache hue sum ko agle numbers se pura karo."

Real-Life Example

Aap shopping kar rahe ho, budget ₹100:

- Item 1 (₹30) khareed liya
- Ab bache hue ₹70 se aage ki shopping karo
- Agla item 2 se shuru karo (item 1 dobara mat khareedo)

Visualization

Include karte waqt:

```

| Current State: |
| x = 10, i = 1 |
|               |
| Decision: Include  $1^2 = 1$  |
|               |
| New State:    |
| x = 9, i = 2  |
| "9 ko 2 se aage ke |
| numbers se banao" |

```

Code Flow

```
# Jab include call hota hai:  
include = ways(9, 2, 2) # x kam ho gaya, i badh gaya  
# Ye function ab explore karega:  
# - Include  $2^2=4$ : ways(5, 2, 3)  
# - Skip 2: ways(9, 2, 3)
```

Why $x - i^{**n}$?

Kyunki humne current number ki power sum mein use kar li. Ab target sum kam ho gaya.

```
Original target: 10  
Used:  $1^2 = 1$   
Remaining target:  $10 - 1 = 9$ 
```

Why $i + 1$?

Taaki same number dobara use na ho (uniqueness).

```
Current i=1  
Next i=2 (1+1)  
1 ko dobara consider nahi karenge
```

Call 2: Skip Current Number

```
skip = ways(x, n, i + 1)
```

Breakdown

Part	Value	Meaning
x	Same	Kuch subtract nahi kiya
n	Same	Power constant

Part	Value	Meaning
$i + 1$	Next number	Agle number par jao

Meaning

"Maine ye number skip kar diya. Same target agle number se pura karo."

Real-Life Example

Shopping:

- Item 1 (₹30) ko ignore kiya
- Abhi bhi ₹100 bache hain
- Agle item (item 2) se shopping shuru karo

Visualization

Skip karte waqt:

```

| Current State: |
| x = 10, i = 1 |
|               |
| Decision: Skip 1² |
|               |
| New State: |
| x = 10, i = 2 |
| "10 ko 2 se aage ke |
| numbers se banao" |

```

Why x Same?

Humne current number use nahi kiya, to target same raha.

Why $i + 1$?

Current number skip kar diya, ab agle number par focus karo. Aur current number ko dubara consider nahi karna (taaki infinite loop na ho).

Dono Calls Together

```
include = ways(x - i**n, n, i + 1)
skip = ways(x, n, i + 1)
return include + skip
```

Mathematical Logic

Total ways to make sum x from numbers $\{i, i+1, i+2, \dots\}$:

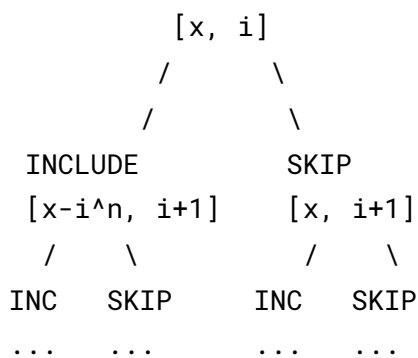
= (Ways that include i) + (Ways that exclude i)

= $\text{ways}(x - i^n, i+1) + \text{ways}(x, i+1)$

Ye partition complete hai:

- Har combination ya to i ko include karegi ya skip
- Dono sets disjoint hain (koi combination dono mein nahi)
- Dono sets ka union = all possible combinations

Visual: Binary Decision Tree



Har level par do choices. Total paths = $2^{\text{(number of levels)}}$.

Detailed Trace: X=10, N=2

```

ways(10, 2, 1) # Start
|
├ include = ways(9, 2, 2) # 1²=1 include kiya
| |
| └ include = ways(5, 2, 3) # 2²=4 include kiya
| | |
| | └ check: 3²=9 > 5 → return 0
| |
| └ skip = ways(9, 2, 3) # 2 skip kiya
|   |
|   └ include = ways(0, 2, 4) # 3²=9 include kiya → return 1
|     └ skip = ways(9, 2, 4) # 4²=16>9 → return 0
|       return 1 + 0 = 1
| return 0 + 1 = 1
|
└ skip = ways(10, 2, 2) # 1 skip kiya
  |
  └ include = ways(6, 2, 3) # 2²=4 include kiya
    |
    └ check: 3²=9 > 6 → return 0
    |
    └ skip = ways(10, 2, 3) # 2 skip kiya
      |
      └ include = ways(1, 2, 4) # 3²=9 include kiya
        |
        └ check: 4²=16 > 1 → return 0
        |
        └ skip = ways(10, 2, 4) # 4²=16 > 10 → return 0
          return 0 + 0 = 0
        return 0 + 0 = 0

```

Total = 1 + 0 = 1

Why Two Calls Exactly?

Question: Teen calls kyun nahi? Ya ek call?

Answer: Har number ke liye sirf **do** possibilities hain:

1. Lena hai (include)
2. Nahi lena hai (skip)

“Thoda lena” ya “aadha lena” possible nahi hai. Power ya to poori loge ya nahi loge.

Counter Example: Agar Teen Calls Hote

```
# Galat approach:
include_full = ways(x - i**n, n, i + 1)
include_half = ways(x - i**n/2, n, i + 1) # Makes no sense!
skip = ways(x, n, i + 1)
```

Power values fixed hain. $2^2 = 4$, aap 4 ya 0 le sakte ho, 2 nahi le sakte.

Why Not Call With Same i?

```
# Galat:
include = ways(x - i**n, n, i) # i same raha!
```

Isse kya hoga?

```
ways(10, 2, 1)
├ include: ways(9, 2, 1) # i still 1
|   └ include: ways(8, 2, 1) # i still 1
|       └ include: ways(7, 2, 1) # i still 1...
|           ... infinite!
```

Hamesha $i+1$ karo taaki aage badho aur kabhi base case hit ho.

Why Not $i+2$ or $i+n$?

```
# Galat:  $i+2$ 
skip = ways(x, n, i + 2) # Numbers skip ho jayenge
```

Example: $i=1, n=2$

- $i+2$ karte to: 1, 3, 5, 7... (2 miss ho jayega)
- $2^2 = 4$ important ho sakta hai

```
# Galat: i+n
skip = ways(x, n, i + n)
# For n=2: 1, 3, 5... (even numbers miss)
# For n=3: 1, 4, 7... (many numbers miss)
```

Hamesha $i+1$ use karo - har natural number consider karo.

The Addition Logic

```
return include + skip
```

Kyun add? Kyunki:

```
Total valid combinations
= Combinations WITH current number
+ Combinations WITHOUT current number
```

Ye addition tabhi valid hai jab dono sets completely disjoint hon. Aur hain bhi - ek combination mein ya to i hai ya nahi, dono nahi ho sakta.

Chapter Summary

Call	Parameters	Meaning
include	$(x-i^n, n, i+1)$	"Ye number le liya, aage badho"
skip	$(x, n, i+1)$	"Ye number skip kiya, aage badho"

- Dono calls complement hain - cover all possibilities
- i hamesha $i+1$ hota hai for uniqueness

- n kabhi change nahi hota
- Results add hote hain for total count

Chapter 12: Complete Dry Runs

Ab hum teen important examples ka complete dry run karenge. Har recursive call, har return value trace karenge.

Dry Run 1: X=10, N=2

Available: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16(>10 \text{ stop})$

Complete Trace

```
CALL 1: ways(10, 2, 1)
├─ Check:  $x=10 \neq 0$ ,  $1^2=1 \leq 10$ 
├─ Call 2: ways(9, 2, 2) [INCLUDE 1]
│   ├─ Check:  $x=9 \neq 0$ ,  $2^2=4 \leq 9$ 
│   └─ Call 3: ways(5, 2, 3) [INCLUDE 2]
│       └─ Check:  $x=5 \neq 0$ ,  $3^2=9 > 5$ 
│           └─ RETURN 0
│   └─ Call 4: ways(9, 2, 3) [SKIP 2]
│       └─ Check:  $x=9 \neq 0$ ,  $3^2=9 \leq 9$ 
│           └─ Call 5: ways(0, 2, 4) [INCLUDE 3]
│               └─ Check:  $x=0$ 
│                   └─ RETURN 1 ✓ WAY FOUND!
│   └─ Call 6: ways(9, 2, 4) [SKIP 3]
│       └─ Check:  $4^2=16 > 9$ 
│           └─ RETURN 0
│   └─ Return:  $1 + 0 = 1$ 
└─ Return:  $0 + 1 = 1$ 
├─ Call 7: ways(10, 2, 2) [SKIP 1]
└─ Check:  $x=10 \neq 0$ ,  $2^2=4 \leq 10$ 
```

```

|   |─ Call 8: ways(6, 2, 3)  [INCLUDE 2]
|   |   |─ Check:  $3^2=9>6$ 
|   |   └─ RETURN 0
|   |─ Call 9: ways(10, 2, 3) [SKIP 2]
|   |   |─ Check:  $3^2=9\leq 10$ 
|   |   |─ Call 10: ways(1, 2, 4) [INCLUDE 3]
|   |   |   |─ Check:  $4^2=16>1$ 
|   |   |   └─ RETURN 0
|   |   |─ Call 11: ways(10, 2, 4) [SKIP 3]
|   |   |   |─ Check:  $4^2=16>10$ 
|   |   |   └─ RETURN 0
|   |   └─ Return:  $0 + 0 = 0$ 
|   └─ Return:  $0 + 0 = 0$ 
└─ FINAL Return:  $1 + 0 = 1$ 

```

Summary

Total Ways = 1

Combination: $1^2 + 3^2 = 1 + 9 = 10$

Dry Run 2: X=17, N=2

Available: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16$, $5^2=25(>17 \text{ stop})$

Skeleton Trace (Important Calls Only)

```

ways(17, 2, 1)
|─ INCLUDE 1: ways(16, 2, 2)
|   |─ INCLUDE 2: ways(12, 2, 3)
|   |   |─ INCLUDE 3: ways(3, 2, 4) →  $4^2=16>3 \rightarrow 0$ 
|   |   └─ SKIP 3: ways(12, 2, 4)
|   |       |─ INCLUDE 4: ways(-4, 2, 5) → Actually check:  $4^2=16\leq 12?$ 
|   |       |   Wait:  $4^2=16$ ,  $12-16=-4$ 
|   |       |   ways(-4, 2, 5): Check  $5^2=25>(-4) \rightarrow 0$ 
|   |       |   (Need  $x<0$  check for safety)
|   |       └─ SKIP 4: ways(12, 2, 5) →  $5^2=25>12 \rightarrow 0$ 

```

```

| | Return 0
| | Return 0
| └─ SKIP 2: ways(16,2,3)
|   └─ INCLUDE 3: ways(7,2,4) →  $4^2=16>7$  → 0
|     └─ SKIP 3: ways(16,2,4)
|       └─ INCLUDE 4: ways(0,2,5) → return 1 ✓ ( $4^2=16$ )
|         └─ SKIP 4: ways(16,2,5) →  $5^2=25>16$  → 0
|           Return 1
|         Return 1
|       Return 0 + 1 = 1
|     SKIP 1: ways(17,2,2)
|       └─ INCLUDE 2: ways(13,2,3)
|         └─ INCLUDE 3: ways(4,2,4)
|           └─  $4^2=16>4$  → 0
|             └─ SKIP 4: ways(4,2,5) →  $25>4$  → 0
|               Return 0
|             SKIP 3: ways(13,2,4)
|               └─ INCLUDE 4: ways(-3,...) → 0
|                 └─ SKIP 4: ways(13,2,5) → 0
|                   Return 0
|                 Return 0
|               SKIP 2: ways(17,2,3)
|                 └─ INCLUDE 3: ways(8,2,4) →  $4^2=16>8$  → 0
|                   └─ SKIP 3: ways(17,2,4)
|                     └─ INCLUDE 4: ways(1,2,5) →  $25>1$  → 0
|                       └─ SKIP 4: ways(17,2,5) →  $25>17$  → 0
|                         Return 0
|                       Return 0
|                     Return 0
|                   Return 0
|                 Return 0
|               Return 0
|             Return 0
|           Return 0
|         Return 0
|       Return 0
|     Return 0
|   Return 0
| Return 0

```

Total = 1 + 0 = 1

Way found: $1^2 + 4^2 = 1 + 16 = 17$

Verification

Kya 17 ko doosre tarike se bana sakte hain squares se?

- $2^2+3^2=4+9=13$ X
- $3^2+?$ 9+?=17, need 8 which is not square
- 4^2+1^2 already counted

Answer: 1 way ✓

Dry Run 3: X=100, N=2

Available: $1^2=1$, $2^2=4$, $3^2=9$, $4^2=16$, $5^2=25$, $6^2=36$, $7^2=49$, $8^2=64$, $9^2=81$, $10^2=100$

Ye bada example hai, isliye sirf successful paths trace karenge.

Success Path 1: $10^2 = 100$

```
ways(100, 2, 1)
SKIP 1: ways(100, 2, 2)
SKIP 2: ways(100, 2, 3)
...
SKIP 9: ways(100, 2, 10)
INCLUDE 10: ways(0, 2, 11) → RETURN 1 ✓
```

Success Path 2: $6^2 + 8^2 = 36 + 64 = 100$

```
ways(100, 2, 1)
SKIP 1: ways(100, 2, 2)
...
SKIP 5: ways(100, 2, 6)
INCLUDE 6: ways(64, 2, 7)
SKIP 7: ways(64, 2, 8)
INCLUDE 8: ways(0, 2, 9) → RETURN 1 ✓
```

Success Path 3: $1^2+3^2+4^2+5^2+7^2 = 1+9+16+25+49 = 100$

```
ways(100, 2, 1)
INCLUDE 1: ways(99, 2, 2)
SKIP 2: ways(99, 2, 3)
INCLUDE 3: ways(90, 2, 4)
INCLUDE 4: ways(74, 2, 5)
```

```

INCLUDE 5: ways(49,2,6)
SKIP 6: ways(49,2,7)
INCLUDE 7: ways(0,2,8) → RETURN 1 ✓

```

Total Count

3 ways found.

Chapter Summary Table

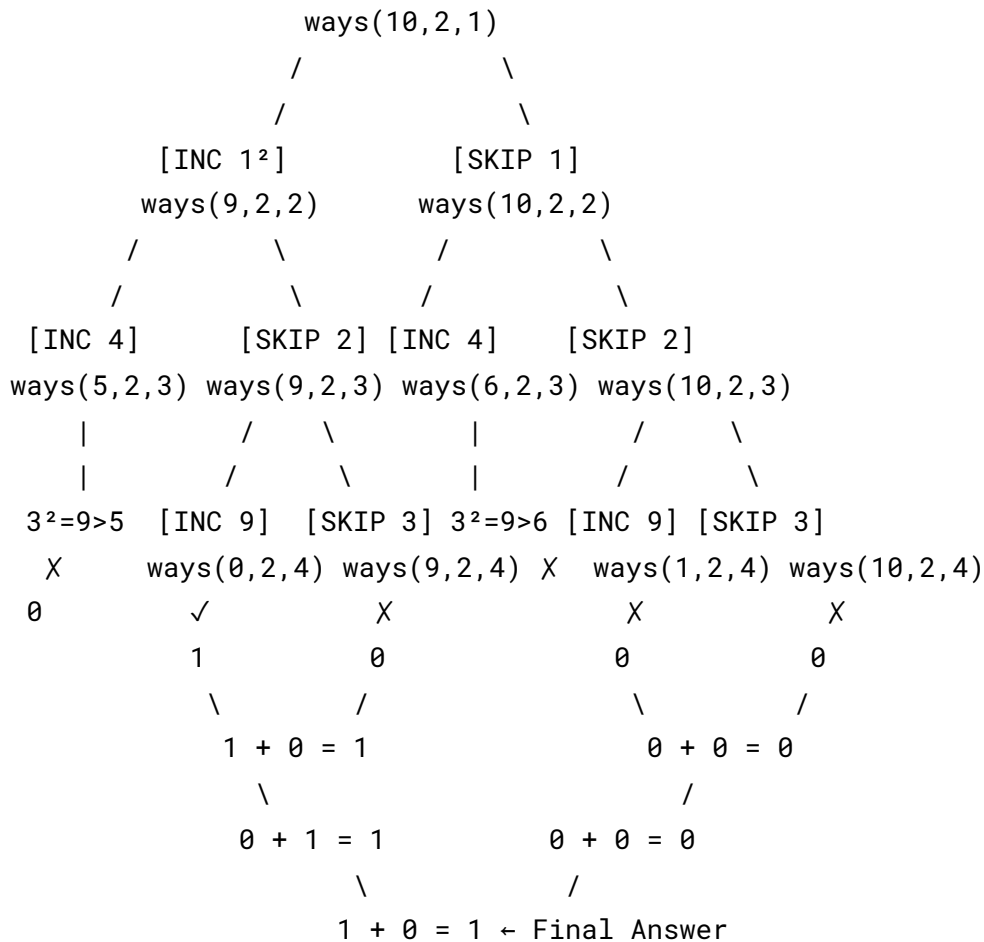
X	N	Ways	Combinations
10	2	1	1^2+3^2
17	2	1	1^2+4^2
100	2	3	$10^2, 6^2+8^2, 1^2+3^2+4^2+5^2+7^2$
100	3	1	$1^3+2^3+3^3+4^3$
13	2	1	2^2+3^2

Chapter 13: Visualization (Recursion Tree Diagrams)

Iss chapter mein hum ASCII art se complete recursion trees draw karenge.

How To Read Our Trees

```
Status indicators:
✓ = Success (return 1)
✗ = Failure (return 0)
```

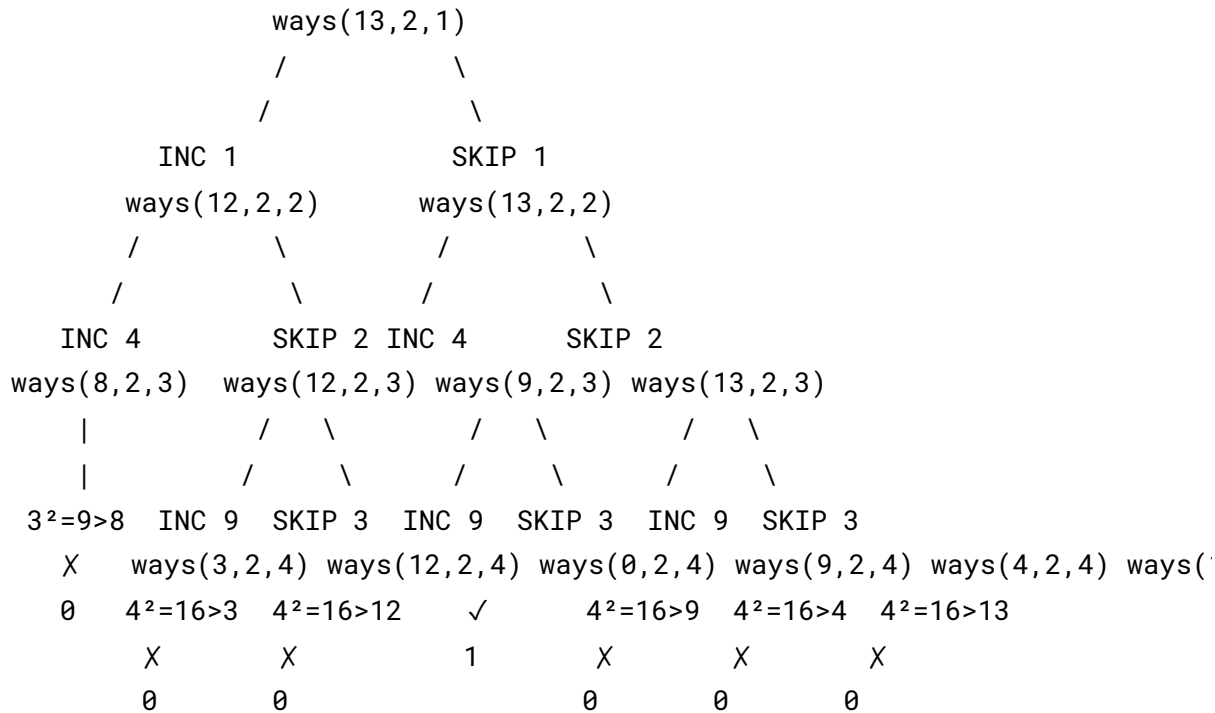


$3^2=9>5$ = Check: $3^2=9 > \text{remaining}=5 \rightarrow \text{Stop}$

✓ = Base case $x=0$ hit, return 1

X = Base case $i^n>x$ hit, return 0

Simplified Tree: X=13, N=2



Result: Sirf ek path successful: SKIP 1 → SKIP 2 → INC 3 → ✓

Combination: $2^2 + 3^2 = 4 + 9 = 13$

Tree Visualization For Understanding Flow

Depth-First Traversal Order

1. ways(10,2,1) ← Start (root)
2. ways(9,2,2) ← Go left (include)
3. ways(5,2,3) ← Go left (include)

```

4. return 0           ← Leaf, go back
5. ways(9,2,3)        ← Go right (skip)
6. ways(0,2,4)        ← Go left (include)
7. return 1 ✓         ← Success leaf!
8. ways(9,2,4)        ← Go right (skip)
9. return 0           ← Leaf
10. return 1          ← Combine children
11. return 1          ← Combine children
12. ways(10,2,2)      ← Go right from root
... continue ...

```

Visual Stack Representation

Stack grows downward (each level = one function call)

```

Level 1: ways(10,2,1) - calculating include...
Level 2:  ways(9,2,2) - calculating include...
Level 3:   ways(5,2,3) → returns 0
Level 2:   ways(9,2,2) - got include=0, calculating skip...
Level 3:   ways(9,2,3) - calculating include...
Level 4:    ways(0,2,4) → returns 1 ✓
Level 3:    ways(9,2,3) - got include=1, calculating skip...
Level 4:    ways(9,2,4) → returns 0
Level 3:    ways(9,2,3) → returns 1+0=1
Level 2:   ways(9,2,2) → returns 0+1=1
Level 1: ways(10,2,1) - got include=1, calculating skip...
...

```

Success Paths Visualization: X=100, N=2

Tree itna bada hai ki pura draw nahi kar sakte. Sirf success paths dikhate hain:

Path 1 (Direct):

```

ways(100,2,1)
└─ SKIP 1

```

```

└─ SKIP 2
  └─ ... (skip 3-9)
    └─ SKIP 9
      └─ INCLUDE 10: ways(0,2,11) → ✓

```

Combination: [10]

Path 2:

```

ways(100,2,1)
└─ SKIP 1
  └─ ...
    └─ SKIP 5
      └─ INCLUDE 6: ways(64,2,7)
        └─ SKIP 7
          └─ INCLUDE 8: ways(0,2,9) → ✓

```

Combination: [6, 8]

Path 3:

```

ways(100,2,1)
└─ INCLUDE 1: ways(99,2,2)
  └─ SKIP 2: ways(99,2,3)
    └─ INCLUDE 3: ways(90,2,4)
      └─ INCLUDE 4: ways(74,2,5)
        └─ INCLUDE 5: ways(49,2,6)
          └─ SKIP 6: ways(49,2,7)
            └─ INCLUDE 7: ways(0,2,8) → ✓

```

Combination: [1, 3, 4, 5, 7]

Chapter Summary

- Recursion tree har possible path dikhata hai

- Tree DFS order mein traverse hota hai
 - Left child = include, right child = skip
 - Leaves par base cases handle hote hain
 - Successful leaves return 1, contribute to total
 - Tree size = $O(2^M)$ where M = valid numbers count
-

Chapter 14: Common Mistakes (Aur Unse Kaise Bachen)

Iss chapter mein hum common mistakes dekhenge jo beginners karte hain, aur unhe kaise avoid karen.

Mistake 1: Wrong Base Condition Order

Galat Code:

```
def ways(x, n, i):  
    if i**n > x:          # Pehle ye check  
        return 0  
    if x == 0:           # Phir ye  
        return 1
```

Problem:

Jab $x=0$ aur i kuch bhi ho, $i**n > 0$ true ho sakta hai!

Example: `ways(0, 2, 5)`

- $5^2 = 25 > 0 \rightarrow$ return 0
- Lekin $x=0$ already achieved tha! Return 1 hona chahiye.

Sahi Code:

```
def ways(x, n, i):  
    if x == 0:          # Pehle success check  
        return 1  
    if i**n > x:        # Phir failure check  
        return 0
```

Mistake 2: Infinite Recursion

Galat Code:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    include = ways(x - i**n, n, i)    # i same!  
    skip = ways(x, n, i)              # i same!  
    return include + skip
```

Problem:

i kabhi change nahi hota. Function baar-baar same parameters se call hota hai. Infinite loop!

Sahi:

```
include = ways(x - i**n, n, i + 1)    # i badhao  
skip = ways(x, n, i + 1)             # i badhao
```

Mistake 3: Wrong Increment (i+n or i+2)

Galat Code:

```
include = ways(x - i**n, n, i + n)
```

Problem:

Agar n=2: 1, 3, 5, 7... (even numbers miss)

Agar n=3: 1, 4, 7, 10... (many numbers miss)

Sahi:

```
include = ways(x - i**n, n, i + 1)    # Hamesha i+1
```

Mistake 4: Changing n

Galat Code:

```
include = ways(x - i**n, n-1, i+1)    # n change kar diya
```

Problem:

Power change ho gayi. Problem requirement: fixed power use karo.

Sahi:

```
include = ways(x - i**n, n, i+1)    # n constant
```

Mistake 5: Not Understanding Return Values

Galat:

```
if x == 0:  
    return True    # Boolean return
```

Problem:

Caller expect karta hai integer taaki add kar sake.

```
include + skip    # Agar True/False hain to unexpected behavior
```

Sahi:

```
if x == 0:  
    return 1      # Integer
```

Mistake 6: Missing Base Case

Galat:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    # Missing: if i**n > x: return 0  
    include = ways(x - i**n, n, i+1)  
    skip = ways(x, n, i+1)  
    return include + skip
```

Problem:

i badhta rahega, kabhi rukega nahi. Stack overflow error.

Sahi:

Add if `i**n > x`: return 0

Mistake 7: Using Loops Inside Recursive Function

Galat:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    total = 0  
    for j in range(i, int(x**(1/n))+1):    # Loop!  
        total += ways(x - j**n, n, j+1)  
    return total
```

Ye technically kaam karega (aur optimized bhi hai!), lekin hum recursion seekh rahe hain. Pehle simple recursion samjho, optimization baad mein.

Mistake 8: Mutable Default Arguments

Galat:

```
def print_combos(x, n, i=1, combo=[]):    # Mutable default!  
    if x == 0:  
        print(combo)  
        return  
    combo.append(i)  
    print_combos(x - i**n, n, i+1, combo)  
    combo.pop()  
    print_combos(x, n, i+1, combo)
```


Problem:

Python mein `combo=[]` sirf ek baar create hota hai. Multiple calls same list use karenge!

Sahi:

```
def print_combos(x, n, i=1, combo=None):  
    if combo is None:  
        combo = []  
    # ... rest of code
```

Mistake 9: Not Handling $x < 0$

Galat:

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    include = ways(x - i**n, n, i+1)  
    # ...
```

Problem:

Agar $x - i^n$ negative ho jaye, to $i^n > x$ true hoga (positive > negative). Lekin ye implicit hai. Explicit check better:

Sahi:

```
def ways(x, n, i):  
    if x == 0:  
        return 1
```

```
if x < 0:                # Explicit negative check
    return 0
if i**n > x:
    return 0
```

Mistake 10: Copy-Paste Errors

Common copy-paste error:

```
include = ways(x - i**n, n, i+1)
skip = ways(x - i**n, n, i+1)    # Copy-paste! Galat!
```

Skip call mein x same hona chahiye, x - i**n nahi.

Sahi:

```
include = ways(x - i**n, n, i+1)
skip = ways(x, n, i+1)           # x unchanged
```

Mistake 11: Wrong Starting Index

Galat:

```
def powerSum(X, N):
    return ways(X, N, 0)    # 0 se start
```

Problem:

$0^2 = 0$, jo infinite loop de sakta hai. Aur 0 include karne ka koi fayda nahi.

Sahi:

```
def powerSum(X, N):  
    return ways(X, N, 1)    # 1 se start
```

Mistake 12: Printing Instead of Returning

Galat:

```
if x == 0:  
    print("Found")  
    return    # Returns None!
```

Sahi:

```
if x == 0:  
    return 1    # Integer return
```

Checklist: Avoid Mistakes

Check	Question
✓	Base case $x==0$ pehle check hora hai?
✓	Base case $i**n > x$ hai?
✓	i hamesha $i+1$ ho raha hai?
✓	n change nahi ho raha?
✓	Return integer ho raha hai?

Check	Question
✓	Default argument mutable to nahi?
✓	Starting index 1 hai?

Chapter 15: Interview Questions (50 Questions)

Conceptual Questions (1-10)

Q1: “Sum of Nth Powers” problem kya hai? Simple words mein samjhao.

Answer Humein count karna hai ki kitne tareeke hain jisse hum unique natural numbers ki Nth powers (jaise squares, cubes) ko jod kar exactly X bana sakte hain. Har number sirf ek baar use kar sakte hain.

Q2: Hum is problem mein loops use kyun nahi kar sakte efficiently?

Answer Kyunki humein nahi pata ki kitne numbers ki zaroorat padegi. Kabhi 2 numbers se sum ban jata hai, kabhi 7 se. Fixed nested loops unknown depth ke liye kaam nahi karte. Recursion naturally variable depth handle karta hai.

Q3: Recursion aur Backtracking mein kya difference hai?

Answer Recursion: Function khud ko call karta hai. Backtracking: Try karo, agar fail to undo karo, doosra option try karo. Backtracking usually recursion use karta hai, lekin har recursion backtracking nahi hai. Is problem mein backtracking ho rahi hai kyunki hum include/skip try kar rahe hain aur fail hone par wapas aa rahe hain.

Q4: Base case $x == 0$ par 1 kyun return karte hain, True nahi?

Answer Hum count kar rahe hain number of ways. Har valid way ek count hai. 1 return karne se caller function mein `include + skip` integer addition properly kaam karta hai. True return karne se semantically galat hai, chahe Python mein $\text{True}=1$ ho.

Q5: Parameter i ko $i+1$ kyun karte hain, i same kyun nahi rehne dete?

Answer Taaki: (1) Unique numbers use ho (ek number dobara consider na ho), (2) Infinite recursion na ho. Agar i same rahega to function baar-baar same number consider karega aur kabhi base case hit nahi karega.

Q6: Is problem ki time complexity kya hai? Samjhao.

Answer Worst case: $O(2^M)$ jahan $M = \text{floor}(X^{(1/N)})$ = number of valid integers. Example: $X=100$, $N=2$, $M=10$, worst case ~ 1024 nodes. Lekin pruning ($i^n > x$) ki wajah se actual nodes bahut kam hote hain. Tree kaafi chhota rehta hai.

Q7: Agar hum same number ko multiple times use kar sakte to kya change hota?

Answer Phir $i+1$ ki jagah i use karte (same number repeat allowed). Ye "unbounded knapsack" type problem ho jaati. Code mein change: `ways(x-i^n, n, i)` aur `ways(x, n, i+1)`.

Q8: Decision tree mein pruning kya hai? Is problem mein kahan use hota hai?

Answer Pruning = unnecessary branches ko explore karne se pehle hi kaat dena. Is problem mein: `if i**n > x: return 0` pruning hai. Kyunki agar current power remaining sum se zyada hai, to aage ke sab numbers ki power aur zyada hogi. Koi solution nahi milega.

Q9: Kya hum DP (Dynamic Programming) se is problem ko optimize kar sakte hain?

Answer Haan, memoization se. `(x, i)` pair ke results cache kar sakte hain. Lekin constraints ($X \leq 1000$, $N \geq 2$) ke liye recursion without memoization bhi fast enough hai. DP advanced topic hai, pehle recursion mastery zaroori hai.

Q10: Call stack kya hai? Recursion mein iski kya bhoomika hai?

Answer Call stack memory mein ek data structure hai jo function calls ko track karta hai. Har recursive call stack mein push hoti hai, return hone par pop. Is problem mein maximum stack depth $\approx X^{(1/N)}$ hogi, jo safe hai.

Output Prediction Questions (11-20)

Q11: Output batao

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    return ways(x-i**n, n, i+1) + ways(x, n, i+1)  
  
print(ways(1, 2, 1))
```

Answer: $1 \cdot 1^2 = 1$, x becomes 0, return 1. Only one way: [1].

Q12: Output batao

```
print(ways(2, 2, 1))
```

Answer: $0 \cdot 1^2 = 1 \rightarrow$ remaining 1, next $2^2 = 4 > 1$. Skip 1 $\rightarrow 2^2 = 4 > 2$. No way.

Q13: Output batao

```
print(ways(4, 2, 1))
```

Answer: 1 Only $2^2=4$. ($1^2+?$ need 3, not a square)

Q14: Output batao

```
print(ways(5, 2, 1))
```

Answer: 1 $1^2+2^2=1+4=5$. One way.

Q15: Output batao

```
print(ways(9, 2, 1))
```

Answer: 1 $3^2=9$. ($1^2+2^2+?=5+?$ need $4=2^2$ but 2 already used. Actually $1^2+2^2+?=5$, need 4 but $2^2=4$ would be duplicate if already used? Check: $1^2+2^2=5$, need 4, but next would be $3^2=9>4$. So only $3^2=9$.)

Q16: Output batao

```
print(ways(8, 3, 1))
```

Answer: 1 $2^3=8$. One way.

Q17: Output batao

```
def f(x, n, i):
    if x == 0: return 1
    if i**n > x: return 0
    return f(x, n, i+1)

print(f(10, 2, 1))
```

Answer: 0 Ye function sirf skip karta hai, include nahi. Kabhi kuch sum nahi banega (jab tak $x=0$ start na ho).

Q18: Output batao

```
def f(x, n, i):  
    if x == 0: return 1  
    if i**n > x: return 0  
    return f(x - i**n, n, i+1)  
  
print(f(5, 2, 1))
```

Answer: 1 Sirf include karta hai, skip nahi. $1^2+2^2=5 \rightarrow$ success. Agar combination skip karna zaroori hota to galat answer deta.

Q19: Output batao

```
def ways(x, n, i):  
    if x == 0: return 1  
    if x < 0: return 0  
    if i**n > x: return 0  
    return ways(x-i**n, n, i+1) + ways(x, n, i+1)  
  
print(ways(25, 2, 1))
```

Answer: 2 Ways: $3^2+4^2=25$, $5^2=25$. Two ways.

Q20: Output batao

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    return ways(x-i**n, n, i) + ways(x, n, i+1)  
  
print(ways(4, 2, 1))
```

Answer: Infinite recursion / Stack overflow `ways(x-i\*\*n, n, i)` mein i same rehta hai, kabhi base case hit nahi hoga agar $x \neq 0$. Infinite loop.

Debugging Questions (21-30)

Q21: Bug find karo

```
def powerSum(X, N):  
    def helper(x, n, i):  
        if i**n > x:  
            return 0  
        if x == 0:  
            return 1  
        return helper(x-i**n, n, i+1) + helper(x, n, i+1)  
    return helper(X, N, 0)
```

Answer Do bugs: 1. Base case order galat - `i\*\*n > x` pehle check ho raha, `x==0` baad mein. `helper(0,2,100)` galat answer dega. 2. Starting index 0 - $0^n = 0$, fayda nahi, aur recursion issues create kar sakta hai. 1 se start karo.

Q22: Ye code kya galat hai?

```
def ways(x, n, i=1):  
    if x == 0:  
        return True  
    if i**n > x:  
        return False  
    return ways(x-i**n, n, i+1) + ways(x, n, i+1)
```

Answer Return type mismatch. `True + False` Python mein 1 deta hai (True=1, False=0), lekin semantically galat hai. Integers return karo: 1 aur 0.

Q23: Bug find karo

```
def ways(x, n, i):  
    if x == 0:  
        return 1  
    if i**n > x:  
        return 0  
    include = ways(x - i**n, n, i+n)
```

```
skip = ways(x, n, i+1)
return include + skip
```

Answer `i+n` galat hai. Agar n=2: 1,3,5,7... skip ho jayenge 2,4,6... Numbers miss honge. `i+1` hona chahiye.

Q24: Recursion kabhi khatam kyun nahi hoti?

```
def ways(x, n, i):
    if x == 0:
        return 1
    include = ways(x - i*n, n, i+1)
    skip = ways(x, n, i)
    return include + skip
```

Answer Skip call mein `i` same hai, increment nahi ho raha. Isse infinite recursion hogi skip branch mein. `ways(x, n, i+1)` hona chahiye.

Q25: Output predict karo aur bug batao

```
def ways(x, n, i=1, memo={}):
    if x == 0: return 1
    if i*n > x: return 0
    if (x,i) in memo: return memo[(x,i)]
    memo[(x,i)] = ways(x-i*n,n,i+1) + ways(x,n,i+1)
    return memo[(x,i)]

print(ways(10,2))
print(ways(100,2))
```

Answer Pehla call sahi dega (1). Doosra call galat de sakta hai kyunki `memo={}` mutable default argument hai - pehli call ka data doosri call mein bhi rahega! Har call ke liye fresh memo chahiye.

Coding Questions (31-45)

Q31: Power sum function likho jo recursive ho.

```
def powerSum(X, N):  
    def helper(rem, n, start):  
        if rem == 0:  
            return 1  
        if start**n > rem:  
            return 0  
        return helper(rem-start**n, n, start+1) + helper(rem, n, start+1)  
    return helper(X, N, 1)
```

Q32: Saare valid combinations print karne wala function likho.

```
def printAllCombos(X, N):  
    def backtrack(rem, n, start, combo):  
        if rem == 0:  
            print(combo)  
            return  
        if start**n > rem:  
            return  
        combo.append(start)  
        backtrack(rem-start**n, n, start+1, combo)  
        combo.pop()  
        backtrack(rem, n, start+1, combo)  
    backtrack(X, N, 1, [])
```

Q33: Aise combinations count karo jahan exactly K numbers use ho.

```
def countWithK(X, N, K):  
    def helper(rem, n, start, count):  
        if rem == 0 and count == K:  
            return 1  
        if rem < 0 or count > K:  
            return 0  
        if start**n > rem:  
            return 0  
        return (helper(rem-start**n, n, start+1, count+1) +
```

```

        helper(rem, n, start+1, count))
    return helper(X, N, 1, 0)

```

Q34: Minimum kitne numbers chahiye X banane ke liye?

```

def minNumbers(X, N):
    def helper(rem, n, start):
        if rem == 0:
            return 0
        if start**n > rem:
            return float('inf')
        include = 1 + helper(rem-start**n, n, start+1)
        skip = helper(rem, n, start+1)
        return min(include, skip)
    ans = helper(X, N, 1)
    return ans if ans != float('inf') else -1

```

Q35: Check karo ki kya X ko represent kar sakte hain (boolean return).

```

def canRepresent(X, N):
    def helper(rem, n, start):
        if rem == 0:
            return True
        if start**n > rem:
            return False
        return helper(rem-start**n, n, start+1) or helper(rem, n, start+1)
    return helper(X, N, 1)

```

Logic Questions (46-50)

Q46: Agar N=2 aur X=1000 ho to maximum recursion depth kitni hogi?

Answer Maximum $i = \text{floor}(\text{sqrt}(1000)) \approx 31$. So maximum depth ≈ 31 calls. Bahut safe hai (Python default recursion limit ~ 1000).

Q47: Is problem mein duplicate combinations count kyun nahi hote?

Answer Kyunki hum numbers ko strictly increasing order mein consider karte hain (i , $i+1$, $i+2$, ...). Agar hum 1^2+3^2 combination lenge to 1 pehle, phir 3. 3^2+1^2 kabhi consider nahi hoga kyunki 3 ke baad hum 4,5,... dekhenge, 1 nahi. Isliye har combination exactly ek baar count hota hai.

Q48: Kya ye problem subset sum problem jaisi hai? Similarities/Differences batao.

Answer Haan, exactly subset sum problem hai! Numbers ki jagah unki Nth powers ka set hai. Difference: Yahan numbers dynamic hain (i^n calculate karte hain) bajay predefined set ke. Pruning bhi natural hai ($i^n > x$).

Q49: Kya hum greedy approach se solve kar sakte hain? Kyun/Kyun nahi?

Answer Nahi. Greedy (sabse bada pehle lo) kaam nahi karega. Example: $X=10$, $N=2$. Greedy: $3^2=9$, remaining $1=1^2$. Ye to kaam kar gaya. Lekin $X=18$: Greedy $4^2=16$, remaining 2, no square. Lekin solution hai: 3^2+3^2 ? No, unique. Actually $18 = 3^2+?$ No. 18 has no solution. Another example: $X=12$, greedy $3^2=9$, remaining 3 (no square). But 2^2+2^2 ? No unique. $12 = 2^2+?$ No. No solution. But subset sum jaise problems mein greedy generally fail hota hai.

Q50: Explain how pruning reduces the search space.

Answer `if  $i^{**n} > x$ : return 0` pruning hai. Ye check karta hai ki agar current number ki power remaining sum se zyada hai, to aage ke sab numbers (jo aur bade hain) ki power bhi zyada hogi. Poori branch ko bina explore kiye discard kar dete hain. Isse tree ka size drastically kam ho jata hai. Example: $X=100$, $N=2$ mein $i=11$ ke baad sab branches automatic prune ho jaati hain.

Chapter 16: Practice Problems (50 Problems)

Easy Problems (1-20)

1. **Sum of first N squares:** $1^2+2^2+\dots+N^2$ recursively nikalna
 2. **Sum of first N cubes:** $1^3+2^3+\dots+N^3$ recursively
 3. **Print numbers 1 to N** using recursion
 4. **Print numbers N to 1** using recursion
 5. **Factorial** using recursion
 6. **Power function:** a^b calculate karo recursively
 7. **Sum of digits** of a number recursively
 8. **Count digits** in a number recursively
 9. **Check if number is perfect square:** recursive approach
 10. **Print all squares $\leq X$:** recursive version
 11. **Print all cubes $\leq X$:** recursive
 12. **Is X a perfect power?:** Check if $X = a^N$ for some a
 13. **Find a such that $a^N = X$,** if exists
 14. **Count numbers whose Nth power $\leq X$**
 15. **Print first K numbers** whose Nth power $\leq X$
 16. **Sum of Nth powers** from 1 to K
 17. **Maximum a** such that $a^N \leq X$
 18. **Print valid base numbers** for given X and N
 19. **Check if X can be written as sum of two squares** (not necessarily unique)
 20. **Count ways to write X as sum of TWO unique squares**
-

Medium Problems (21-40)

21. **Count ways to represent X as sum of unique squares** (N=2 fixed)
22. **Count ways to represent X as sum of unique cubes** (N=3 fixed)

23. **Print all ways to represent X as sum of unique squares**
 24. **Print all ways to represent X as sum of unique cubes**
 25. **Find the combination with minimum count of numbers**
 26. **Find the combination with maximum count of numbers**
 27. **Check if there exists a combination with exactly K numbers**
 28. **Count combinations with exactly K numbers**
 29. **Find the combination with the smallest largest number**
 30. **Find the combination with the largest smallest number**
 31. **Sum of Nth powers - memoized version** (with caching)
 32. **Check if X can be represented using only odd numbers' powers**
 33. **Check if X can be represented using only even numbers' powers**
 34. **Count ways using numbers starting from M** (not necessarily 1)
 35. **Check if representation exists without using number K**
 36. **Count ways without using number K**
 37. **Find if X can be represented as sum of exactly TWO Nth powers**
 38. **Count combinations where sum of base numbers is also X** (sum of bases = X)
 39. **Return the lexicographically smallest combination** (as list of numbers)
 40. **Return the lexicographically largest combination**
-

Hard Problems (41-50)

41. **Power Sum with bounded K:** Count ways using AT MOST K numbers
 42. **Power Sum with repetition allowed:** Each number can be used multiple times
 43. **Power Sum with both squares and cubes:** Numbers can be raised to EITHER 2 or 3
 44. **Find X that can be represented in maximum ways** for given N and range
 45. **Power sum for multiple targets:** Given list of X values, solve for each
 46. **Minimum X that can be represented in exactly W ways** for given N
 47. **Power sum with constraints:** Some numbers are forbidden
 48. **Count ways modulo M** (for large results)
 49. **Power sum DP solution:** Bottom-up approach
 50. **Generalized power sum:** Given custom set of numbers, count subset sum equal to X
-

Solutions for Practice Problems (Selected Few)

Problem 19: Check if X can be written as sum of two squares

```
def sum_of_two_squares(x):  
    for a in range(1, int(x**0.5) + 1):  
        b_sq = x - a*a  
        b = int(b_sq**0.5)  
        if b*b == b_sq and b != a: # b!=a for uniqueness  
            return True, (a, b)  
    return False, None
```

Problem 25: Minimum count combination

```
def min_count_combo(X, N):  
    def helper(rem, n, start):  
        if rem == 0:  
            return 0 # 0 more numbers needed  
        if start**n > rem:  
            return float('inf')  
        include = 1 + helper(rem - start**n, n, start + 1)  
        skip = helper(rem, n, start + 1)  
        return min(include, skip)  
  
    ans = helper(X, N, 1)  
    return ans if ans != float('inf') else -1
```

Problem 41: At most K numbers

```
def count_at_most_K(X, N, K):  
    def helper(rem, n, start, count):  
        if rem == 0:  
            return 1  
        if count >= K:  
            return 0  
        if start**n > rem:  
            return 0  
        include = helper(rem - start**n, n, start + 1, count + 1)  
        skip = helper(rem, n, start + 1, count)
```



```
    return include + skip
return helper(X, N, 1, 0)
```

Problem 49: DP Solution

```
def power_sum_dp(X, N):
    # Generate all valid powers
    powers = []
    i = 1
    while i**N <= X:
        powers.append(i**N)
        i += 1

    # DP table: dp[i][j] = ways to make sum j using first i powers
    dp = [[0]*(X+1) for _ in range(len(powers)+1)]
    dp[0][0] = 1

    for i in range(1, len(powers)+1):
        for j in range(X+1):
            # Skip current power
            dp[i][j] = dp[i-1][j]
            # Include current power (if possible)
            if j >= powers[i-1]:
                dp[i][j] += dp[i-1][j - powers[i-1]]

    return dp[len(powers)][X]
```

Chapter 17: Cheat Sheet (One Page Summary)

The Power Sum Problem - Complete Cheat Sheet

Problem Statement

Count ways to represent X as sum of **unique** Nth powers of natural numbers.

Function Template

```
def ways(remaining, power, current_num):
    # BASE CASES (order matters!)
    if remaining == 0:          # ✓ Found a way
        return 1
    if current_num**power > remaining: # X Can't proceed
        return 0

    # RECURSIVE CASES
    include = ways(remaining - current_num**power, power, current_num + 1)
    skip = ways(remaining, power, current_num + 1)

    return include + skip
```

Key Rules

Rule	Explanation
i+1 always	For uniqueness, never i, i+n, i+2
n constant	Power never changes
Return integers	1 for success, 0 for failure
Base case order	$x==0$ BEFORE $i^n > x$
Start from i=1	Not 0 ($0^n=0$ causes issues)

Common Mistakes

- ✗ return True/False instead of 1/0
- ✗ i not incrementing (infinite recursion)
- ✗ Wrong base case order
- ✗ Mutable default arguments `def func(lst=[])`

- ❌ Missing $i^n > x$ check

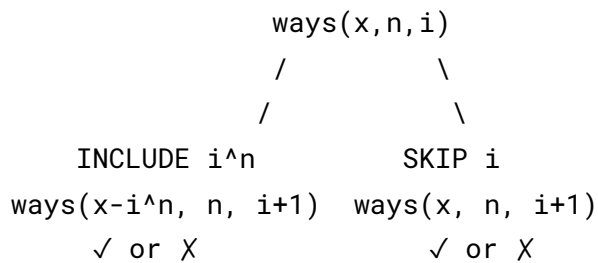
Complexity

- **Time:** $O(2^M)$ worst, where $M = \lfloor X^{(1/N)} \rfloor$ (pruning makes it much smaller)
- **Space:** $O(M)$ for recursion stack

Memory Tricks

- **"IPS"** Rule: **I**ncrement, **P**ower-constant, **S**tart-from-1
- **Base Case order:** **S**uccess before **F**ailure (SBF)
- **Two choices:** **I**nclude or **S**kip (I-S)

Decision Tree Pattern



Backtracking Template (for printing combinations)

```

def backtrack(rem, n, i, combo):
    if rem == 0: print(combo); return
    if i**n > rem: return
    combo.append(i)                # CHOOSE
    backtrack(rem - i**n, n, i+1, combo) # EXPLORE
    combo.pop()                    # UNCHOOSE
    backtrack(rem, n, i+1, combo)    # EXPLORE OTHER

```

Sample Input/Output

X	N	Ways	Combinations
10	2	1	1^2+3^2
13	2	1	2^2+3^2
25	2	2	$3^2+4^2, 5^2$
100	2	3	$6^2+8^2, 1^2+3^2+4^2+5^2+7^2, 10^2$
100	3	1	$1^3+2^3+3^3+4^3$

Quick Debug Checklist

- [] Is $x==0$ checked first?
- [] Is $i*n > x$ checked?
- [] Is i incremented by 1?
- [] Is n unchanged?
- [] Return type is integer?
- [] Starting $i = 1$?

When to Use What

- **Counting ways:** Simple recursion (no list needed)
- **Printing combinations:** Backtracking with list
- **Finding one solution:** Return True/False, stop on first
- **Optimization (min/max):** Track best, compare at leaves

Congratulations!

Aapne **Sum of Nth Powers of Unique Natural Numbers** problem ko zero se hero level tak seekh liya!

Aap ab kar sakte hain:

- Problem ko real-life examples se samajhna
- Recursion tree draw karna
- Dry run karna
- Code likhna
- Debug karna
- Interview questions answer karna
- Similar problems solve karna

Yaad rakho: Practice makes perfect. Jitna zyada dry run karoge, utna better samajh aaega.

Happy Coding! 🚀